



CompTIA

CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.

Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates

First attempt guaranteed success.

<https://www.CertyIQ.com>

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

<https://www.certyiq.com>

Mail us on - certyiqofficial@gmail.com



Lifetime Free Updates

We provide lifetime free updates to our customers. To make life easier for our valued customers and fulfill their needs



Free Exam PDF

You are sure to pass the exam completely free of charge



Money Back Guarantee

We Provide 100% money back guarantee to our customer in case of any failure

John

October 19, 2022



Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022



Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago



Customer support is really fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022



Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study. Thank You certyiq team!

Henry Rome

2 months ago



These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago



Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's. Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today -3 September with 942 score.

Databricks

(Certified Generative AI Engineer Associate)

Certified Generative AI Engineer Associate

Total: **110 Questions**

Link: <https://certyiq.com/papers/databricks/certified-generative-ai-engineer-associate>

Question: 1

A Generative AI Engineer has created a RAG application to look up answers to questions about a series of fantasy novels that are being asked on the author's web forum. The fantasy novel texts are chunked and embedded into a vector store with metadata (page number, chapter number, book title), retrieved with the user's query, and provided to an LLM for response generation. The Generative AI Engineer used their intuition to pick the chunking strategy and associated configurations but now wants to more methodically choose the best values. Which TWO strategies should the Generative AI Engineer take to optimize their chunking strategy and parameters? (Choose two.)

- A. Change embedding models and compare performance.
- B. Add a classifier for user queries that predicts which book will best contain the answer. Use this to filter retrieval.
- C. Choose an appropriate evaluation metric (such as recall or NDCG) and experiment with changes in the chunking strategy, such as splitting chunks by paragraphs or chapters. Choose the strategy that gives the best performance metric.
- D. Pass known questions and best answers to an LLM and instruct the LLM to provide the best token count. Use a summary statistic (mean, median, etc.) of the best token counts to choose chunk size.
- E. Create an LLM-as-a-judge metric to evaluate how well previous questions are answered by the most appropriate chunk. Optimize the chunking parameters based upon the values of the metric.

Answer: CE

Explanation:

The correct answer is C and E. Here's why:

C. Choose an appropriate evaluation metric (such as recall or NDCG) and experiment with changes in the chunking strategy, such as splitting chunks by paragraphs or chapters. Choose the strategy that gives the best performance metric.

This strategy focuses on empirical evaluation, which is critical for optimizing RAG pipelines. Chunking is crucial for retrieval accuracy and relevance. By selecting suitable metrics like Recall (ensuring relevant chunks are retrieved) or Normalized Discounted Cumulative Gain (NDCG) (which considers the ranking of retrieved chunks), the engineer can systematically test different chunking strategies (paragraph, chapter, sentence, token-based, etc.) and sizes. Measuring the performance of the RAG system with different configurations allows for a data-driven decision on the optimal chunking approach. This is superior to intuition-based choices.

E. Create an LLM-as-a-judge metric to evaluate how well previous questions are answered by the most appropriate chunk. Optimize the chunking parameters based upon the values of the metric.

Using an LLM as a judge is a powerful method to evaluate the quality of the RAG system. This involves providing the LLM with a question and the retrieved chunk and instructing it to assess the relevance and completeness of the chunk in answering the question. By assigning a score (the metric) to each question-chunk pair, the engineer can quantify the effectiveness of the chunking strategy. Iteratively optimizing the chunking parameters based on the LLM-as-a-judge scores leads to a chunking strategy that maximizes the information quality and relevance of the retrieved context, ultimately improving the quality of the LLM-generated responses. This approach leverages the LLM's understanding of language and context to provide nuanced feedback on chunking quality.

Why other options are incorrect:

A. Change embedding models and compare performance. While changing embedding models can improve performance, it's a separate optimization step from optimizing chunking. Chunking affects the content within a chunk that the embedding model operates on. Optimizing chunking first isolates the effect of chunking on retrieval.

B. Add a classifier for user queries that predicts which book will best contain the answer. Use this to filter

retrieval. While helpful, this is orthogonal to chunking strategy. It is a filtering mechanism, which is applied after the documents have been chunked. This improves retrieval efficiency, not necessarily chunk quality.

D. Pass known questions and best answers to an LLM and instruct the LLM to provide the best token count.

Use a summary statistic (mean, median, etc.) of the best token counts to choose chunk size. This method has flaws. The token count an LLM suggests based on ideal answers might not be the best chunk size for retrieval. Chunk size is also about information density and cohesion, which is not something just answering a question from the chunk can measure. Here are some authoritative links for further research:

RAG overview: <https://www.promptingguide.ai/techniques/rag>

Chunking Strategies: https://python.langchain.com/docs/modules/data_connection/document_transformers/

Evaluating RAG systems: <https://www.pinecone.io/learn/vector-database-rag/>

Question: 2

CertyIQ

A Generative AI Engineer is designing a RAG application for answering user questions on technical regulations as they learn a new sport.

What are the steps needed to build this RAG application and deploy it?

A. Ingest documents from a source → Index the documents and save to Vector Search → User submits queries against an LLM → LLM retrieves relevant documents → Evaluate model → LLM generates a response → Deploy it using Model Serving

B. Ingest documents from a source → Index the documents and save to Vector Search → User submits queries against an LLM → LLM retrieves relevant documents → LLM generates a response → Evaluate model → Deploy it using Model Serving

C. Ingest documents from a source → Index the documents and save to Vector Search → Evaluate model → Deploy it using Model Serving

D. User submits queries against an LLM → Ingest documents from a source → Index the documents and save to Vector Search → LLM retrieves relevant documents → LLM generates a response → Evaluate model → Deploy it using Model Serving

Answer: B

Explanation:

The correct answer is B because it outlines the standard workflow for building and deploying a Retrieval Augmented Generation (RAG) application. Let's break down why:

Ingest Documents: The process starts with ingesting the technical regulations documents. This is the foundation of the knowledge base.

Index and Vector Search: Next, the documents are indexed and saved into a vector search database. This transforms the documents into vector embeddings, enabling semantic search. Vector databases like ChromaDB, Pinecone, and Weaviate are commonly used for this.

User Query: A user submits a question to the system.

Retrieve Relevant Documents: Based on the user's query, the vector database retrieves the most relevant documents based on semantic similarity.

Generate Response: The retrieved documents are then fed into a Large Language Model (LLM) like OpenAI's GPT or a Databricks Foundation Model, along with the user's query. The LLM leverages its understanding and the context from the retrieved documents to generate a comprehensive response.

Evaluate Model: The generated responses need to be evaluated for accuracy, relevance, and coherence. This step is critical for iteratively improving the RAG application's performance. This could include metrics like precision, recall, and F1-score.

Deploy using Model Serving: Finally, the RAG application, including the LLM and vector database connection, is deployed using a model serving platform like Databricks Model Serving to make it accessible to users.

Option A is incorrect because the LLM doesn't directly retrieve documents from the vector search. The vector database does this based on the query. Option C is missing critical steps like user query and LLM response

generation. Option D has the steps in the wrong order; you can't query before ingesting and indexing the data.

Key Cloud Computing Concepts:

Vector Embeddings: Representing text as numerical vectors for semantic search.

Vector Databases: Specialized databases for storing and querying vector embeddings.

Large Language Models (LLMs): Powerful models trained on vast datasets, capable of understanding and generating human-like text.

Model Serving: Deploying machine learning models as APIs for real-time inference.

RAG Architecture: A framework for augmenting LLMs with external knowledge bases.

Authoritative Links for Further Research:

Databricks Model Serving: <https://www.databricks.com/product/machine-learning/model-serving>

RAG (Retrieval-Augmented Generation): <https://arxiv.org/abs/2005.11401>

Vector Databases Overview: <https://www.pinecone.io/learn/vector-database/>

Question: 3

CertyIQ

A Generative AI Engineer just deployed an LLM application at a digital marketing company that assists with answering customer service inquiries.

Which metric should they monitor for their customer service LLM application in production?

- A. Number of customer inquiries processed per unit of time
- B. Energy usage per query
- C. Final perplexity scores for the training of the model
- D. HuggingFace Leaderboard values for the base LLM

Answer: A

Explanation:

A. Number of customer inquiries processed per unit of time .

For a customer service LLM application in production, what matters most operationally is:

Throughput & scalability

Can the system handle real customer demand?

Responsiveness at scale

Higher inquiries processed per unit time usually correlates with better user experience and lower wait times.

This is a core production KPI for customer support systems.

Question: 4

CertyIQ

A Generative AI Engineer is building a Generative AI system that suggests the best matched employee team member to newly scoped projects. The team member is selected from a very large team. The match should be based upon project date availability and how well their employee profile matches the project scope. Both the employee profile and project scope are unstructured text.

How should the Generative AI Engineer architect their system?

- A. Create a tool for finding available team members given project dates. Embed all project scopes into a vector

store, perform a retrieval using team member profiles to find the best team member.

B.Create a tool for finding team member availability given project dates, and another tool that uses an LLM to extract keywords from project scopes. Iterate through available team members' profiles and perform keyword matching to find the best available team member.

C.Create a tool to find available team members given project dates. Create a second tool that can calculate a similarity score for a combination of team member profile and the project scope. Iterate through the team members and rank by best score to select a team member.

D.Create a tool for finding available team members given project dates. Embed team profiles into a vector store and use the project scope and filtering to perform retrieval to find the available best matched team members.

Answer: D

Explanation:

Create a tool for finding available team members given project dates. Embed team profiles into a vector store and use the project scope and filtering to perform retrieval to find the available best matched team members.

This problem has two different types of constraints:

Hard constraint → availability by project dates

Soft constraint → semantic match between unstructured text (project scope ↔ employee profile)

Option D handles both efficiently and at scale:

Availability tool

Filters the very large team down to only those available for the project dates.

Vector store of employee profiles

Embeddings capture semantic meaning in unstructured profiles.

Project scope as the query + metadata filtering

Retrieves only available candidates.

Ranks them by semantic similarity to find the best match.

This is a scalable, production-grade retrieval design and aligns with modern RAG-style architectures.

Question: 5

CertyIQ

A Generative AI Engineer is designing an LLM-powered live sports commentary platform. The platform provides real-time updates and LLM-generated analyses for any users who would like to have live summaries, rather than reading a series of potentially outdated news articles.

Which tool below will give the platform access to real-time data for generating game analyses based on the latest game scores?

A.DatabricksIQ

B.Foundation Model APIs

C.Feature Serving

D.AutoML

Answer: C

Explanation:

The correct answer is C, Feature Serving. Here's why:

Feature Serving is the process of providing precomputed features to machine learning models, often in real-time or near real-time. In the context of the live sports commentary platform, real-time game scores and related statistics are considered features. Feature Serving allows the LLM to access this up-to-date data without needing to fetch it directly from source databases or APIs on every request, which would be inefficient and introduce latency. A feature store acts as a centralized repository to manage these features, ensuring consistency and serving them with low latency.

DatabricksIQ (A) is an AI-powered assistant within the Databricks environment, designed to help users with tasks like code generation and debugging but it doesn't directly provide real-time data access. Foundation Model APIs (B) are used to access and interact with pre-trained language models, but they don't inherently handle real-time data ingestion or serving. AutoML (D) automates the process of building machine learning models, including feature engineering and hyperparameter tuning, but it's not the primary tool for delivering real-time features to an operational LLM-powered application.

The live sports commentary platform needs the latest game scores instantly. Feature Serving ensures that the LLM receives these "features" in a timely manner, enabling it to generate relevant and up-to-date game analyses. Feature stores such as Feast or Databricks Feature Store are typically used for this purpose, enabling low-latency access to fresh data.

Further resources:

Databricks Feature Store: <https://docs.databricks.com/machine-learning/feature-store/index.html>

Feast Feature Store: <https://feast.dev/>

Question: 6

CertyIQ

A Generative AI Engineer has a provisioned throughput model serving endpoint as part of a RAG application and would like to monitor the serving endpoint's incoming requests and outgoing responses. The current approach is to include a micro-service in between the endpoint and the user interface to write logs to a remote server. Which Databricks feature should they use instead which will perform the same task?

- A.Vector Search
- B.Lakeview
- C.DBSQL
- D.Inference Tables

Answer: D

Explanation:

The most suitable Databricks feature for monitoring a serving endpoint's incoming requests and outgoing responses in a RAG application is **Inference Tables**.

Here's why: Inference Tables provide a managed, scalable, and cost-effective solution for observing the inputs and outputs of deployed models. Instead of introducing an external micro-service, Inference Tables automatically capture request and response data without modifying the serving endpoint's code. This integrated approach eliminates the overhead of maintaining a separate logging service.

Vector Search is irrelevant, focusing instead on similarity search of embeddings, not monitoring. Lakeview is a Databricks dashboarding tool, allowing visualization of existing data, not real-time logging and capturing inference data. DBSQL is geared towards data warehousing and querying large datasets, and does not natively offer monitoring capabilities for model serving endpoints.

Inference Tables directly address the monitoring need. They provide detailed visibility into model

performance and usage patterns. Captured data can be utilized for debugging, performance analysis, and model improvement, all within the Databricks environment. This eliminates the need for custom logging solutions and simplifies the monitoring process. Using Inference Tables facilitates the identification of unexpected inputs, biased outputs, or performance bottlenecks. All of this is accomplished with minimal code changes and reduced operational overhead. By centralizing inference data, Inference Tables improve observability of production AI systems.

Reference links:

[Databricks Inference Tables](#)

[Monitor online models with Inference Tables](#)

Question: 7

CertyIQ

A Generative AI Engineer is tasked with improving the RAG quality by addressing its inflammatory outputs. Which action would be most effective in mitigating the problem of offensive text outputs?

- A. Increase the frequency of upstream data updates
- B. Inform the user of the expected RAG behavior
- C. Restrict access to the data sources to a limited number of users
- D. Curate upstream data properly that includes manual review before it is fed into the RAG system

Answer: D

Explanation:

The most effective action to mitigate inflammatory outputs from a RAG (Retrieval-Augmented Generation) system is to curate the upstream data properly, including manual review before it is fed into the system. This addresses the root cause of the problem: offensive content within the data the RAG system is using. Option D directly tackles this issue.

Here's why the other options are less effective:

A. Increase the frequency of upstream data updates: While keeping data fresh is generally good practice, it doesn't inherently address the presence of offensive content. More frequent updates of bad data simply result in more frequent generation of offensive outputs.

B. Inform the user of the expected RAG behavior: This is a helpful step for managing user expectations but does not solve the underlying problem of the RAG system generating offensive text. It's a workaround, not a fix.

C. Restrict access to the data sources to a limited number of users: Limiting access may improve security, but it doesn't prevent the RAG system from generating offensive content if that content already exists in the data it uses.

Data curation, including manual review, allows for the identification and removal or modification of offensive or biased content before it is incorporated into the RAG system's knowledge base. This process involves careful examination of data sources, identifying potential biases, removing inappropriate content, and ensuring the data aligns with ethical and safety guidelines. Manual review provides a crucial layer of human oversight, which is particularly important when dealing with sensitive or potentially harmful content. Automating parts of data cleaning and bias detection is possible using tools like those offered by AWS AI services or Google Cloud AI Platform but requires careful design and validation to ensure effective bias mitigation and accurate removal of offensive language. Ultimately, preventing the RAG system from accessing offensive data is the most proactive and effective method for preventing offensive outputs.

Relevant resources for further learning:

Question: 8

CertyIQ

A Generative AI Engineer is creating an LLM-based application. The documents for its retriever have been chunked to a maximum of 512 tokens each. The Generative AI Engineer knows that cost and latency are more important than quality for this application. They have several context length levels to choose from.

Which will fulfill their need?

- A. context length 514; smallest model is 0.44GB and embedding dimension 768
- B. context length 2048; smallest model is 11GB and embedding dimension 2560
- C. context length 32768; smallest model is 14GB and embedding dimension 4096
- D. context length 512; smallest model is 0.13GB and embedding dimension 384

Answer: D

Explanation:

The optimal choice for the Generative AI Engineer is context length 512 (Option D). Since cost and latency are prioritized over quality, a smaller model with a context length matching the chunk size is most suitable.

Here's why:

1. **Cost Optimization:** Smaller models inherently require fewer computational resources. This translates to lower costs for model hosting, inference, and training (if fine-tuning is involved).
2. **Latency Reduction:** Smaller models have fewer parameters to process, leading to faster inference times. Reduced latency provides a better user experience, crucial when speed is a priority.
3. **Matching Chunk Size:** The documents are chunked to 512 tokens. Using a context length of 512 means the entire chunk can be processed directly without truncation or needing to consider longer dependencies, potentially speeding up processing.
4. **Model Size and Embedding Dimension:** Option D has the smallest model size (0.13GB) and a reasonable embedding dimension (384). Larger context lengths like 2048 or 32768 (Options B and C) would require significantly larger models (11GB and 14GB respectively), increasing cost and latency unnecessarily since the chunks are only 512 tokens. The higher embedding dimensions of B and C may give more detailed understanding for same tokens, however, the performance increase is outweighed by their significantly greater cost and latency. Option A (context length 514, model size 0.44GB) is larger and only minimally better than D in context length. Thus, option D is the best choice overall.

In summary, prioritizing cost and latency necessitates selecting the smallest model with a context length that aligns with the existing chunk size. Option D best fulfills these requirements, while other options introduce unnecessary overhead.

For further research on LLMs, consider exploring the following:

Hugging Face Transformers Library: <https://huggingface.co/transformers/>

TensorFlow: <https://www.tensorflow.org/>

PyTorch: <https://pytorch.org/>

LangChain: <https://www.langchain.com/>

Question: 9

CertyIQ

A small and cost-conscious startup in the cancer research field wants to build a RAG application using Foundation Model APIs.

Which strategy would allow the startup to build a good-quality RAG application while being cost-conscious and able to cater to customer needs?

- A. Limit the number of relevant documents available for the RAG application to retrieve from
- B. Pick a smaller LLM that is domain-specific
- C. Limit the number of queries a customer can send per day
- D. Use the largest LLM possible because that gives the best performance for any general queries

Answer: B

Explanation:

Here's a detailed justification for why option B is the best strategy for a cost-conscious startup building a RAG application for cancer research, using Foundation Model APIs:

Option B: Pick a smaller LLM that is domain-specific

Cost-effectiveness: Larger LLMs generally consume significantly more computational resources (GPU time, memory) during inference, leading to higher API costs. Smaller, domain-specific models are trained on targeted datasets, making them more efficient for specific tasks. This directly translates to lower API usage costs.

Performance: For specialized domains like cancer research, a model pre-trained or fine-tuned on relevant scientific literature, clinical trial data, and medical reports is likely to provide more accurate and relevant answers than a large, general-purpose LLM. While large models might have broad knowledge, they lack the depth of understanding necessary for nuanced queries in a specific domain.

Reduced latency: Smaller models typically have lower inference latency, providing a faster response time to user queries. This is crucial for a good user experience.

Resource requirements: Smaller LLMs demand less memory and computational power during both training and inference, saving costs related to infrastructure.

Trade-off consideration: The primary trade-off is that a smaller, domain-specific model might not perform well on general knowledge queries outside its training domain. However, given the startup's focus on cancer research, this trade-off is acceptable.

Why the other options are less suitable:

Option A: Limit the number of relevant documents available: While limiting document retrieval can save costs on retrieval and potentially on context window usage for the LLM, it sacrifices the quality of answers. The RAG application might miss crucial information, leading to inaccurate or incomplete responses.

Option C: Limit the number of queries a customer can send per day: This addresses cost concerns directly, but at the expense of customer satisfaction and usability. This approach restricts the service and reduces its usefulness.

Option D: Use the largest LLM possible: This is the most expensive option and is contrary to the startup's cost-conscious objective. While large models might seem appealing, they are often overkill for specialized tasks and can be inefficient in terms of both cost and latency.

In summary, Choosing a smaller, domain-specific LLM provides the best balance of cost-effectiveness, performance, and user experience for a startup focused on cancer research. It optimizes for the specific domain without compromising on essential functionality.

Supporting Links:

Retrieval-Augmented Generation (RAG): <https://www.pinecone.io/learn/retrieval-augmented-generation/>
Choosing the Right LLM: <https://www.anyscale.com/blog/choosing-the-right-open-llm-for-your-application>

Question: 10

A Generative AI Engineer is responsible for developing a chatbot to enable their company's internal HelpDesk Call Center team to more quickly find related tickets and provide resolution. While creating the GenAI application work breakdown tasks for this project, they realize they need to start planning which data sources (either Unity Catalog volume or Delta table) they could choose for this application. They have collected several candidate data sources for consideration: **call_rep_history**: a Delta table with primary keys `representative_id`, `call_id`. This table is maintained to calculate representatives' call resolution from fields `call_duration` and `call_start_time`. **transcript Volume**: a Unity Catalog Volume of all recordings as `*.wav` files, but also a text transcript as `*.txt` files. **call_cust_history**: a Delta table with primary keys `customer_id`, `call_id`. This table is maintained to calculate how much internal customers use the HelpDesk to make sure that the charge back model is consistent with actual service use. **call_detail**: a Delta table that includes a snapshot of all call details updated hourly. It includes `root_cause` and `resolution` fields, but those fields may be empty for calls that are still active. **maintenance_schedule** – a Delta table that includes a listing of both HelpDesk application outages as well as planned upcoming maintenance downtimes. They need sources that could add context to best identify ticket root cause and resolution. Which TWO sources do that? (Choose two.)

- A. `call_cust_history`
- B. `maintenance_schedule`
- C. `call_rep_history`
- D. `call_detail`
- E. `transcript Volume`

Answer: DE

Explanation:

The correct answer is D and E. Here's why:

D. `call_detail`: This Delta table includes `root_cause` and `resolution` fields, which are precisely the information needed to identify the ticket's underlying issue and its solution. Although these fields might be empty for active calls, the accumulated historical data for completed calls provides valuable context for new tickets. The hourly updates ensure relatively recent information is available.

E. `transcript Volume`: The volume contains `.txt` files, meaning it holds text transcripts of the call recordings. These transcripts will contain the specific details of the customer's issue, the troubleshooting steps taken, and the ultimate resolution (if achieved). Analyzing this text data can help identify patterns, keywords, and phrases associated with specific root causes and resolutions. Processing this data will require additional work but will unlock critical context.

Let's examine why the other options are less suitable:

A. `call_cust_history`: This table focuses on customer usage and chargebacks, not the root cause or resolution of issues. It provides insights into customer behavior but not the technical aspects needed for the chatbot.

B. `maintenance_schedule`: While knowing about outages can be useful, it only provides a very narrow context. The primary goal is to identify root causes beyond scheduled maintenance.

C. `call_rep_history`: This table focuses on representative performance metrics, which, while important for management, doesn't directly contribute to identifying the root cause and resolution of tickets.

In the context of a GenAI application for a HelpDesk, the `call_detail` table provides structured data on root causes and resolutions, while the `transcript Volume` offers unstructured but highly valuable textual data containing the conversation details. The combination of these two sources provides a rich data foundation for the chatbot to effectively assist the call center team.

Further Research:

Delta Lake: <https://delta.io/>

Unity Catalog Volumes: <https://docs.databricks.com/en/unity-catalog/volume.html>

Question: 11

What is the most suitable library for building a multi-step LLM-based workflow?

- A. Pandas
- B. TensorFlow
- C. PySpark
- D. LangChain

Answer: D

Explanation:

D is correct because LangChain is specifically designed for creating multi-step workflows with language models, enabling smooth chaining and integration of LLMs with various data sources and tools.

LangChain's architecture supports modular design, allowing developers to easily sequence multiple LLM calls, manage input and output effectively, and implement complex logic within generative AI tasks. It also provides components for prompt management, memory, and serialization of workflows, crucial for sophisticated applications in natural language processing.

Evaluation of Other Options:

A. Pandas: While powerful for data manipulation and analysis, Pandas does not natively support building workflows around LLMs. It lacks the functionality for integrating LLM components and managing complex workflows involving multiple steps and interactions.

B. TensorFlow: This library is predominantly focused on deep learning and model training. Although it can be used to implement language models, it does not facilitate the user-friendly creation of multi-step workflows and lacks the specific tooling for chaining LLMs together like LangChain provides.

C. PySpark: Primarily aimed at big data processing through distributed computing, PySpark does not inherently support the functionality required to build multi-step LLM workflows. It can handle large datasets effectively-computationally, but lacks the specialized abstractions and integrations with language models necessary for smooth LLM operations.

In summary, LangChain's unique features tailored for LLM workflows make it the most appropriate choice compared to the other options.

References:

<https://python.langchain.com/>

<https://towardsdatascience.com/what-is-langchain-and-how-does-it-work-b7707e49cebf>

<https://www.analyticsvidhya.com/blog/2023/05/getting-started-with-langchain-a-comprehensive-guide/>

Question: 12

When developing an LLM application, it's crucial to ensure that the data used for training the model complies with licensing requirements to avoid legal risks.

Which action is NOT appropriate to avoid legal risks?

- A. Reach out to the data curators directly before you have started using the trained model to let them know.
- B. Use any available data you personally created which is completely original and you can decide what license to use.

C. Only use data explicitly labeled with an open license and ensure the license terms are followed.

D. Reach out to the data curators directly after you have started using the trained model to let them know.

Answer: D

Explanation:

The correct answer is D because it represents a reactive approach to legal compliance that is inherently risky in the context of LLM training data.

Here's why:

Proactive Compliance is Key: Legal compliance, especially with licensing, demands a proactive approach. Waiting until after the model is trained and used to contact data curators places the developer in a potentially precarious position. If the data usage violates the license, significant rework, legal action, or model takedown could be necessary.

Intellectual Property Rights: Data used to train LLMs often carries intellectual property rights. Using data without properly understanding and adhering to its license is a violation of these rights.

"A" is Correct: Reaching out before using the model (A) allows for negotiation of terms or even identifying better data sources upfront. This is a crucial step in responsible AI development.

"B" is Correct: Using your own original data (B) eliminates the need for external licensing considerations, as you own the copyright and can license it as you see fit. This is the safest approach.

"C" is Correct: Strictly adhering to open licenses (C) provides a clear and established framework for responsible data usage. The license terms are defined and, if followed, minimize legal risk. This provides a structured means for compliance.

Timing is Critical: Contacting data curators after the fact doesn't mitigate the risk of past infringement. You've already potentially violated the license.

Costly Rework: If a licensing issue is discovered late, retraining the model with compliant data can be incredibly expensive and time-consuming.

In summary, responsible LLM development requires understanding and respecting data licenses before any data is used for training. Contacting data curators after the model is trained is akin to driving without a license and then asking for permission; it doesn't undo any prior violations.

Supporting Links:

Creative Commons (Open Licensing): creativecommons.org

Stanford's AI Index Report (Discussing Data Availability and Licensing): <https://aiindex.stanford.edu/>

Model Cards for Model Reporting: <https://modelcards.withgoogle.com/>

Question: 13

CertyIQ

A Generative AI Engineer is testing a simple prompt template in LangChain using the code below, but is getting an error.


```

from langchain.chains import LLMChain
from langchain_community.llms import OpenAI
from langchain_core.prompts import PromptTemplate

prompt_template = "Tell me a {adjective} joke"

prompt = PromptTemplate(
    input_variables=["adjective"],
    template=prompt_template
)

llm = LLMChain(prompt=prompt)
llm.generate([{"adjective": "funny"}])

```

Assuming the API key was properly defined, what change does the Generative AI Engineer need to make to fix their chain?

- A.
- ```

prompt_template = "Tell me a {adjective} joke"

prompt = PromptTemplate(
 input_variables=["adjective"],
 template=prompt_template
)

llm = LLMChain(prompt=prompt)
llm.generate("funny")

prompt_template = "Tell me a {adjective} joke"

prompt = PromptTemplate(
 input_variables=["adjective"],
 template=prompt_template
)

llm = LLMChain(prompt=prompt.format("funny"))
llm.generate()

```
- B.

```
prompt_template = "Tell me a {adjective} joke"
```

```
prompt = PromptTemplate(
 input_variables=["adjective"],
 template=prompt_template
 llm=OpenAI()
)
```

```
llm = LLMChain(prompt=prompt)
llm.generate([{"adjective": "funny"}])
```

C.

```
prompt_template = "Tell me a {adjective} joke"
```

```
prompt = PromptTemplate(
 input_variables=["adjective"],
 template=prompt_template
)
```

```
llm = LLMChain(llm=OpenAI(), prompt=prompt)
llm.generate([{"adjective": "funny"}])
```

D.

**Answer: D**

**Explanation:**

```
from langchain.chains import LLMChain

from langchain_community.llms import OpenAI

from langchain_core.prompts import PromptTemplate

prompt_template = "Tell me a adjective joke"

prompt = PromptTemplate(

 input_variables=["adjective"], template=prompt_template
)

llm = LLMChain(llm=OpenAI(), prompt=prompt)
```

#### Question: 14

CertyIQ

A Generative AI Engineer is creating an LLM system that will retrieve news articles from the year 1918 and related to a user's query and summarize them. The engineer has noticed that the summaries are generated well but often also include an explanation of how the summary was generated, which is undesirable.

Which change could the Generative AI Engineer perform to mitigate this issue?



- A.Split the LLM output by newline characters to truncate away the summarization explanation.
- B.Tune the chunk size of news articles or experiment with different embedding models.
- C.Revisit their document ingestion logic, ensuring that the news articles are being ingested properly.
- D.Provide few shot examples of desired output format to the system and/or user prompt.

**Answer: D**

**Explanation:**

D is correct because providing few-shot examples establishes clear expectations for the desired output format, reducing the likelihood of extraneous explanations in the responses.

**Justification for D**

Few-shot learning effectively guides the model by presenting it with examples of the desired output structure, thereby influencing its response generation. When the model sees clear instances of what constitutes a succinct summary, it is more likely to follow that format and omit unnecessary explanations. This aligns the model's output with user expectations, enhancing relevance and usability.

**Evaluation of Other Options**

**A. Split the LLM output by newline characters to truncate away the summarization explanation.**

This approach is a workaround rather than a solution. It may unintentionally truncate valuable content or lead to incomplete information, thereby compromising the summary's quality. Moreover, it doesn't address the root cause of why the model generates explanations.

**B. Tune the chunk size of news articles or experiment with different embedding models.**

While experimenting with chunk size or embedding can improve retrieval and relevance, it does not directly target the issue of unwarranted explanations in generated summaries. Tuning these parameters might enhance overall performance but won't specifically mitigate the verbose nature of the summaries.

**C. Revisit their document ingestion logic, ensuring that the news articles are being ingested properly.**

This option focuses on the ingestion phase and does not directly relate to the summarization output. It may improve the quality of input data but does not address the model's tendency to elaborate on its reasoning during output generation, which is the primary concern.

**References**

<https://arxiv.org/abs/2005.14165>

<https://www.microsoft.com/en-us/research/project/generative-Pretrained-Transformers/>

<https://www.google.com/search/howsearchworks/generative-ai-research/>

**Question: 15**

**CertyIQ**

A Generative AI Engineer has developed an LLM application to answer questions about internal company policies. The Generative AI Engineer must ensure that the application doesn't hallucinate or leak confidential data. Which approach should NOT be used to mitigate hallucination or confidential data leakage?

- A.Add guardrails to filter outputs from the LLM before it is shown to the user
- B.Fine-tune the model on your data, hoping it will learn what is appropriate and not
- C.Limit the data available based on the user's access level
- D.Use a strong system prompt to ensure the model aligns with your needs.

**Answer: B**

### Explanation:

The correct answer is B, which suggests fine-tuning the model on your data, hoping it will learn what is appropriate and not, as a method to mitigate hallucination or confidential data leakage. This is the least effective and most unreliable approach among the options.

Here's why:

**Unpredictable Outcomes of Fine-tuning:** While fine-tuning can improve a model's performance on a specific task, it's not a guaranteed solution for preventing hallucination or data leakage. The model might still generate incorrect or misleading information, or inadvertently expose sensitive data present in the training dataset.

**Lack of Granular Control:** Fine-tuning provides a more generalized influence on the model's behavior rather than precise control over specific outputs. It's difficult to ensure that the model has effectively learned which information to withhold or how to avoid hallucination.

**Importance of Input Context (Prompt Engineering):** A strong system prompt guides the model's behavior by setting constraints and providing context. This makes the model more likely to follow instructions and avoid producing incorrect outputs. <https://www.promptingguide.ai/>

**Data Access Control is Critical:** Limiting data access ensures that the model only has access to information that the user is authorized to view. This mitigates the risk of confidential data leakage by preventing the model from even seeing restricted information.

**Guardrails for Output Filtering:** Guardrails are crucial for detecting and filtering potentially harmful or inappropriate outputs from the LLM. These can include techniques like content filtering, toxicity detection, and PII (Personally Identifiable Information) redaction.

**Auth0:** "Auth0 is an identity management platform that can be integrated with LLMs to control data access based on user roles and permissions. This ensures that users only have access to the data they are authorized to see, preventing data leakage."

In contrast, options A, C, and D provide more direct and reliable methods for controlling the model's output and protecting confidential data. Fine-tuning, while useful in other contexts, is a less effective method for addressing these specific security and reliability concerns.

Therefore, fine-tuning alone is insufficient to guarantee the prevention of hallucination or data leakage. It should be used in conjunction with other security measures, such as guardrails, data access control, and system prompts, to provide a more robust defense against these risks.

### Question: 16

CertyIQ

A Generative AI Engineer interfaces with an LLM with prompt/response behavior that has been trained on customer calls inquiring about product availability. The LLM is designed to output "In Stock" if the product is available or only the term "Out of Stock" if not.

Which prompt will work to allow the engineer to respond to call classification labels correctly?

A. Respond with "In Stock" if the customer asks for a product.

B. You will be given a customer call transcript where the customer asks about product availability. The outputs are either "In Stock" or "Out of Stock". Format the output in JSON, for example: "call\_id": "123", "label": "In Stock".

C. Respond with "Out of Stock" if the customer asks for a product.

D. You will be given a customer call transcript where the customer inquires about product availability. Respond with "In Stock" if the product is available or "Out of Stock" if not.

**Answer: D**

**Explanation:**

The correct answer is D because it directly instructs the LLM on how to process the provided customer call transcript and respond based on product availability, aligning perfectly with the LLM's training. It explicitly states the input (customer call transcript about product availability) and the expected outputs ("In Stock" or "Out of Stock") based on the availability. This aligns with the prompt engineering best practice of clearly defining the task and expected format for the LLM.

Option A and C are too simplistic. They only cover one case (product availability leading to an "In Stock" or "Out of Stock" response). The LLM needs instructions to handle both scenarios for comprehensive classification. They lack the context that a customer call transcript is provided as input.

Option B, while instructing the LLM about input and desired outputs, introduces an unnecessary complication by requesting a JSON format output containing a "call\_id" field. The problem states the LLM should directly output "In Stock" or "Out of Stock," and including unnecessary formatting constraints can hinder performance and introduce error. Adding elements not required by the underlying goal goes against prompt engineering principles focused on simplicity and efficiency. Therefore, option D offers the most effective and direct prompt, enabling the generative AI engineer to classify customer calls accurately using the pre-trained LLM.

For further research on prompt engineering best practices, you can consult resources like:

1. **Prompt Engineering Guide:** <https://www.promptingguide.ai/>
2. **OpenAI documentation on prompting:** <https://platform.openai.com/docs/guides/prompt-engineering>

**Question: 17**

**CertyIQ**

A Generative AI Engineer is tasked with developing a RAG application that will help a small internal group of experts at their company answer specific questions, augmented by an internal knowledge base. They want the best possible quality in the answers, and neither latency nor throughput is a huge concern given that the user group is small and they're willing to wait for the best answer. The topics are sensitive in nature and the data is highly confidential and so, due to regulatory requirements, none of the information is allowed to be transmitted to third parties.

Which model meets all the Generative AI Engineer's needs in this situation?

- A. Dolly 1.5B
- B. OpenAI GPT-4
- C. BGE-large
- D. Llama2-70B

**Answer: D**

**Explanation:**

Here's a detailed justification for why Llama2-70B is the most suitable model in this scenario:

The key requirements are high answer quality, data confidentiality (no third-party transmission), and tolerance for latency. The use case involves a small group of internal experts querying a sensitive internal knowledge base.

**Llama2-70B** is an open-source large language model (LLM). Being open-source, it can be deployed on-premise or within the company's private cloud environment, ensuring that no data is sent to external third parties, fulfilling the data confidentiality requirement.

With 70 billion parameters, Llama2-70B is a powerful model capable of generating high-quality, nuanced

responses. This is crucial since answer quality is prioritized.

The tolerance for latency means the relatively larger size and computational demands of Llama2-70B are not a significant constraint.

**Dolly 1.5** is also an open-source option, which satisfies the data privacy concerns, but it is significantly smaller and likely to generate lower-quality answers compared to Llama2-70B.

**OpenAI's GPT-4** offers excellent quality, but it violates the crucial requirement of no data transmission to third parties. Using OpenAI services inherently involves sending data to OpenAI.

**BGE-large** is a good embedding model, mainly for generating embeddings used in the vector store component of RAG. While important for RAG, BGE does not address the core requirement of having a strong generative model that can be used on-premise. It does not directly provide the generative capabilities required for the final answer generation.

In summary, Llama2-70B provides the best balance of high-quality responses and the crucial ability to operate entirely within the company's infrastructure, meeting all the requirements of the generative AI engineer.

#### Supporting Links:

**Llama 2:** <https://ai.meta.com/research/llama/> (Official Meta Llama 2 page)

**Retrieval Augmented Generation (RAG):** Search for this term in cloud vendor documentation (AWS, Azure, GCP) for more information on how RAG systems are implemented.

#### Question: 18

CertyIQ

A Generative AI Engineer would like an LLM to generate formatted JSON from emails. This will require parsing and extracting the following information: order ID, date, and sender email. Here's a sample email:

```
Date: April 23, 2024
Time: 4:22 PM
From: anjali.thayer@computex.org
To: cust_service@realtek.com
Subject: Shipment details
```

Hey there,

I have a shipment (order ID is CD34RFT) can you please send me an update?

Thank you,  
Anjali

They will need to write a prompt that will extract the relevant information in JSON format with the highest level of output accuracy.

Which prompt will do that?

A.You will receive customer emails and need to extract date, sender email, and order ID. You should return the date, sender email, and order ID information in JSON format.

B.You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in JSON format.

Here's an example: "date": "April 16, 2024", "sender\_email": "[email protected]", "order\_id": "RE987D"

C.You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in a human-readable format.

D.You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in JSON format.

**Answer: B**

**Explanation:**

B.You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in JSON format.

Here's an example: "date": "April 16, 2024", "sender\_email": "[email protected]", "order\_id": "RE987D"

Explicit Formatting Instruction: All options mention the desired output format (JSON).

The Power of Examples: Prompt B provides a specific, well-structured example: "date": "April 16, 2024", "sender\_email": "[email protected]", "order\_id": "RE987D".

Consistency: Providing an example is the single most effective way to ensure an AI model's output is consistent. It acts as a clear template.

Precision: The example dictates the exact key names (date, sender\_email, order\_id) and the expected data format for each value (e.g., the specific date format). Without this, the model might use variations like email\_address, customer\_email,

### Question: 19

CertyIQ

A Generative AI Engineer has been asked to build an LLM-based question-answering application. The application should take into account new documents that are frequently published. The engineer wants to build this application with the least cost and least development effort and have it operate at the lowest cost possible. Which combination of chaining components and configuration meets these requirements?

- A.For the application a prompt, a retriever, and an LLM are required. The retriever output is inserted into the prompt which is given to the LLM to generate answers.
- B.The LLM needs to be frequently with the new documents in order to provide most up-to-date answers.
- C.For the question-answering application, prompt engineering and an LLM are required to generate answers.
- D.For the application a prompt, an agent and a fine-tuned LLM are required. The agent is used by the LLM to retrieve relevant content that is inserted into the prompt which is given to the LLM to generate answers.

**Answer: A**

**Explanation:**

The best approach for building an LLM-based question-answering application with minimal cost and development effort, while ensuring it incorporates new documents, is option A: prompt, retriever, and LLM.

Here's why:

**Prompt Engineering:** A well-crafted prompt is essential to instruct the LLM on how to use the retrieved information to answer the question correctly. This is foundational.

**Retriever:** A retriever component enables the application to dynamically search and retrieve relevant snippets from the new documents without retraining the LLM. This avoids the high costs associated with continual fine-tuning or frequent LLM updates. Vector databases are often used for efficient retrieval.

**LLM:** The core component for generating answers.

**Why other options are not suitable:**

**Option B:** Continuously training the LLM with new documents is expensive and computationally intensive. It is not a practical low-cost solution for frequently updated documents.

**Option C:** Relying solely on prompt engineering and the LLM is insufficient for dynamic content updates. The LLM would have no knowledge of the new documents without a mechanism to access them.

**Option D:** Using an agent and fine-tuning is an overkill and not cost-effective. Agents introduce complexity and higher costs. Fine-tuning is only necessary if the LLM's fundamental understanding needs modification,

not for incorporating frequently changing document data.

Option A minimizes cost because it avoids continuous retraining and complex agent configurations. The retriever component dynamically fetches relevant information. The prompt directs the LLM on how to incorporate the retrieved context into a coherent answer. This architecture offers a balance between accuracy, development effort, and operational costs.

### Supporting Concepts

**Retrieval-Augmented Generation (RAG):** Option A embodies the RAG pattern, where retrieval complements the LLM's generation capabilities. <https://www.pinecone.io/learn/retrieval-augmented-generation/>

**Prompt Engineering:** Effective prompting guides LLMs. <https://www.promptingguide.ai/>

**Vector Databases:** Efficient storage and retrieval of document embeddings. <https://www.pinecone.io/>

## Question: 20

CertyIQ

A Generative AI Engineer is creating an agent-based LLM system for their favorite monster truck team. The system can answer text based questions about the monster truck team, lookup event dates via an API call, or query tables on the team's latest standings.

How could the Generative AI Engineer best design these capabilities into their system?

- A. Ingest PDF documents about the monster truck team into a vector store and query it in a RAG architecture.
- B. Write a system prompt for the agent listing available tools and bundle it into an agent system that runs a number of calls to solve a query.
- C. Instruct the LLM to respond with "RAG", "API", or "TABLE" depending on the query, then use text parsing and conditional statements to resolve the query.
- D. Build a system prompt with all possible event dates and table information in the system prompt. Use a RAG architecture to lookup generic text questions and otherwise leverage the information in the system prompt.

### Answer: B

#### Explanation:

Write a system prompt for the agent listing available tools and bundle it into an agent system that runs a number of calls to solve a query.

This system needs to handle multiple capabilities dynamically:

Answer text-based questions → LLM reasoning / RAG

Look up event dates → API calls

Query standings tables → structured data access

An agent-based architecture with tools is exactly meant for this:

The system prompt defines available tools (RAG, API, SQL/table query)

The agent:

Interprets the user's intent

Decides which tool(s) to call

Chains actions if needed (reason → act → observe → respond)

This scales cleanly as you add more tools later



### Question: 21

A Generative AI Engineer has been asked to design an LLM-based application that accomplishes the following business objective: answer employee HR questions using HR PDF documentation.

Which set of high level tasks should the Generative AI Engineer's system perform?

- A. Calculate averaged embeddings for each HR document, compare embeddings to user query to find the best document. Pass the best document with the user query into an LLM with a large context window to generate a response to the employee.
- B. Use an LLM to summarize HR documentation. Provide summaries of documentation and user query into an LLM with a large context window to generate a response to the user.
- C. Create an interaction matrix of historical employee questions and HR documentation. Use ALS to factorize the matrix and create embeddings. Calculate the embeddings of new queries and use them to find the best HR documentation. Use an LLM to generate a response to the employee question based upon the documentation retrieved.
- D. Split HR documentation into chunks and embed into a vector store. Use the employee question to retrieve best matched chunks of documentation, and use the LLM to generate a response to the employee based upon the documentation retrieved.

**Answer: D**

#### Explanation:

Here's a detailed justification for why option D is the best approach for an LLM-based HR question answering application, along with supporting concepts and links:

Option D outlines a Retrieval-Augmented Generation (RAG) architecture, which is well-suited for question answering tasks using a knowledge base (in this case, HR documentation). RAG addresses limitations of standard LLMs by grounding their responses in specific, relevant information.

1. **Chunking:** Splitting the HR documentation into smaller chunks is crucial. LLMs have context window limitations, meaning they can only process a limited amount of text at once. Breaking down large documents ensures relevant sections can fit within the context window. This avoids information loss and improves accuracy.
2. **Embedding:** Embedding each chunk into a vector store transforms textual data into numerical representations that capture semantic meaning. This allows for efficient similarity searches. Embeddings facilitate finding the sections of the documentation that are most relevant to the employee's query.
3. **Vector Store:** A vector store (like FAISS, Pinecone, or Weaviate) provides an efficient way to store and search these embeddings. When an employee asks a question, the question is also embedded. The vector store identifies the chunks of HR documentation with embeddings most similar to the question's embedding.
4. **Retrieval:** The "best matched chunks of documentation" are retrieved from the vector store. This is the "retrieval" part of RAG. These chunks represent the most relevant information to answer the employee's question.
5. **LLM Generation:** Finally, the LLM uses the retrieved chunks of documentation, along with the original employee question, to generate a coherent and contextually accurate response. The LLM essentially synthesizes an answer based on the provided knowledge. This is the "generation" aspect of RAG.

Options A, B, and C are less ideal:

**Option A:** Averaging embeddings across entire documents loses granular information. Moreover, the LLM still needs to process the entire best document, which could be too large for the context window.

**Option B:** Summarization can lead to information loss. Important details that would answer specific questions might be omitted. Providing summaries and the user query to the LLM will still be less accurate and more costly than using RAG.

**Option C:** Collaborative filtering techniques (ALS) and interaction matrices are better suited for recommendation systems, not for question answering from documents. They rely on historical interactions, which might not be available or representative of all possible questions.

In conclusion, the RAG approach (Option D) effectively uses a vector database to efficiently retrieve the most relevant documentation chunks, which are then provided to the LLM to generate the answer, making it the most appropriate solution for the given scenario.

#### Authoritative Links:

**Retrieval-Augmented Generation:** <https://arxiv.org/abs/2005.11401>

**Vector Databases:** <https://www.pinecone.io/learn/vector-database/>

**LangChain (RAG framework):** <https://www.langchain.com/>

### Question: 22

CertyIQ

Generative AI Engineer at an electronics company just deployed a RAG application for customers to ask questions about products that the company carries. However, they received feedback that the RAG response often returns information about an irrelevant product.

What can the engineer do to improve the relevance of the RAG's response?

- A. Assess the quality of the retrieved context
- B. Implement caching for frequently asked questions
- C. Use a different LLM to improve the generated response
- D. Use a different semantic similarity search algorithm

**Answer: D**

#### Explanation:

The issue described is that the RAG (Retrieval-Augmented Generation) application retrieves irrelevant context, which indicates that the retrieval mechanism is not accurately identifying the most relevant information. By using a different semantic similarity search algorithm, the engineer can improve the retrieval step to ensure that the most contextually relevant documents or data are selected before passing them to the LLM for generating responses. Options like A (assessing quality) may help diagnose the issue but do not directly fix it. B (caching) is not related to improving relevance, and C (using a different LLM) addresses generation quality, not retrieval accuracy.

### Question: 23

CertyIQ

A Generative AI Engineer is developing a chatbot designed to assist users with insurance-related queries. The chatbot is built on a large language model (LLM) and is conversational. However, to maintain the chatbot's focus and to comply with company policy, it must not provide responses to questions about politics. Instead, when presented with political inquiries, the chatbot should respond with a standard message:

"Sorry, I cannot answer that. I am a chatbot that can only answer questions around insurance."

Which framework type should be implemented to solve this?

- A. Safety Guardrail
- B. Security Guardrail
- C. Contextual Guardrail

**Answer: A****Explanation:**

The correct answer is **A. Safety Guardrail**. Here's why:

A safety guardrail in the context of generative AI focuses on preventing harmful or inappropriate outputs from the model. This directly aligns with the scenario where the chatbot needs to avoid providing responses to political inquiries, which could be considered a sensitive or potentially controversial topic. The aim is to ensure responsible AI usage by preventing the chatbot from generating potentially biased, offensive, or inaccurate political statements.

**Safety Guardrails** ensure that the AI system doesn't generate outputs that are harmful, unethical, biased, or discriminatory. In this case, preventing political discussion falls under ethical and responsible AI usage.

**Security Guardrails** relate to protecting the system and data from unauthorized access, breaches, or malicious attacks, which is not the primary concern in the given scenario.

**Contextual Guardrails** focus on adapting the model's behavior based on the specific context of the conversation. While context is important, the primary goal here is to block an entire category of topics, not fine-tune the response within a permitted area.

**Compliance Guardrails** are designed to ensure that the AI system adheres to legal and regulatory requirements. While avoiding political discussion might be part of a broader compliance strategy, the immediate need is to filter out specific types of content, which aligns with safety.

In essence, the requirement to avoid political discussion is a safety measure aimed at preventing the chatbot from engaging in potentially problematic or controversial topics, thus ensuring responsible and safe AI usage.

**Further Research:**

**AI Safety:** <https://aisafety.info/>

**Responsible AI practices:** Look for resources from organizations like Microsoft, Google, and IBM on their AI ethics and responsible AI frameworks.

**Question: 24****CertyIQ**

A Generative AI Engineer I using the code below to test setting up a vector store:

```
from databricks.vector_search.client import VectorSearchClient

vsc = VectorSearchClient()

vsc.create_endpoint(
 name= "vector_search_test",
 endpoint_type= "STANDARD"
)
```

Assuming they intend to use Databricks managed embeddings with the default embedding model, what should be the next logical function call?

- A.vsc.get\_index()
- B.vsc.create\_delta\_sync\_index()
- C.vsc.create\_direct\_access\_index()
- D.vsc.similarity\_search()

**Answer: B**

**Explanation:**

After setting up a vector store endpoint, creating an index is necessary to start populating and organizing the data. The `create_delta_sync_index()` function specifically creates an index that synchronizes with a Delta table, allowing automatic updates as the data changes. This is likely the most appropriate choice if the engineer plans to use dynamic data that is updated over time.

**Question: 25**

**CertyIQ**

A Generative AI Engineer is tasked with deploying an application that takes advantage of a custom MLflow Pyfunc model to return some interim results.

How should they configure the endpoint to pass the secrets and credentials?

- A. Use `spark.conf.set ()`
- B. Pass variables using the Databricks Feature Store API
- C. Add credentials using environment variables
- D. Pass the secrets in plain text

**Answer: C**

**Explanation:**

The correct answer is C: Add credentials using environment variables.

Here's why:

When deploying applications, especially those using MLflow models, securely managing secrets and credentials is paramount. Environment variables provide a secure and flexible way to pass sensitive information to the application without hardcoding them directly into the code. Hardcoding secrets exposes them and presents a serious security risk. Using `spark.conf.set()` (Option A) is more suitable for configuring Spark sessions and isn't the intended use case for passing sensitive credentials. The Databricks Feature Store API (Option B) focuses on managing and accessing features for machine learning models, not credential management. Passing secrets in plain text (Option D) is highly insecure and should be avoided at all costs.

Environment variables, on the other hand, can be managed securely within Databricks using Secret Scopes, ensuring that credentials are encrypted and access is controlled. The application running within the Databricks environment can then retrieve these secrets from the environment variables at runtime, enabling secure authentication and authorization to external services or resources.

Therefore, using environment variables combined with Databricks Secret Scopes is the best practice for passing secrets and credentials to a deployed application using a custom MLflow Pyfunc model because it promotes security, flexibility, and maintainability. This approach aligns with security best practices in cloud computing and ensures that sensitive information is handled responsibly.

Relevant documentation:

Databricks Secrets: <https://docs.databricks.com/security/secrets/index.html>

MLflow deployment: <https://www.mlflow.org/docs/latest/deployments/index.html>

**Question: 26**

**CertyIQ**

A Generative AI Engineer wants to build an LLM-based solution to help a restaurant improve its online customer

experience with bookings by automatically handling common customer inquiries. The goal of the solution is to minimize escalations to human intervention and phone calls while maintaining a personalized interaction. To design the solution, the Generative AI Engineer needs to define the input data to the LLM and the task it should perform. Which input/output pair will support their goal?

- A. Input: Online chat logs; Output: Group the chat logs by users, followed by summarizing each user's interactions
- B. Input: Online chat logs; Output: Buttons that represent choices for booking details
- C. Input: Customer reviews; Output: Classify review sentiment
- D. Input: Online chat logs; Output: Cancellation options

**Answer: B**

**Explanation:**

B. Input: Online chat logs; Output: Buttons that represent choices for booking details.

The goal is to:

Automatically handle common booking inquiries

Reduce phone calls and human escalation

Maintain a personalized, low-friction experience

Option B enables this by:

Using online chat logs as input to understand user intent and context

Producing structured, actionable outputs (buttons) such as:

Date

Time

Party size

Seating preference

Buttons:

Guide users through the booking flow

Reduce ambiguity and back-and-forth

Minimize errors and escalation

This is a common best practice in conversational booking systems

The LLM acts as an intent interpreter, while the UI handles structured data collection efficiently.

## Question: 27

CertyIQ

What is an effective method to preprocess prompts using custom code before sending them to an LLM?

- A. Directly modify the LLM's internal architecture to include preprocessing steps
- B. It is better not to introduce custom code to preprocess prompts as the LLM has not been trained with examples of the preprocessed prompts
- C. Rather than preprocessing prompts, it's more effective to postprocess the LLM outputs to align the outputs

to desired outcomes

D. Write a MLflow PyFunc model that has a separate function to process the prompts

**Answer: D**

**Explanation:**

The correct answer is D: Write a MLflow PyFunc model that has a separate function to process the prompts. Here's why:

This approach leverages MLflow, a popular open-source platform for managing the machine learning lifecycle, which includes model tracking, experiment management, and model deployment. MLflow's PyFunc model allows you to wrap custom Python code, including preprocessing functions, into a standardized and deployable format. By using a separate function within the PyFunc model to preprocess prompts, you gain several advantages.

First, it promotes modularity. The preprocessing logic is isolated from the LLM interaction, making it easier to maintain, test, and update. Second, it enables versioning and tracking. MLflow tracks the preprocessing function along with the model, ensuring reproducibility. Third, it offers flexibility. The preprocessing function can perform various tasks such as cleaning, formatting, augmenting, or even translating the prompt before sending it to the LLM. This is crucial for adapting prompts to specific LLM requirements or optimizing them for performance. Fourth, it ensures consistency in prompt processing across different deployments and applications. The same preprocessing logic is consistently applied wherever the MLflow model is deployed.

Option A is incorrect because directly modifying the LLM's internal architecture is often impractical, technically difficult, and potentially harmful, requiring deep expertise and access to the model's internals, which is rarely the case for most users.

Option B is incorrect because preprocessing prompts is a common and often necessary practice to improve LLM performance and ensure desired outputs. LLMs are trained on a diverse range of data, and preprocessing can help tailor prompts to specific tasks or formats.

Option C is not as effective as preprocessing. While postprocessing is also valuable for refining LLM outputs, preprocessing can prevent issues from arising in the first place by ensuring the prompt is well-formed and aligned with the LLM's capabilities. Addressing issues before they reach the LLM typically yields better results. Preprocessing and postprocessing are not mutually exclusive; both can be used in conjunction for optimal results.

Authoritative Links:

**MLflow:** <https://mlflow.org/>

**MLflow PyFunc Model:** [https://www.mlflow.org/docs/latest/python\\_api/mlflow.pyfunc.html](https://www.mlflow.org/docs/latest/python_api/mlflow.pyfunc.html)

## Question: 28

CertyIQ

A Generative AI Engineer is developing an LLM application that users can use to generate personalized birthday poems based on their names.

Which technique would be most effective in safeguarding the application, given the potential for malicious user inputs?

- A. Implement a safety filter that detects any harmful inputs and ask the LLM to respond that it is unable to assist
- B. Reduce the time that the users can interact with the LLM
- C. Ask the LLM to remind the user that the input is malicious but continue the conversation with the user
- D. Increase the amount of compute that powers the LLM to process input faster



**Answer: A**

**Explanation:**

The most effective technique for safeguarding the LLM application against malicious user inputs when generating personalized birthday poems is option A: Implementing a safety filter that detects harmful inputs and instructing the LLM to respond that it's unable to assist. This approach directly addresses the risk of prompt injection and adversarial attacks.

Here's why:

1. **Prompt Injection Mitigation:** Malicious users may attempt to manipulate the LLM's behavior through crafted inputs. A safety filter acts as a gatekeeper, analyzing the input for keywords, patterns, or sentiments associated with harmful content like hate speech, profanity, or attempts to bypass safety protocols.
2. **Content Moderation:** Safety filters serve as a form of content moderation, preventing the LLM from generating responses that violate ethical guidelines or legal regulations. This protects the application and its users from offensive or harmful output.
3. **Controlled Response:** Rather than attempting to engage with malicious content, the LLM is instructed to provide a neutral response, avoiding escalation or generation of potentially harmful text. This is a safer and more responsible approach.
4. **No Direct Engagement:** The LLM does not interact or engage with potentially dangerous content, reducing the risk of the LLM being used to generate harmful text, provide instructions for illegal activities, or provide harmful advice.
5. **Reduced Attack Surface:** By rejecting malicious requests, the application limits the attacker's ability to probe for vulnerabilities or manipulate the LLM's behavior. The attack surface is therefore reduced.
6. Options B, C, and D are not suitable as they do not directly address harmful inputs or can lead to unwanted or unsafe conversations.

Option B: Reducing interaction time doesn't prevent malicious content generation if a harmful prompt is processed in the brief interaction window.

Option C: Asking the LLM to point out malicious input and continue is counterproductive and unsafe because the LLM is processing the user's harmful content to provide a response. This will potentially lead to unwanted outputs.

Option D: Increasing compute does not mitigate the threat of malicious inputs. It simply speeds up processing, regardless of the content's safety.

Therefore, implementing a robust safety filter is the most effective method to protect the LLM application from malicious user input by rejecting harmful requests and preventing the generation of inappropriate content.

**Supporting Links:**

OWASP Prompt Injection: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>  
Microsoft's Responsible AI Practices: <https://www.microsoft.com/en-us/ai/responsible-ai>

**Question: 29**

**CertyIQ**

Which indicator should be considered to evaluate the safety of the LLM outputs when qualitatively assessing LLM responses for a translation use case?

- A.The ability to generate responses in code
- B.The similarity to the previous language
- C.The latency of the response and the length of text generated
- D.The accuracy and relevance of the responses

**Answer: D**

**Explanation:**

The correct answer is **D. The accuracy and relevance of the responses.**

When qualitatively assessing LLM outputs for a translation use case, safety isn't solely about preventing harmful or biased content (though that's important). It's also about ensuring the translated output accurately reflects the meaning and intent of the original text and is relevant to the context. A safe translation minimizes misinterpretations and ensures clear communication.

Option A, generating responses in code, is irrelevant in a translation use case. It's more applicable in scenarios involving code generation or software development. Option B, similarity to the previous language, is a poor indicator of safety or quality. A good translation prioritizes meaning, not verbatim similarity, which could lead to awkward or incorrect phrasing. Option C, latency and text length, are performance metrics, not indicators of the safety or correctness of the translation. A fast but inaccurate translation is more dangerous than a slower but accurate one.

Accuracy is paramount because an inaccurate translation can lead to misunderstandings, potentially with severe consequences depending on the context (e.g., medical instructions, legal documents). Relevance ensures the translation maintains the appropriate tone and style for the target audience. For example, a formal document requires a different translation style than a casual conversation. Therefore, to consider the safety of the LLM output in this use case, it is more valuable to test accuracy and relevance. Testing accuracy and relevance means assessing the overall outcome of the LLM.

Therefore, focusing on accuracy and relevance ensures that the translated output is both faithful to the original text and appropriate for its intended use.

**Question: 30**

**CertyIQ**

A Generative AI Engineer is developing a patient-facing healthcare-focused chatbot. If the patient's question is not a medical emergency, the chatbot should solicit more information from the patient to pass to the doctor's office and suggest a few relevant pre-approved medical articles for reading. If the patient's question is urgent, direct the patient to calling their local emergency services.

Given the following user input:

"I have been experiencing severe headaches and dizziness for the past two days."

Which response is most appropriate for the chatbot to generate?

- A.Here are a few relevant articles for your browsing. Let me know if you have questions after reading them.
- B.Please call your local emergency services.
- C.Headaches can be tough. Hope you feel better soon!
- D.Please provide your age, recent activities, and any other symptoms you have noticed along with your headaches and dizziness.

**Answer: B**

**Explanation:**

B. Please call your local emergency services.

The symptoms described — severe headaches and dizziness lasting two days — can indicate a potential medical emergency.

The chatbot's design explicitly states:

Urgent → direct the patient to emergency services

Non-urgent → ask follow-up questions + suggest articles

Given the risk involved, the safest and most responsible action is to escalate immediately, not gather more information or provide reading material.

### Question: 31

CertyIQ

After changing the response generating LLM in a RAG pipeline from GPT-4 to a model with a shorter context length that the company self-hosts, the Generative AI Engineer is getting the following error:

```
{"error_code": "BAD_REQUEST", "message": "Bad request: rpc error: code = InvalidArgument desc = prompt token count (4595) cannot exceed 4096..."}
```

What TWO solutions should the Generative AI Engineer implement without changing the response generating model? (Choose two.)

- A. Use a smaller embedding model to generate embeddings
- B. Reduce the maximum output tokens of the new model
- C. Decrease the chunk size of embedded documents
- D. Reduce the number of records retrieved from the vector database
- E. Retrain the response generating model using ALiBi

**Answer: CD**

**Explanation:**

C: You can reduce the number of tokens included in the prompt by reducing the chunk size of the embedded document.

D: Reducing the number of records retrieved from the vector database can solve the token overflow issue by reducing the size of the final prompt.

### Question: 32

CertyIQ

A Generative AI Engineer is building a system which will answer questions on latest stock news articles. Which will NOT help with ensuring the outputs are relevant to financial news?

- A. Implement a comprehensive guardrail framework that includes policies for content filters tailored to the finance sector.
- B. Increase the compute to improve processing speed of questions to allow greater relevancy analysis
- C. Implement a profanity filter to screen out offensive language.
- D. Incorporate manual reviews to correct any problematic outputs prior to sending to the users

**Answer: B**

**Explanation:**

The correct answer is B because increasing compute power primarily addresses processing speed, not the relevancy of the generated content to financial news. While faster processing is beneficial, it doesn't inherently ensure the AI model focuses on financial topics. Options A, C, and D directly contribute to relevance:

**A (Implement a comprehensive guardrail framework):** Guardrails, specifically tailored content filters for the finance sector, actively restrict the model's output to align with financial terminology, topics, and reporting standards. This ensures the AI provides information within the desired domain.

**C (Implement a profanity filter):** While a profanity filter doesn't directly ensure financial relevance, it improves the overall quality and professionalism of the output, indirectly contributing to a more credible and trustworthy response related to financial news. A professional tone is expected in financial reporting.

**D (Incorporate manual reviews):** Manual reviews introduce human oversight to identify and correct outputs that stray from financial news relevance. This allows for fine-tuning of the system's performance based on real-world examples and ensures the answers align with the expected content.

Increasing compute, on the other hand, might enable the model to process more data or utilize more complex algorithms, but without specific guidance or filtering mechanisms, it won't automatically focus on financial news. The model could still generate irrelevant outputs, just at a faster rate. Relevancy in Generative AI relies on a combination of data, filtering, and post-generation checks.

#### Supporting Links:

**Guardrails for AI:** <https://www.promptflow.ai/blog/robust-rag-llm-application-with-guardrails>

**AI Content Filtering:** (Search relevant publications on content moderation and AI safety from sources such as arXiv, Google Scholar, or academic journals).

### Question: 33

CertyIQ

A Generative AI Engineer is building a RAG application that answers questions about internal documents for the company SnoPen AI.

The source documents may contain a significant amount of irrelevant content, such as advertisements, sports news, or entertainment news, or content about other companies.

Which approach is advisable when building a RAG application to achieve this goal of filtering irrelevant information?

- A. Keep all articles because the RAG application needs to understand non-company content to avoid answering questions about them.
- B. Include in the system prompt that any information it sees will be about SnoPenAI, even if no data filtering is performed.
- C. Include in the system prompt that the application is not supposed to answer any questions unrelated to SnoPen AI.
- D. Consolidate all SnoPen AI related documents into a single chunk in the vector database.

**Answer: C**

**Explanation:**

**C. Include in the system prompt that the application is not supposed to answer any questions unrelated to SnoPen AI.**

Here's why:

**System Prompt Guidance:** System prompts are crucial for guiding the LLM's behavior. By explicitly stating the application's focus on SnoPen AI, you instruct the model to prioritize information related to the company

and disregard other content. This is a form of prompt engineering that leverages the LLM's inherent understanding of context and instructions.

**Focus on Relevance:** The primary goal is to ensure the RAG application answers questions accurately and relevantly. While understanding irrelevant content is important, the application should ideally avoid generating responses based on it. Option C directly addresses this.

**Option A's Drawbacks:** Keeping all articles, as suggested in Option A, increases the noise in the vector database. This can degrade the performance of the retrieval process and lead to irrelevant or inaccurate answers. The model's context window is limited, and including irrelevant information consumes valuable space.

**Option B's Limitations:** While including information about SnoPen AI in the system prompt might help to some extent, it does not explicitly tell the model to ignore unrelated information. A direct instruction, as in option C, is a better approach.

**Option D's Inefficiency:** Consolidating all documents into a single chunk, as in option D, is not advisable. This defeats the purpose of a vector database, which thrives on smaller, semantically meaningful chunks. The RAG application's retrieval process relies on identifying relevant chunks, which becomes impossible with a single consolidated document. Moreover, the chunk size would likely exceed the LLM's context window.

Therefore, instructing the model to disregard unrelated content in the system prompt offers a balance between relevance and efficiency. It guides the model to prioritize SnoPen AI-related information without overloading the vector database with irrelevant content.

#### Authoritative Links for further research:

Prompt Engineering Guide: <https://www.promptingguide.ai/>

RAG architecture on Azure: <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/retrieval-augmented-generation>

### Question: 34

CertyIQ

A Generative AI Engineer has successfully ingested unstructured documents and chunked them by document sections. They would like to store the chunks in a Vector Search index. The current format of the dataframe has two columns: (i) original document file name (ii) an array of text chunks for each document. What is the most performant way to store this dataframe?

- A. Split the data into train and test set, create a unique identifier for each document, then save to a Delta table
- B. Flatten the dataframe to one chunk per row, create a unique identifier for each row, and save to a Delta table
- C. First create a unique identifier for each document, then save to a Delta table
- D. Store each chunk as an independent JSON file in Unity Catalog Volume. For each JSON file, the key is the document section name and the value is the array of text chunks for that section

**Answer: B**

#### Explanation:

Here's a detailed justification for why option B ("Flatten the dataframe to one chunk per row, create a unique identifier for each row, and save to a Delta table") is the most performant approach for storing the chunked document data in a vector search index, along with why the other options are less suitable:

#### Justification for Option B:

Option B promotes a normalized, row-oriented structure optimized for vector search indexing. Here's why this leads to the best performance:

1. **Scalability and Parallelism:** Flattening the DataFrame allows for maximum parallelism during indexing. Each row (representing a single chunk) can be processed independently and concurrently by the vector search engine. Delta Lake, with its partitioning and data skipping features, can further enhance this parallelism, as chunks from different documents can be indexed simultaneously.
2. **Efficient Vector Embedding and Storage:** Most vector search indexes work most efficiently with a one-to-one mapping between a document (in this case, a chunk) and its vector embedding. Option B directly facilitates this. Each row can contain the chunk text, a unique ID, and the computed vector embedding, simplifying the indexing process.
3. **Query Optimization:** Having each chunk as a separate row allows the vector search engine to retrieve the most relevant chunks based on vector similarity without needing to parse nested arrays or JSON structures. This significantly speeds up query execution. Delta Lake's data skipping and z-ordering capabilities can further optimize vector search queries by rapidly filtering out irrelevant chunks.
4. **Unique Identification:** Creating a unique identifier for each chunk ensures that each data point in the index is distinct and addressable. This is crucial for update/delete operations and for tracing the source of a given result back to its original document.
5. **Delta Lake Benefits:** Storing the flattened data in a Delta table brings all the advantages of Delta Lake: ACID transactions, schema evolution, data versioning, and optimized query performance. These are essential for maintaining data quality and managing the vector search index effectively.

#### Why other options are less suitable:

**Option A (Split data into train and test, unique document ID, Delta table):** Train/test splits are relevant for machine learning model evaluation, but irrelevant at this stage. The focus is optimal storage for vector search, and train/test splitting doesn't directly address that. It also only creates a unique ID for each document, not each chunk, which creates the same problem as Option C.

**Option C (Unique document ID, Delta table):** While a unique document ID is helpful, this option doesn't flatten the data. The array of chunks for each document will create problems when trying to insert into the vector database. The vector database will ideally require the entire dataset to be a collection of text chunks.

**Option D (JSON files in Unity Catalog Volume):** While Unity Catalog provides governance, storing each chunk as an independent JSON file introduces significant overhead in terms of file management, I/O operations, and query performance. Vector search engines are designed to work with structured data formats, and JSON file access is far less efficient than querying a Delta table. Also, managing a large number of small JSON files can quickly become a scalability bottleneck.

#### Authoritative Links:

**Databricks Vector Search:** <https://www.databricks.com/product/lakehouse-ai/vector-search>

**Delta Lake:** <https://delta.io/>

In summary, flattening the DataFrame and storing it in a Delta table with unique identifiers for each chunk (Option B) provides the most performant and scalable solution for indexing and querying unstructured document data with vector search.

#### Question: 35

CertyIQ

A Generative AI Engineer has created a RAG application which can help employees retrieve answers from an internal knowledge base, such as Confluence pages or Google Drive. The prototype application is now working with some positive feedback from internal company testers. Now the Generative AI Engineer wants to formally evaluate the system's performance and understand where to focus their efforts to further improve the system. How should the Generative AI Engineer evaluate the system?



- A. Use cosine similarity score to comprehensively evaluate the quality of the final generated answers.
- B. Curate a dataset that can test the retrieval and generation components of the system separately. Use MLflow's built in evaluation metrics to perform the evaluation on the retrieval and generation components.
- C. Benchmark multiple LLMs with the same data and pick the best LLM for the job.
- D. Use an LLM-as-a-judge to evaluate the quality of the final answers generated.

**Answer: B**

**Explanation:**

The correct answer is B. Here's why:

Evaluating a RAG (Retrieval-Augmented Generation) system requires a nuanced approach that addresses both its retrieval and generation capabilities. Option B provides the most structured and comprehensive methodology for this evaluation.

**Why Option B is correct:**

**Separate Evaluation:** RAG systems consist of two key parts: the retrieval component (finding relevant documents) and the generation component (creating an answer based on those documents). Evaluating them separately allows for targeted improvements. If the system fails, you can pinpoint whether it's because of poor retrieval, a weak generation model, or both.

**Curated Dataset:** A well-designed evaluation dataset is crucial. This dataset should contain questions, the expected relevant documents (for retrieval evaluation), and the ideal answers (for generation evaluation). This dataset acts as a "ground truth."

**MLflow Integration:** MLflow is a popular open-source platform for managing the ML lifecycle, including experiment tracking, model management, and model deployment. MLflow provides built-in metrics for both retrieval and generation tasks.

**Retrieval Metrics:** MLflow can use metrics like Precision@K (what percentage of the top K retrieved documents are relevant?), Recall@K (what percentage of the relevant documents are retrieved in the top K?), and Mean Reciprocal Rank (MRR) to evaluate the retrieval component. These metrics directly assess how well the system can find the right documents to answer a question.

**Generation Metrics:** For generation, MLflow can leverage metrics like ROUGE, BLEU, METEOR, or even more advanced LLM-based evaluation, to determine the quality, coherence, and accuracy of the generated answer in comparison to the ground truth answer.

**Why other options are not as good:**

**A. Use cosine similarity score to comprehensively evaluate the quality of the final generated answers.**

Cosine similarity is useful to compare text, but on its own it is not an effective measure of accuracy and appropriateness for the particular query.

**C. Benchmark multiple LLMs with the same data and pick the best LLM for the job.** While important, this is just one aspect. Without assessing the retrieval component, you can't know if a better LLM is truly improving the entire RAG system. Focus only on LLMs neglects the retrieval aspect.

**D. Use an LLM-as-a-judge to evaluate the quality of the final answers generated.** LLM-as-a-judge (using a powerful LLM to evaluate the output of another LLM) is a valid technique for evaluating the generation component. However, it doesn't address the performance of the retrieval component, which is vital for a RAG system. Using an LLM as a judge on its own neglects the retrieval aspect.

**In Summary:** Option B offers a more thorough, component-specific, and measurable evaluation strategy using industry-standard tools, making it the optimal approach for evaluating a RAG system.

## Authoritative Links:

**MLflow:** <https://www.mlflow.org/>

**RAG Systems:** (Research papers and articles available on Google Scholar about RAG evaluation metrics would provide further authoritative details).

**Evaluation Metrics for Text Generation:** <https://towardsdatascience.com/evaluating-text-output-in-nlp-bleu-at-your-service-e4954b909335>

## Question: 36

CertyIQ

A Generative AI Engineer has already trained an LLM on Databricks and it is now ready to be deployed. Which of the following steps correctly outlines the easiest process for deploying a model on Databricks?

- A. Log the model as a pickle object, upload the object to Unity Catalog Volume, register it to Unity Catalog using MLflow, and start a serving endpoint
- B. Log the model using MLflow during training, directly register the model to Unity Catalog using the MLflow API, and start a serving endpoint
- C. Save the model along with its dependencies in a local directory, build the Docker image, and run the Docker container
- D. Wrap the LLM's prediction function into a Flask application and serve using Gunicorn

**Answer: B**

### Explanation:

The correct answer is B because it leverages MLflow's capabilities for model management and deployment within the Databricks environment, ensuring a seamless and efficient workflow.

Here's a detailed justification:

Option B, "Log the model using MLflow during training, directly register the model to Unity Catalog using the MLflow API, and start a serving endpoint," represents the most streamlined and Databricks-native approach for deploying a trained LLM. MLflow is specifically designed for tracking, managing, and deploying machine learning models, deeply integrated within the Databricks ecosystem.

1. **MLflow Logging:** Logging the model using MLflow during training captures all necessary artifacts, parameters, and dependencies, creating a reproducible and deployable model version.
2. **Unity Catalog Integration:** Registering the model directly to Unity Catalog using the MLflow API provides centralized governance, versioning, and access control for the model. Unity Catalog simplifies collaboration and ensures model discoverability.
3. **Serving Endpoint:** Finally, starting a serving endpoint directly from the registered model in Unity Catalog enables real-time inference. Databricks provides managed serving endpoints that handle scaling, monitoring, and other operational aspects.

In contrast, other options present less efficient or less integrated approaches:

**Option A:** Pickling and manually uploading to Unity Catalog is cumbersome and bypasses MLflow's tracking and metadata management features.

**Option C:** Building and managing Docker images manually adds complexity and overhead.

**Option D:** Using Flask and Gunicorn introduces more manual configuration and management compared to Databricks' managed serving endpoints.

By using MLflow's logging, registration with Unity Catalog, and native serving capabilities, engineers can minimize the operational overhead and focus on optimizing the model's performance and business value.

### Supporting Links:

**MLflow Documentation:** <https://mlflow.org/>

**Databricks Model Serving:** <https://www.databricks.com/product/machine-learning/model-serving>

**Unity Catalog:** <https://www.databricks.com/product/unity-catalog>

### Question: 37

CertyIQ

A Generative AI Engineer developed an LLM application using the provisioned throughput Foundation Model API. Now that the application is ready to be deployed, they realize their volume of requests are not sufficiently high enough to create their own provisioned throughput endpoint. They want to choose a strategy that ensures the best cost-effectiveness for their application.

What strategy should the Generative AI Engineer use?

- A. Switch to using External Models instead
- B. Deploy the model using pay-per-token throughput as it comes with cost guarantees
- C. Change to a model with a fewer number of parameters in order to reduce hardware constraint issues
- D. Throttle the incoming batch of requests manually to avoid rate limiting issues

**Answer: B**

#### Explanation:

The correct answer is B: Deploy the model using pay-per-token throughput as it comes with cost guarantees.

Here's why: The scenario describes a situation where a provisioned throughput endpoint is too expensive because the request volume is low. Provisioned throughput implies a commitment to a certain level of compute resources, which is only cost-effective with high utilization.

Pay-per-token throughput, on the other hand, directly ties cost to usage. The engineer only pays for the actual tokens processed, making it ideal for applications with variable or low request volumes. This approach avoids the idle costs associated with underutilized provisioned endpoints. Option A, switching to External Models, might introduce integration complexities and dependencies on external services, potentially increasing latency and reducing control. Option C, changing to a model with fewer parameters, impacts model quality and may not be necessary if the goal is cost-effectiveness with the current model's capabilities. Option D, manually throttling requests, addresses rate limiting but doesn't inherently optimize costs; it might simply delay requests and not reduce the total cost of processing them. Pay-per-token offers a direct cost guarantee tied to the application's actual usage, making it the most economically sound choice.

### Question: 38

CertyIQ

A Generative AI Engineer is building an LLM to generate article summaries in the form of a type of poem, such as a haiku, given the article content. However, the initial output from the LLM does not match the desired tone or style. Which approach will NOT improve the LLM's response to achieve the desired response?

- A. Provide the LLM with a prompt that explicitly instructs it to generate text in the desired tone and style
- B. Use a neutralizer to normalize the tone and style of the underlying documents
- C. Include few-shot examples in the prompt to the LLM
- D. Fine-tune the LLM on a dataset of desired tone and style

**Answer: B**

#### Explanation:

The correct answer is B. Here's why:

The goal is to improve the LLM's ability to generate article summaries in a specific poetic style. Options A, C, and D all directly address how the LLM generates its output.

**A (Providing explicit instructions):** Prompt engineering is a fundamental technique in GenAI. Clearly defining the desired tone, style (e.g., "Generate a haiku"), and format in the prompt directly influences the LLM's output. This leverages the model's understanding of language and style when given clear guidance.

**C (Few-shot examples):** Few-shot learning demonstrates the desired style by providing example input-output pairs in the prompt. The LLM learns from these examples to better emulate the desired style in its generated summaries. This provides a direct demonstration of what's expected.

**D (Fine-tuning):** Fine-tuning involves training the LLM on a dataset of text in the desired style (e.g., a collection of haikus). This adapts the model's weights to better reflect the characteristics of the target style, resulting in improved generation quality. This method is more intensive, but yields significant style improvements if the fine-tuning dataset is well-constructed.

Option B, however, suggests using a "neutralizer" on the underlying documents (the articles being summarized). A neutralizer aims to remove stylistic elements from the input. This will work against the ultimate goal, which is to create summaries exhibiting a specific style. It is counterintuitive because if the input is neutralized, the LLM will have to produce the style entirely from scratch. It won't be able to draw inspiration or vocabulary that might be useful in creating the summary in the desired poetic style. Instead, if anything is done to the underlying documents, it should be to extract or highlight features most conducive to generating summaries in the target style.

Therefore, using a neutralizer is the approach that will not improve the LLM's response; it is more likely to hinder the LLM's ability to generate the specified style.

### Question: 39

CertyIQ

A Generative AI Engineer is creating an LLM-powered application that will need access to up-to-date news articles and stock prices.

The design requires the use of stock prices which are stored in Delta tables and finding the latest relevant news articles by searching the internet.

How should the Generative AI Engineer architect their LLM system?

A. Use an LLM to summarize the latest news articles and lookup stock tickers from the summaries to find stock prices.

B. Query the Delta table for volatile stock prices and use an LLM to generate a search query to investigate potential causes of the stock volatility.

C. Download and store news articles and stock price information in a vector store. Use a RAG architecture to retrieve and generate at runtime.

D. Create an agent with tools for SQL querying of Delta tables and web searching, provide retrieved values to an LLM for generation of response.

**Answer: D**

**Explanation:**

The correct answer is D because it leverages the strengths of both structured data access (Delta tables) and unstructured data retrieval (web searching) within a cohesive LLM-powered system.

Here's a detailed justification:

**Agents as Orchestrators:** Using an agent allows you to create a system that can intelligently choose the

appropriate tool (SQL query or web search) based on the specific requirements of the task. Agents empower LLMs to interact with external resources.

**Structured Data Access (SQL querying of Delta tables):** Stock prices are stored in Delta tables, which are ideal for structured data. SQL queries allow precise and efficient retrieval of this information. Delta Lake ensures ACID properties, data versioning, and schema enforcement, crucial for reliable financial data.

**Unstructured Data Retrieval (Web Searching):** News articles are unstructured and constantly updated. A web search tool enables the LLM to find the latest, relevant articles.

**RAG and Real-time Information:** While RAG is valuable, pre-downloading and storing news articles (option C) introduces staleness. Real-time web searching ensures the most up-to-date information. An agent with web search integrates retrieval and generation, but in a more dynamically orchestrated way compared to a standard RAG pipeline which pre-indexes a fixed dataset.

**LLM Integration:** The LLM serves as the central reasoning engine, taking the retrieved stock prices and news articles as context and generating a coherent response.

**Alternatives Evaluated:** Option A is inefficient as it relies on the LLM to extract stock tickers from potentially noisy news summaries. Option B focuses solely on volatile stocks and doesn't provide a comprehensive solution for accessing both news and stock data. Option C, while utilizing RAG, suffers from data staleness. The agent-based approach combines the benefits of both data access methods while maintaining currency of information.

#### Authoritative Links:

**Databricks Agents:** <https://www.databricks.com/blog/2023/11/02/introducing-foundation-model-apis-and-agents-databricks.html>

**Delta Lake:** <https://delta.io/>

**Retrieval-Augmented Generation (RAG):** <https://www.promptingguide.ai/techniques/rag>

#### Question: 40

CertyIQ

A Generative AI Engineer is designing a chatbot for a gaming company that aims to engage users on its platform while its users play online video games.

Which metric would help them increase user engagement and retention for their platform?

- A.Randomness
- B.Diversity of responses
- C.Lack of relevance
- D.Repetition of responses

**Answer: B**

**Explanation:**

The correct answer is **B. Diversity of responses**. Here's why:

A chatbot for a gaming company, intended to engage users during gameplay, needs to provide varied and interesting interactions to hold their attention. Diversity of responses directly contributes to user engagement and retention.

Here's a breakdown:

**Engagement:** A chatbot that gives diverse responses keeps the conversation fresh and stimulating. Users are more likely to continue interacting if they find the bot's answers interesting and unexpected. This is essential to hold attention while the user is also focused on the game.

**Retention:** If the chatbot consistently provides novel and relevant information or entertainment, users are more likely to return to the platform. The chatbot becomes a valuable companion, enhancing the gaming

experience.

**Randomness (A):** While a degree of randomness can introduce some variation, pure randomness without context or relevance would likely lead to nonsensical and frustrating interactions. The chatbot responses need to be aligned with the context of the user's inquiry.

**Lack of relevance (C):** This is detrimental to user experience. Irrelevant responses will frustrate users and cause them to disengage with the chatbot and potentially the platform.

**Repetition of responses (D):** This is the opposite of what's needed for engagement. Repetitive answers demonstrate a lack of intelligence and will quickly bore users, leading them to abandon the chatbot. A Generative AI system must have the ability to provide unique responses to similar prompts, this is usually achieved via Large Language Models.

In the context of Generative AI and specifically large language models (LLMs), diversity of responses ties into the model's ability to generate creative text formats, translate languages, write different kinds of creative content, and answer your questions in an informative way. For a gaming chatbot, this translates to generating varied in-game tips, jokes, or assistance in different, creative styles each time the user interacts with it.

To improve the diversity of responses, techniques like top-p sampling (nucleus sampling) or temperature scaling can be applied during the text generation process of LLMs. These techniques control the probability distribution of the model's output, encouraging more diverse and creative responses.

Authoritative links for further research:

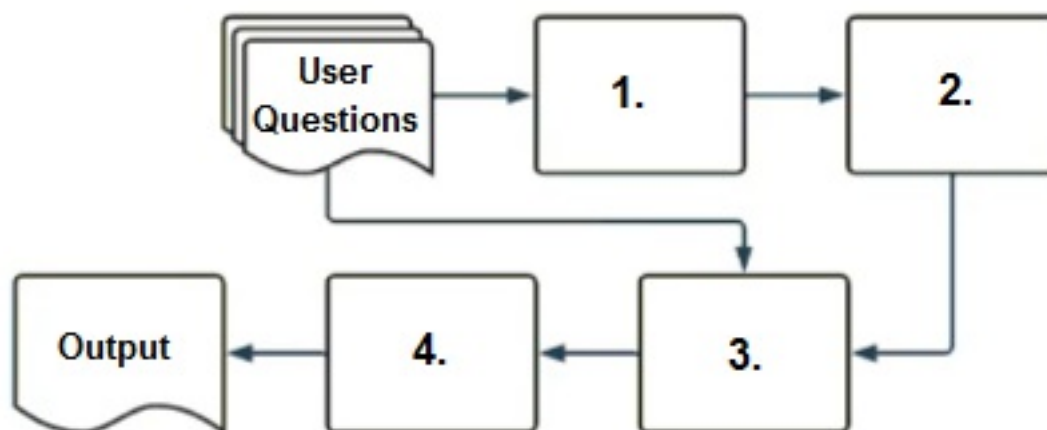
**Nucleus sampling (Top-p sampling):** <https://huggingface.co/blog/how-to-generate>

**Temperature scaling:** <https://towardsdatascience.com/how-to-control-the-creativity-of-a-generative-neural-network-using-temperature-59e25d71e234>

#### Question: 41

CertyIQ

A company has a typical RAG-enabled, customer-facing chatbot on its website.



Select the correct sequence of components a user's questions will go through before the final output is returned. Use the diagram above for reference.

- A.1.embedding model, 2.vector search, 3.context-augmented prompt, 4.response-generating LLM
- B.1.context-augmented prompt, 2.vector search, 3.embedding model, 4.response-generating LLM
- C.1.response-generating LLM, 2.vector search, 3.context-augmented prompt, 4.embedding model
- D.1.response-generating LLM, 2.context-augmented prompt, 3.vector search, 4.embedding model

**Answer: A**

**Explanation:**



A. 1.embeddingmodel, 2.vectorsearch, 3.context-augmented prompt, 4.response-generating LLM:

**Embedding model:** When a user's question is submitted, it is first processed by an embedding model. This model converts the text of the question into a vector representation, which captures the semantic meaning of the question in a format that can be used for efficient search and retrieval.

**Vector search:** The vector representation of the question is then used to perform a vector search within a knowledge base. This search retrieves relevant documents or pieces of information that are most closely related to the user's question based on the vector similarity.

**Context-augmented prompt:** The retrieved documents or information are then combined with the original user question to form a context-augmented prompt. This augmented prompt provides additional context to the response-generating model, ensuring that the response is based on the most relevant information available.

**Response-generating LLM:** Finally, the context-augmented prompt is passed to the response-generating LLM (Large Language Model), which uses the provided context to generate a coherent and relevant response to the user's question.

## Question: 42

CertyIQ

A team wants to serve a code generation model as an assistant for their software developers. It should support multiple programming languages. Quality is the primary objective.

Which of the Databricks Foundation Model APIs, or models available in the Marketplace, would be the best fit?

- A.Llama2-70b
- B.BGE-large
- C.MPT-7b
- D.CodeLlama-34B

**Answer: D**

**Explanation:**

D is the correct choice because CodeLlama-34B is specifically designed for code generation across multiple programming languages, ensuring high-quality output conducive to software development.

**Reasoning:**

1. **Specialization in Code Tasks:** CodeLlama-34B is tailored for programming-related tasks, thus optimized for understanding syntax, semantics, and the diverse requirements of various programming languages. This specialization enhances the quality of generated code compared to general models.
2. **Quality Focus:** The architecture of CodeLlama facilitates superior performance in code generation tasks through fine-tuning on code datasets, allowing for better comprehension and contextuality in generated responses.
3. **Multilingual Support:** The model's training includes multiple programming languages, ensuring versatility and adaptability in real-world scenarios, meeting the team's requirement for supporting diverse code bases.

**Critical Evaluation of Other Options:**

**A. Llama2-70b:** Although Llama2 has general capabilities in language processing, it lacks specific optimizations for code generation. Its general-purpose architecture may not adequately address the nuances

of programming languages, leading to lower code quality.

**B. BGE-large:** This model focuses more on general language generation rather than specialized coding tasks. While it may generate text effectively, it does not provide the precision or contextual relevance necessary for high-quality code creation.

**C. MPT-7b:** Similar to Llama2, MPT-7b is not specifically engineered for code generation. Its capabilities are broader and less focused, which may result in missed language-specific requirements and reduced quality in coding tasks.

In sum, while all models have their merits, CodeLlama-34B excels due to its deliberate design aimed at code generation, making it the optimal choice for enhancing developer productivity.

References:

<https://www.databricks.com/blog/2022/09/14/how-large-language-models-are-transforming-software-development.html>

[https://huggingface.co/docs/transformers/model\\_doc/codellama](https://huggingface.co/docs/transformers/model_doc/codellama)

[https://www.researchgate.net/publication/365234295\\_Coding\\_with\\_CodeLlama\\_A\\_New\\_Gear\\_In\\_A\\_Developer's\\_Toolbox](https://www.researchgate.net/publication/365234295_Coding_with_CodeLlama_A_New_Gear_In_A_Developer's_Toolbox)

### Question: 43

CertyIQ

A Generative AI Engineer is building a RAG application that will rely on context retrieved from source documents that are currently in PDF format. These PDFs can contain both text and images. They want to develop a solution using the least amount of lines of code.

Which Python package should be used to extract the text from the source documents?

- A.flask
- B.beautifulsoup
- C.unstructured
- D.numpy

**Answer: C**

**Explanation:**

The correct answer is C, unstructured. Here's why:

unstructured is specifically designed for pre-processing unstructured data, including PDFs with both text and images, for downstream machine learning tasks like RAG. Its core purpose is to extract text and other elements from various file types in a structured way, facilitating easier use with LLMs. It handles different document layouts and formats efficiently. It is an ideal library for extracting text from complex documents like PDFs.

flask (A) is a micro web framework for building web applications and APIs. While it could be used to serve a RAG application, it doesn't provide any text extraction capabilities itself.

beautifulsoup (B) is primarily used for parsing HTML and XML. While it could be used if the PDF were somehow converted to HTML (which would be inefficient), it is not directly suited for processing PDFs and won't handle images.

numpy (D) is a numerical computing library. It provides support for arrays and mathematical operations but is irrelevant to text extraction from PDFs.

unstructured excels because it aims to minimize code complexity by providing a single, unified interface for extracting text and other data from a range of document types, including PDFs with images, which directly

aligns with the prompt's requirement of using the least amount of code. It handles the complexities of PDF parsing, including layout analysis and image handling. The library returns a structured format containing text blocks, which is ideal for feeding directly into vector databases and subsequent RAG pipeline components.

For more information, please refer to the official unstructured documentation: <https://unstructured-io.github.io/unstructured/>

#### Question: 44

CertyIQ

A Generative AI Engineer received the following business requirements for an external chatbot. The chatbot needs to know what types of questions the user asks and routes to appropriate models to answer the questions. For example, the user might ask about upcoming event details. Another user might ask about purchasing tickets for a particular event. What is an ideal workflow for such a chatbot?

- A. The chatbot should only look at previous event information
- B. There should be two different chatbots handling different types of user queries.
- C. The chatbot should be implemented as a multi-step LLM workflow. First, identify the type of question asked, then route the question to the appropriate model. If it's an upcoming event question, send the query to a text-to-SQL model. If it's about ticket purchasing, the customer should be redirected to a payment platform.
- D. The chatbot should only process payments

#### Answer: C

#### Explanation:

The correct answer is C because it describes a robust and scalable architecture that leverages the strengths of Large Language Models (LLMs) within a multi-step workflow to address the specific business requirements. The core principle here is intelligent routing based on intent. Option C outlines this: first, the chatbot uses an LLM to classify the user's question (identifying the type or category). This is crucial for understanding the user's intent.

Following intent recognition, the question is routed to a specialized model or service best suited for that category. This is a more efficient and accurate approach than a single model trying to handle all types of queries. For example, an event detail question might be best answered by a text-to-SQL model querying an event database, whereas a ticket purchase inquiry logically triggers a redirection to a payment platform for secure transaction processing. This separation of concerns and delegation of tasks to appropriate systems ensures optimal performance, scalability, and security.

Option A is incorrect because limiting the chatbot's knowledge to previous event information severely restricts its utility and doesn't address all possible user queries. Option B, creating two separate chatbots, duplicates resources, increases management overhead, and offers a poorer user experience compared to a single, intelligently routed chatbot. Option D is also incorrect as it only focuses on payments and ignores the other requirement of answering questions.

The multi-step LLM workflow represents a best practice for building conversational AI systems. This approach helps to solve complex problems in a structured and efficient way. By breaking down the process into distinct stages, it facilitates modularity, easier debugging, and the ability to update individual components without impacting the entire system.

For further research, consider the concepts of "Chain-of-Thought prompting" and "LLM Agents" within the LangChain framework, which directly supports the development of multi-step LLM workflows: <https://www.langchain.com/> and research papers on "Intent Recognition in Conversational AI."

**Question: 45****CertyIQ**

A Generative AI Engineer is tasked with developing an application that is based on an open source large language model (LLM). They need a foundation LLM with a large context window. Which model fits this need?

- A. DistilBERT
- B. MPT-30B
- C. Llama2-70B
- D. DBRX

**Answer: D****Explanation:**

D. DBRX.

Large context window (up to ~32k tokens)

Open-source foundation model

Designed for:

Long documents

Multi-turn reasoning

Complex enterprise use cases

This makes DBRX the best match when a large context window is the primary requirement.

**Question: 46****CertyIQ**

A Generative AI Engineer is building an LLM-based application that has an important transcription (speech-to-text) task. Speed is essential for the success of the application.

Which open Generative AI models should be used?

- A. DBRX
- B. MPT-30B-Instruct
- C. Llama-2-70b-chat-hf
- D. whisper-large-v3 (1.6B)

**Answer: D****Explanation:**

D. whisper-large-v3 (1.6B).

Whisper is a suite of models specifically designed and trained by OpenAI for Automatic Speech Recognition (ASR), which includes transcription, speech translation, and language identification. It is the leading open-source model for transcription accuracy and robustness.

**Question: 47**

A Generative AI Engineer has a provisioned throughput model serving endpoint as part of a RAG application and would like to monitor the serving endpoint's incoming requests and outgoing responses.

Which Databricks feature should they use?

- A.AutoML
- B.Vector Search
- C.Inference Tables
- D.Feature Serving

**Answer: C**

**Explanation:**

C. Inference Tables.

Why Inference Tables are the right choice

Inference Tables in Databricks are designed specifically to:

Capture incoming requests sent to a model serving endpoint

Store outgoing responses generated by the model

Provide observability for:

Debugging

Auditing

Monitoring behavior and drift

Work seamlessly with provisioned throughput model serving endpoints

This makes them ideal for monitoring a RAG application's production traffic.

**Question: 48**

A Generative AI Engineer is deciding between using LSH (Locality Sensitive Hashing) and HNSW (Hierarchical Navigable Small World) for indexing their vector database. Their top priority is semantic accuracy.

Which approach should the Generative AI Engineer use to evaluate these two techniques?

- A.Compare the cosine similarities of the embeddings of returned results against those of a representative sample of test inputs
- B.Compare the Bilingual Evaluation Understudy (BLEU) scores of returned results for a representative sample of test inputs
- C.Compare the Recall-Oriented-Understudy for Gisting Evaluation (ROUGE) scores of returned results for a representative sample of test inputs
- D.Compare the Levenshtein distances of returned results against a representative sample of test inputs

**Answer: A**

**Explanation:**

The correct choice is A, as comparing the cosine similarities of the embeddings of returned results provides a direct measure of semantic accuracy in high-dimensional vector spaces.

**Reasoning for Choice A:** Cosine similarity effectively measures the angle between two vectors, emphasizing the semantic proximity of embeddings. This metric is particularly suitable for evaluating techniques such as LSH and HNSW, which rely on the geometric relationships in the embedding space. By comparing the cosine similarities, the Generative AI Engineer can ascertain the fidelity of each indexing technique in preserving semantic relationships among the data points.

#### Critical Evaluation of Other Options:

**B (BLEU scores):** BLEU is primarily a text generation evaluation metric that compares generated outputs to reference texts. It is unsuitable for gauging the semantic accuracy of embedding retrieval since it does not account for the vector space's geometric characteristics crucial for accurate semantic alignment.

<https://www.aclweb.org/anthology/W02-0909.pdf>

**C (ROUGE scores):** ROUGE is designed to evaluate summarization tasks by comparing the overlap between recall and precision. Like BLEU, it focuses on textual output rather than the underlying vector relationships, rendering it ineffective for analyzing indexing accuracy in vector databases. <https://aclanthology.org/W04-1013.pdf>

**D (Levenshtein distance):** This metric quantifies edit distances between strings, making it inapplicable for evaluating semantic relationships in embeddings. It disregards the semantic context represented in vector spaces and instead focuses solely on string manipulation. Thus, it fails to address the core goal of semantic accuracy when assessing LSH versus HNSW. [https://en.wikipedia.org/wiki/Levenshtein\\_distance](https://en.wikipedia.org/wiki/Levenshtein_distance)

In summary, choice A clearly aligns with the evaluation of semantic accuracy through cosine similarity, while the other options focus either on textual evaluation or unrelated metrics.

#### Question: 49

CertyIQ

When developing an LLM application, it's crucial to ensure that the data used for training the model complies with licensing requirements to avoid legal risks.

Which action is most appropriate to avoid legal risks?

- A. Only use data explicitly labeled with an open license and ensure the license terms are followed.
- B. Any LLM outputs are reasonable to use because they do not reveal the original sources of data directly.
- C. Reach out to the data curators directly to gain written consent for using their data.
- D. Use any publicly available data as public data does not have legal restrictions.

**Answer: A**

**Explanation:**

A. Only use data explicitly labeled with an open license and ensure the license terms are followed.

Why A is the most appropriate action

Licensing compliance is non-negotiable when training or fine-tuning LLMs.

Using data with an explicit open license (e.g., Apache-2.0, MIT, CC-BY) ensures:

Legal permission to use the data

Clear conditions on attribution, redistribution, and commercial use



Following the license terms protects the company from:

Copyright infringement claims

Contractual violations

Reputational risk

### Question: 50

CertyIQ

A Generative AI Engineer interfaces with an LLM with instruction-following capabilities trained on customer calls inquiring about product availability. The LLM should output “Success” if the product is available or “Fail” if not.

Which prompt allows the engineer to receive call classification labels correctly?

A.You are a helpful assistant that reads customer call transcripts. Walk through the transcript and think step-by-step if the customer’s inquiries are addressed successfully. Answer “Success” if yes; otherwise, answer “Fail”.

B.You will be given a customer call transcript where the customer asks about product availability. Classify the call as “Success” if the product is available and “Fail” if the product is unavailable.

C.You will be given a customer call transcript where the customer asks about product availability. The outputs are either “Success” or “Fail”. Format the output in JSON, for example: "call\_id": "123", "label": "Success" .

D.You will be given a customer call transcript. Answer “Success” if the customer call has been resolved successfully. Answer “Fail” if the call is redirected or if the question is not resolved.

### Answer: B

#### Explanation:

Option B is the most effective prompt, as it directly instructs the LLM to classify the availability of a product based on customer inquiries with a clear binary output.

#### Justification for Option B:

1. **Clarity of Instruction:** The prompt explicitly states that the task is to classify the call based on product availability, making it easy for the LLM to understand its objectives.
2. **Direct Mapping of Inquiry to Output:** By correlating the inquiry about product availability with the outputs “Success” and “Fail,” it establishes a straightforward classification logic.
3. **Focus on the Essential Detail:** The essential variable here is product availability, which Option B concentrates on, ensuring accuracy in classification.

#### Evaluation of Other Options:

**Option A:** Although it encourages a step-by-step analysis, this approach could introduce unnecessary complexity, potentially leading to ambiguous interpretations of success and failure. The simplicity of generation regarding binary outcomes is lost.

**Option C:** While it offers a structured output in JSON format, which is useful for integration purposes, it does not emphasize the binary classification itself as effectively as Option B. The prompt's focus on formatting could detract from the primary goal of assessment.

**Option D:** This implies a broader resolution framework rather than focusing solely on product availability. The reference to call redirection or non-resolution could introduce inconsistencies, as not all unsuccessful inquiries will relate to product availability specifically, thereby complicating the task at hand.

#### References:

<https://www.microsoft.com/en-us/research/publication/language-models-are-few-shot-learners/>  
<https://openai.com/research/guidelines-for-using-large-language-models>  
<https://www.ibm.com/cloud/learn/generative-ai>

### Question: 51

CertyIQ

Which TWO chain components are required for building a basic LLM-enabled chat application that includes conversational capabilities, knowledge retrieval, and contextual memory? (Choose two.)

- A. Vector Stores
- B. Conversation Buffer Memory
- C. External tools
- D. Chat loaders
- E. React Components

**Answer: AB**

#### Explanation:

A. Vector Stores.

Vector stores are used to handle knowledge retrieval. They store embeddings (vector representations of data) that can be queried to retrieve relevant information. This is essential for making the chat application capable of pulling in useful knowledge for conversational purposes.

B. Conversation Buffer Memory.

This is crucial for maintaining contextual memory, which allows the chat application to remember previous conversations or exchanges. It helps the model understand context over multiple interactions, leading to more coherent and contextually aware responses.

### Question: 52

CertyIQ

A Generative AI Engineer has written scalable PySpark code to ingest unstructured PDF documents and chunk them in preparation for storing in a Databricks Vector Search index. Currently, the two columns of their dataframe include the original filename as a string and an array of text chunks from that document.

What set of steps should the Generative AI Engineer perform to store the chunks in a ready-to-ingest manner for Databricks Vector Search?

- A. Use PySpark's autoloader to apply a UDF across all chunks, formatting them in a JSON structure for Vector Search ingestion.
- B. Flatten the dataframe to one chunk per row, create a unique identifier for each row, and enable change feed on the output Delta table.
- C. Utilize the original filename as the unique identifier and save the dataframe as is.
- D. Create a unique identifier for each document, flatten the dataframe to one chunk per row and save to an output Delta table.

**Answer: B**

#### Explanation:

Flatten the dataframe to one chunk per row: Databricks Vector Search indices expect the source Delta table to be structured so that each row corresponds to a single record (in this case, one text chunk). Having an array

of chunks in a single row is not the standard ingestion format.

Create a unique identifier for each row: A primary key is required for the Delta table and the resulting vector index to uniquely identify each chunk.

Enable Change Data Feed (CDF) on the output Delta table: This is a critical requirement for "Delta Sync" indices. Enabling CDF allows the Vector Search index to automatically detect and sync inserts, updates, and deletes from the Delta table with low latency.

Why other options are incorrect:

Option A: While you can use PySpark for transformations, simply formatting into JSON is not the required ingestion pattern for a managed Delta Sync index.

Option C: Using only the filename as an ID is insufficient because a single document contains multiple chunks. If you save the dataframe "as is" (with an array of chunks in one row), the index cannot efficiently retrieve or update individual chunks.

Option D: This is very close to Option B, but it misses the essential requirement of enabling Change Data Feed (CDF). Without CDF enabled on the source table, the index cannot perform continuous or incremental synchronization.

### Question: 53

CertyIQ

A Generative AI Engineer is asked to build an LLM application that would excel at code generation. They need to select a model that has been specifically trained to generate code.

Which model would likely produce the best results out of the box?

- A.CodeLlama-34b-Instruct-hf
- B.Mixtral-8x7B-v0.1
- C.Llama-2-70b-hf
- D.mpt-7b-8k-instruct

**Answer: A**

**Explanation:**

A. CodeLlama-34b-Instruct-hf.

CodeLlama is an open-source Large Language Model built upon the Llama 2 architecture, but it was further trained on a large volume of code data and then fine-tuned for conversational/instructional use. This specialization gives it a significant advantage over general-purpose models for coding tasks.

The -Instruct-hf variant means it has been fine-tuned for generating code when given natural language instructions (e.g., "Write a Python function to calculate the factorial of a number.").

### Question: 54

CertyIQ

A Generative AI Engineer needs to design an LLM pipeline to conduct multi-stage reasoning that leverages external tools. To be effective at this, the LLM will need to plan and adapt actions while performing complex reasoning tasks.

Which approach will do this?

- A. Train the LLM to generate a single, comprehensive response without interacting with any external tools, relying solely on its pre-trained knowledge.
- B. Use a Chain-of-Thought (CoT) prompting technique to guide the LLM through a series of reasoning steps, then manually input the results from external tools for the final answer.
- C. Implement a framework like ReAct, which allows the LLM to generate reasoning traces and perform task-specific actions that leverage external tools if necessary.
- D. Encourage the LLM to make multiple API calls in sequence without planning or structuring the calls, allowing the LLM to decide when and how to use external tools spontaneously.

**Answer: C**

**Explanation:**

C. Implement a framework like ReAct, which allows the LLM to generate reasoning traces and perform task-specific actions that leverage external tools if necessary.

The requirement is multi-stage reasoning + external tools + adaptive planning.

ReAct (Reason + Act) is designed exactly for this:

The LLM:

Plans what to do next

Reasons step by step

Acts by calling external tools (search, calculators, APIs)

Observes results and adapts its next action

This creates a tight reasoning–action–feedback loop, ideal for complex tasks.

### Question: 55

CertyIQ

A Generative AI Engineer at an automotive company would like to build a question-answering chatbot for customers to inquire about their vehicles. They have a database containing various documents of different vehicle makes, their hardware parts, and common maintenance information.

Which of the following components will NOT be useful in building such a chatbot?

- A. Invite users to submit long, rather than concise, questions
- B. Response-generating LLM
- C. Embedding model
- D. Vector database

**Answer: A**

**Explanation:**

A is correct because inviting users to submit long questions can hinder the efficiency and effectiveness of a question-answering chatbot.

Long questions often introduce ambiguity and complexity, making it challenging for natural language processing (NLP) models to accurately discern intent and extract relevant information. A concise question allows for clearer understanding and context, which is essential for generating relevant responses. Therefore, facilitating brevity would streamline the interaction, enhancing user experience and model performance.

## Evaluation of Other Options:

**B. Response-generating LLM:** Essential for generating human-like responses. A large language model (LLM) can understand the context of questions and provide accurate answers based on the embedded knowledge of vehicle maintenance and parts, making it indispensable for the chatbot's functionality.

**C. Embedding model:** Integral for converting user queries and documents into vector representations. This helps in semantic similarity searches, enabling the chatbot to effectively match questions with relevant documents or knowledge in the database, which supports accurate and contextually relevant responses.

**D. Vector database:** Critical for storing and retrieving vector representations efficiently. It allows for fast querying against the embedded data. This component is necessary for enabling the chatbot to quickly retrieve information that aligns closely with user inquiries.

Each of the components B, C, and D contributes fundamentally to the infrastructure of a question-answering system, while option A detracts from the clarity and effectiveness of interactions.

## References

<https://towardsdatascience.com/generative-ai-a-complete-guide-to-large-language-models-5b1bc40b1709>

<https://www.analyticsvidhya.com/blog/2022/11/a-comprehensive-guide-to-deploying-chatbots-using-databricks/>

<https://www.ibm.com/cloud/learn/what-is-natural-language-processing>

## Question: 56

CertyIQ

A Generative AI Engineer is building an LLM to generate article headlines given the article content. However, the initial output from the LLM does not match the desired tone or style.

Which approach would be most effective for adjusting the LLM's response to achieve the desired response?

- A.Exclude any article headlines that do not match the desired output
- B.Fine-tune the LLM on a dataset of desired tone and style
- C.Provide the LLM with a prompt that explicitly instructs it to generate text in the desired tone and style
- D.All of the above

**Answer: C**

**Explanation:**

C. Provide the LLM with a prompt that explicitly instructs it to generate text in the desired tone and style ,

Prompt engineering is the fastest, simplest, and most cost-effective way to control tone and style.

It requires no retraining, no new datasets, and no infrastructure changes.

Best practice is to start with prompting and only move to fine-tuning if prompts are insufficient.

## Question: 57

CertyIQ

A Generative AI Engineer is creating a customer support bot that should respond differently to an end user based on the sentiment in their initial message. For example, if the end user's message was angry, the bot should try to de-escalate their negative sentiments as it solves the customer query. They want to make sure their approach follows best practices.

Which approach will do this?

- A. Use an encoder-only LLM model to both detect sentiment and generate replies based upon the detected sentiment.
- B. Implement a RAG architecture for how to respond to users depending on detected sentiment.
- C. Use linear regression model to classify sentiment and feed the result to a system prompt for the LLM to respond.
- D. Create a chain which first uses an LLM to classify sentiment, then changes system prompt for the customer interaction LLM based upon the initial customer query sentiment.

**Answer: D**

**Explanation:**

D. Create a chain which first uses an LLM to classify sentiment, then changes the system prompt for the customer interaction LLM based upon the initial customer query sentiment.

Clear separation of responsibilities.

Step 1: Detect sentiment (angry, neutral, positive)

Step 2: Generate a response tailored to that sentiment

Dynamic behavior control

The sentiment output directly adjusts the system prompt, enabling:

De-escalation and empathy for angry users

Efficient, neutral responses otherwise

Modular and scalable

Easy to extend with urgency detection, escalation rules, or compliance checks.

## Question: 58

CertyIQ

A Generative AI Engineer is ready to deploy an LLM application written using Foundation Model APIs. They want to follow security best practices for production scenarios.

Which authentication method should they choose?

- A. Use OAuth machine-to-machine authentication
- B. Use an access token belonging to service principals
- C. Use an access token belonging to any workspace user
- D. Use a frequently rotated access token belonging to either a workspace user or a service principal

**Answer: B**

**Explanation:**

The correct answer is **B. Use an access token belonging to service principals**. Here's a detailed justification:

In production environments, security best practices dictate avoiding the use of personal access tokens tied to individual users. Workspace user accounts are associated with specific individuals, and their access can be disrupted if the user leaves the organization, changes roles, or their account is compromised. Using an access token tied to a user introduces a single point of failure and complicates access management. OAuth machine-

to-machine authentication (option A) is relevant in many API scenarios, but it typically involves a client ID and secret, which would need to be securely managed. This adds unnecessary complexity compared to service principals. Option C is clearly insecure due to the user dependency and risk of lateral movement. Option D suggests frequent rotation, which can improve security but doesn't solve the underlying problem of using a user-bound token; also rotating token frequently may break the applications.

Service principals, on the other hand, are non-human identities specifically designed for applications and automated processes. They offer several advantages:

1. **Independence from Users:** Service principals are not tied to individual user accounts, eliminating the risk of access disruption when users leave or change roles.
2. **Centralized Management:** Service principals can be centrally managed through the cloud platform's identity and access management (IAM) system, enabling fine-grained control over their permissions and access rights.
3. **Auditing:** Service principal activity is easily auditable, allowing you to track and monitor how your LLM application is accessing resources.
4. **Security:** Service principals adhere to the principle of least privilege, granting applications only the permissions they need to perform their tasks, minimizing the potential impact of a security breach.

Therefore, using an access token belonging to a service principal is the most secure and manageable authentication method for production LLM applications leveraging Foundation Model APIs. It ensures that the application can access resources securely and reliably, without being dependent on individual user accounts.

Further Research:

**Azure Service Principals:** <https://learn.microsoft.com/en-us/azure/active-directory/develop/app-objects-and-service-principals>

**AWS IAM Roles:** [https://docs.aws.amazon.com/IAM/latest/UserGuide/id\\_roles.html](https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html) (Analogous concept in AWS)

**Google Cloud Service Accounts:** <https://cloud.google.com/iam/docs/service-accounts> (Analogous concept in GCP)

## Question: 59

CertyIQ

A Generative AI Engineer is developing a RAG system for their company to perform internal document Q&A for structured HR policies, but the answers returned are frequently incomplete and unstructured. It seems that the retriever is not returning all relevant context. The Generative AI Engineer has experimented with different embedding and response generating LLMs but that did not improve results.

Which TWO options could be used to improve the response quality? (Choose two.)

- A. Add the section header as a prefix to chunks
- B. Split the document by sentence
- C. Use a larger embedding model
- D. Increase the document chunk size
- E. Fine tune the response generation model

**Answer: AD**

**Explanation:**

Here's a detailed justification for why options A and D are the correct choices, and why the others are less suitable for improving the response quality of a RAG system in this scenario:

The problem the engineer is facing is incomplete and unstructured answers, stemming from the retriever



failing to provide sufficient context. This points towards issues with the way the documents are chunked and indexed, affecting the retriever's ability to identify and retrieve the most relevant passages.

**A. Add the section header as a prefix to chunks:** This is a valid strategy. By prepending the section header to each chunk, you provide the embedding model with crucial contextual information. This helps the retriever understand the overall topic of each chunk more effectively. Section headers often summarize the main point of the section and provide semantic context. Consequently, the similarity search during retrieval will be more accurate, bringing in relevant chunks even if the question only mentions details found within that section. This directly addresses the problem of incomplete answers.

**D. Increase the document chunk size:** A key problem is the retriever not returning enough relevant context. Smaller chunks may break up logical units of information, leading to insufficient context for generating comprehensive answers. By increasing the chunk size, you ensure that more related information is included in each chunk. This provides the LLM with a broader context window, resulting in more complete and structured answers. However, it's important to avoid excessively large chunks, which can dilute the context and make retrieval less precise.

**Why the other options are less suitable:**

**B. Split the document by sentence:** While very small chunk sizes can be useful in some niche scenarios, splitting by sentence is generally too granular for HR policies. Sentence-level chunks typically lack sufficient context, making retrieval less effective. The LLM will have to stitch together information scattered across many unrelated sentences, leading to worse performance. The lack of contextual information will also affect how the embedding model can generate a good vector representation.

**C. Use a larger embedding model:** While a larger embedding model might slightly improve the quality of embeddings, it's unlikely to be the primary bottleneck in this scenario. The problem is that the chunks themselves do not contain enough relevant information. The retriever's accuracy depends on the content it has to work with. Improving embeddings is typically a second-order optimization, not the primary lever for solving context retrieval issues.

**E. Fine-tune the response generation model:** Fine-tuning is effective when the LLM struggles to generate good responses given adequate context. However, the core issue here is that the retriever isn't providing sufficient context in the first place. Fine-tuning won't solve the underlying problem of incomplete context retrieval. It's like trying to improve a chef's cooking when the ingredients they receive are subpar.

**Authoritative Links for further research:**

**RAG (Retrieval-Augmented Generation):** You can learn more about RAG techniques in general from various online resources. A good starting point would be the original paper that introduced the concept:

<https://arxiv.org/abs/2005.11401>

**Chunking Strategies:** Cohere's documentation provides good examples of different chunking strategies and their trade-offs <https://txt.cohere.com/>

In summary, focusing on improving the quality and completeness of the chunks provided to the retriever, specifically by adding section headers and increasing chunk size, will have the greatest impact on improving the response quality of the RAG system in this scenario.

## Question: 60

CertyIQ

A Generative AI Engineer is building a production-ready LLM system which replies directly to customers. The solution makes use of the Foundation Model API via provisioned throughput. They are concerned that the LLM could potentially respond in a toxic or otherwise unsafe way. They also wish to perform this with the least amount of effort.

Which approach will do this?

- A. Ask users to report unsafe responses
- B. Host Llama Guard on Foundation Model API and use it to detect unsafe responses.
- C. Add some LLM calls to their chain to detect unsafe content before returning text
- D. Add a regex expression on inputs and outputs to detect unsafe responses.

**Answer: B**

**Explanation:**

The best approach to mitigate unsafe LLM responses in a production system using the Foundation Model API, while minimizing effort, is option B: hosting Llama Guard on the Foundation Model API to detect unsafe responses. Here's why:

**Proactive Safety:** Llama Guard acts as a gatekeeper, inspecting LLM outputs before they reach the customer. This is a proactive measure, preventing potentially harmful content from being delivered.

**Reduced Manual Effort:** Implementing Llama Guard automates the safety check process. Unlike relying on user reports (A), which are reactive and depend on customers encountering unsafe responses, or manually adding LLM calls to a chain (C) which requires more processing and maintenance. Llama Guard also avoids the simplistic approach of regex expressions (D), which is insufficient to detect nuanced forms of toxicity.

**Specialized Model:** Llama Guard is specifically designed for detecting and blocking unsafe content in LLM outputs. Its model is optimized for this task, offering better accuracy and coverage than generic approaches.

**Foundation Model API Integration:** The question specifies using the Foundation Model API. Hosting Llama Guard directly on this API would offer a streamlined and integrated solution, avoiding the need to build separate infrastructure for content moderation.

**Scalability:** A dedicated safety model like Llama Guard is designed to handle the volume of requests expected in a production environment, ensuring low latency and high availability for safety checks.

**Customization:** Llama Guard can be further fine-tuned to meet the specific requirements of the application, allowing for customization of safety thresholds and content policies.

**Cost-Effectiveness:** While there may be a cost associated with hosting Llama Guard, it's likely to be more cost-effective than the manual effort of reviewing user reports or the computational cost of additional LLM calls for content detection.

**Relevant Links:**

Meta AI (Llama Guard): <https://ai.meta.com/blog/llama-guard-open-source-input-output-safeguard-llm/>

Databricks Foundation Model API: (Please note that you will need to review Databricks official documentation for Foundation Model API specifically to check integration options.)

**Question: 61**

**CertyIQ**

A Generative AI Engineer would like an LLM to parse and extract the following information: date, sender email, and order ID. The output should be formatted into JSON. Here's an email sample:

Date: April 16, 2024  
Time: 3:45 PM  
From: sarah.lee925@gmail.com  
To: customer\_service@smart\_thermostat.com  
Subject: Order update

Hi,

Can I get a status update on my order? The order ID is RE987D.

Thank you,  
Sarah

They need a prompt that will extract and output the required information in JSON with the highest level of output accuracy.

Which prompt will do that?

A.You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in a human-readable format.

B.You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in JSON format.

C.You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in JSON format. Here's an example: "date":"April 16, 2024", "sender\_email":"[email protected]", "order\_id":"RE987D"

D.You will receive customer emails and need to extract date, sender email, and order ID. You should return the date, sender email, and order ID information in JSON format.

**Answer: C**

**Explanation:**

C.You will receive customer emails and need to extract date, sender email, and order ID. Return the extracted information in JSON format. Here's an example: "date":"April 16, 2024", "sender\_email":"[email protected]", "order\_id":"RE987D"

Option C provides a specific example of the desired JSON structure: "date":"April 16, 2024", "sender\_email":"[email protected]", "order\_id":"RE987D". This is the most effective technique for ensuring the AI's output is structured correctly.

AI models are trained on vast amounts of data and can follow patterns very well. By giving a clear example, you are showing the model exactly how you want the keys ("date", "sender\_email", "order\_id") and their corresponding values to be formatted.

This reduces the chance of the AI hallucinating or using different key names (e.g., email\_address instead of sender\_email) or formatting the date differently than intended.

## Question: 62

CertyIQ

A Generative AI Engineer has built an LLM-based system that will automatically translate user text between two languages. They now want to benchmark multiple LLM's on this task and pick the best one. They have an

evaluation set with known high quality translation examples. They want to evaluate each LLM using the evaluation set with a performant metric.

Which metric should they choose for this evaluation?

- A. BLEU metric
- B. NDCG metric
- C. ROUGE metric
- D. RECALL metric

**Answer: A**

**Explanation:**

The correct metric for evaluating the performance of an LLM-based translation system, given a labeled evaluation set of high-quality translations, is the BLEU (Bilingual Evaluation Understudy) metric.

Here's why:

**BLEU's Suitability for Translation:** BLEU is specifically designed for evaluating machine translation quality. It measures the similarity between the machine-generated translation and one or more reference translations.

**N-gram Precision:** BLEU assesses the precision of n-grams (sequences of n words) in the generated text compared to the reference text. A higher precision indicates that the generated text contains more of the same phrases as the reference.

**Brevity Penalty:** To prevent systems from generating very short translations to artificially inflate precision, BLEU incorporates a brevity penalty. This penalty penalizes translations that are shorter than the reference translations.

**Efficiency:** BLEU is relatively efficient to compute, making it suitable for benchmarking multiple LLMs on a large evaluation set. It doesn't require human judgment for each translation, offering a scalable evaluation approach.

**Alternatives Considered:**

**ROUGE (Recall-Oriented Understudy for Gisting Evaluation):** While ROUGE is also used for text evaluation, it focuses on recall (how much of the reference text is captured in the generated text). It's more commonly used for summarization tasks where capturing all the key information is crucial.

**NDCG (Normalized Discounted Cumulative Gain):** NDCG is primarily used for ranking tasks where the order of items is important (e.g., search results). It's not applicable to translation, where the goal is to generate a text that matches the meaning of the original.

**Recall:** Recall, in general, focuses on how many of the relevant items (in this case, words/phrases) are retrieved. While important, it doesn't account for the overall fluency or quality of the generated translation as comprehensively as BLEU.

Therefore, BLEU is the most appropriate and performant metric to use in this scenario, as it directly assesses the accuracy and fluency of the translated text compared to the high-quality reference translations.

**Authoritative Links:**

**BLEU: a Method for Automatic Evaluation of Machine Translation:** <https://aclanthology.org/P02-1040/>  
**Wikipedia - BLEU:** <https://en.wikipedia.org/wiki/BLEU>

## Question: 63

CertyIQ

A Generative AI Engineer is using an LLM to classify species of edible mushrooms based on text descriptions of certain features. The model is returning accurate responses in testing and the Generative AI Engineer is confident they have the correct list of possible labels, but the output frequently contains additional reasoning in the answer

when the Generative AI Engineer only wants to return the label with no additional text.

Which action should they take to elicit the desired behavior from this LLM?

- A. Use few shot prompting to instruct the model on expected output format
- B. Use zero shot prompting to instruct the model on expected output format
- C. Use zero shot chain-of-thought prompting to prevent a verbose output format
- D. Use a system prompt to instruct the model to be succinct in its answer

**Answer: A**

**Explanation:**

The correct answer is A: Use few-shot prompting to instruct the model on the expected output format. Here's why:

The problem states that the LLM is providing accurate classifications but also includes unnecessary reasoning. The goal is to get the model to output only the label without the extra explanation. Different prompting techniques can influence the LLM's behavior.

**Few-shot prompting** involves providing the LLM with a few examples of the desired input-output format. This allows the model to learn the pattern and replicate it for new, unseen inputs. In this case, you would give examples where the input is a mushroom description, and the output is simply the species label, without any extra text. This directly demonstrates to the model the required behavior. This technique excels when you have a clear idea of the desired output format.

**Zero-shot prompting** relies on the model's pre-existing knowledge without providing any specific examples. While it's useful for general tasks, it might not be precise enough to enforce a specific output format. Simply asking the model to "only output the label" in a zero-shot setting might not be sufficient, as the model may still include reasoning based on its inherent training.

**Zero-shot chain-of-thought prompting** is designed to elicit reasoning from the model, not prevent it. It typically involves adding "Let's think step by step" to the prompt, encouraging the model to explain its reasoning process. This is the opposite of what the problem requires.

A **system prompt** can set the tone and context for the interaction, but it may not be as effective as few-shot prompting for enforcing a specific output format. While you could use a system prompt to instruct the model to be succinct, few-shot prompting provides clearer guidance through demonstration, making it more reliable for achieving the desired output. Furthermore, system prompts are often geared towards persona definition or task overarching task instructions. Few-shot prompting is more granular control.

In summary, few-shot prompting is the most appropriate technique because it demonstrates the exact input-output relationship desired, helping the LLM understand and replicate the succinct label-only output format.

For more information on prompting techniques:

[Prompt Engineering Guide](#): Covers various prompting techniques, including few-shot learning.

[Google AI - Prompting](#): Discusses different prompting strategies and their impact on LLM behavior.

## Question: 64

CertyIQ

A Generative AI Engineer is working with a retail company that wants to enhance its customer experience by automatically handling common customer inquiries. They are working on an LLM-powered AI solution that should improve response times while maintaining a personalized interaction. They want to define the appropriate input and LLM task to do this.

Which input/output pair will do this?

- A. Input: Customer service chat logs; Output: Group the chat logs by users, followed by summarizing each user's interactions, then respond
- B. Input: Customer service chat logs; Output: Find the answers to similar questions and respond with a summary
- C. Input: Customer reviews; Output: Classify review sentiment
- D. Input: Customer reviews; Output: Group the reviews by users and aggregate per-user average rating, then respond

**Answer: A**

**Explanation:**

A. Input: Customer service chat logs; Output: Group the chat logs by users, followed by summarizing each user's interactions, then respond

Automatically handle common inquiries

Improve response time

Maintain personalized interaction

Option A directly supports all three:

Uses customer service chat logs → relevant, conversational context

Groups by user → preserves personalization

Summarizes each user's past interactions → lets the LLM respond with awareness of history ("I see you contacted us before about...")

Enables fast, automated, yet context-aware responses.

### Question: 65

**CertyIQ**

A Generative AI Engineer is developing a RAG application and would like to experiment with different embedding models to improve the application performance.

Which strategy for picking an embedding model should they choose?

- A. Pick an embedding model with multilingual support to support potential multilingual user questions
- B. Pick the most recent and most performant open LLM released at the time
- C. Pick an embedding model trained on related domain knowledge
- D. Pick the embedding model ranked highest on the Massive Text Embedding Benchmark (MTEB) leaderboard hosted by HuggingFace

**Answer: C**

**Explanation:**

Choosing option C is correct because selecting an embedding model trained on related domain knowledge directly enhances the relevance and contextual understanding of the model for specific applications.

**Reasoning**

An embedding model designed with specialized domain knowledge significantly improves the quality of embeddings in relevant tasks, which is critical for applications involving retrieval-augmented generation

(RAG). Domain-specific embeddings resonate more with user queries, improving the accuracy and relevance of generated responses. This targeted approach to selecting embeddings helps in mitigating the semantic gap often encountered when applying general models in specialized contexts.

### Evaluation of Other Options

#### A. Pick an embedding model with multilingual support to support potential multilingual user questions

While multilingual capability is beneficial, it does not ensure the quality of embeddings in a specific domain. A model may produce average embeddings for varied languages without addressing the particular nuances of a specialized field, leading to potential misunderstandings in user intent.

#### B. Pick the most recent and most performant open LLM released at the time

Recent and high-performance models may not necessarily cater to specific domain knowledge. Performance in benchmark tests may reflect general capabilities but do not guarantee alignment with specialized applications, risking the effectiveness of the RAG application.

#### D. Pick the embedding model ranked highest on the Massive Text Embedding Benchmark (MTEB) leaderboard

Ranking on benchmarks provides limited insight into domain applicability. A model excelling in general evaluations may lack the contextual understanding required for specific domains, which can detract from the performance in real-world applications requiring nuanced comprehension of content.

### References

<https://huggingface.co/>

<https://arxiv.org/abs/2202.14202>

<https://www.microsoft.com/en-us/research/project/transformers/>

### Question: 66

CertyIQ

A Generative AI Engineer wants their finetuned LLMs in their prod Databricks workspace available for testing in their dev workspace as well. All of their workspaces are Unity Catalog enabled and they are currently logging their models into the Model Registry in MLflow.

What is the most cost-effective and secure option for the Generative AI Engineer to accomplish their goal?

- A. Use an external model registry which can be accessed from all workspaces.
- B. Use MLflow to log the model directly into Unity Catalog, and enable READ access in the dev workspace to the model.
- C. Setup a duplicate training pipeline in dev, so that an identical model is available in dev.
- D. Setup a script to export the model from prod and import it to dev.

### Answer: B

#### Explanation:

B is correct because using MLflow to log the model directly into Unity Catalog and enabling READ access in the dev workspace efficiently balances cost, security, and operational simplicity.

This approach leverages the existing MLflow infrastructure, ensuring that the model is centrally managed and eliminates redundancy. By utilizing Unity Catalog, the organization benefits from fine-grained access controls and governance over data and models, adhering to security best practices. The READ access configuration ensures that the dev workspace can utilize the model without compromising the security protocols established in the production environment. This method promotes collaborative testing while minimizing unnecessary costs associated with duplicating resources.



### Critique of Other Options:

**A. Use an external model registry:** While an external registry could centralize model storage, it introduces additional complexity and potential latencies. Moreover, integrating external registries with existing Unity Catalog structures raises security concerns, as sensitive models might be exposed to non-secured environments.

**C. Setup a duplicate training pipeline in dev:** This option entails significant resource duplication and additional maintenance overhead. Running duplicate pipelines not only incurs higher operational costs but also risks inconsistency between models in production and development, potentially leading to discrepancies in model performance.

**D. Setup a script to export the model from prod and import it to dev:** Although this option enables model sharing, it is the least efficient and most error-prone. This method requires manual management and could lead to version control issues, diminishing the advantages of using an automated model registry.

In summary, option B effectively aligns with principles of operational efficiency and security, addressing both immediate testing requirements and long-term model management.

References:

<https://www.databricks.com/glossary/unity-catalog>

<https://www.mlflow.org/docs/latest/index.html>

<https://www.databricks.com/solutions/machine-learning>

### Question: 67

CertyIQ

A Generative AI Engineer has just deployed an LLM application at a manufacturing company that assists with answering customer service inquiries. They need to identify the key enterprise metrics to monitor the application in production.

Which is NOT a metric they will implement for their customer service LLM application in production?

- A. Massive Multi-task Language Understanding (MMLU) score
- B. Number of customer inquiries processed per unit of time
- C. Factual accuracy of the response
- D. Time taken for LLM to generate a response

**Answer: A**

**Explanation:**

The correct answer is A, Massive Multi-task Language Understanding (MMLU) score. Here's why:

MMLU is a benchmark used to evaluate the general knowledge and reasoning abilities of large language models (LLMs) on a wide range of subjects, typically before deployment or during model development. It assesses the pre-training performance of a model across multiple tasks. It's not something you would continuously monitor in a live, production environment specifically for a customer service application.

The other options (B, C, and D) are directly relevant to the performance and quality of the customer service LLM application in a real-world production setting. "Number of customer inquiries processed per unit of time" (B) helps gauge the application's throughput and scalability. "Factual accuracy of the response" (C) is critical to ensuring the application provides correct and reliable information, maintaining customer trust. "Time taken for LLM to generate a response" (D) directly impacts the user experience; faster responses lead to better customer satisfaction. Monitoring these metrics allows engineers to identify and address performance bottlenecks, factual errors, or slow response times, ensuring the application meets business requirements.

and provides a positive customer experience. These production metrics allow for monitoring the model's serving quality, identifying data drift, and trigger model retraining if necessary. Therefore, MMLU, while important during development, isn't a key production metric.

Further Reading:

**Evaluating LLMs:** <https://www.promptingguide.ai/introduction/evaluating>

**Generative AI Production Monitoring:** (Search on relevant cloud provider sites like AWS, Azure, GCP for "Generative AI Model Monitoring" for service-specific guidance)

## Question: 68

CertyIQ

Generative AI Engineer is helping a cinema extend its website's chat bot to be able to respond to questions about specific showtimes for movies currently playing at their local theater. They already have the location of the user provided by location services to their agent, and a Delta table which is continually updated with the latest showtime information by location. They want to implement this new capability in their RAG application.

Which option will do this with the least effort and in the most performant way?

- A. Create a Feature Serving Endpoint from a FeatureSpec that references an online store synced from the Delta table. Query the Feature Serving Endpoint as part of the agent logic / tool implementation.
- B. Query the Delta table directly via a SQL query constructed from the user's input using a text-to-SQL LLM in the agent logic / tool implementation.
- C. Set up a task in Databricks Workflows to write the information in the Delta table periodically to an external database such as MySQL and query the information from there as part of the agent logic / tool implementation.
- D. Write the Delta table contents to a text column, then embed those texts using an embedding model and store these in the vector index. Look up the information based on the embedding as part of the agent logic / tool implementation.

**Answer: A**

**Explanation:**

Option A is correct because it leverages a Feature Serving Endpoint, ensuring efficient access to real-time data from the Delta table with minimal latency.

Choosing this method facilitates seamless integration with existing infrastructure, promoting scalability when retrieving dynamic showtime information from various locations. By employing a Feature Serving Endpoint that interfaces directly with an online store synchronized from the Delta table, the architecture reduces the number of operational steps required for querying data, thus enhancing performance. Additionally, this approach minimizes the complexity involved in transforming user queries into SQL syntax, thereby reducing computational overhead while enhancing response times.

**Evaluation of Other Options:**

**Option B:** Querying the Delta table directly via a text-to-SQL LLM would introduce several inefficiencies, including the need for real-time translation of natural language queries into SQL query structure. This adds latency and increases the probability of query generation errors, leading to potentially incorrect or slow responses to user inquiries.

**Option C:** Setting up a task to periodically sync the Delta table to an external database introduces unnecessary overhead, such as maintaining data integrity and syncing delays. This periodic sync makes the system susceptible to outdated information, which is especially detrimental for a service reliant on immediate and accurate data availability like showtimes.

**Option D:** Embedding the contents of the Delta table into a vector index creates an additional layer of

complexity that involves transforming structured data into unstructured formats for similarity searches. This method is computationally intensive and may lead to slower response times due to vector lookups, making it less efficient for real-time applications.

In conclusion, option A offers the most efficient and performant means to extend the chatbot's capabilities by integrating directly with the Delta table through a Feature Serving Endpoint, whereas the alternatives introduce complexities or delays detrimental to user experience.

References:

<https://databricks.com/product/databricks-unified-analytics-platform>

<https://www.microsoft.com/en-us/research/project/spark/>

<https://www.oreilly.com/library/view/streaming-architecture/9781491960306/>

### Question: 69

CertyIQ

Generative AI Engineer needs to build an LLM application that can understand medical documents, including recently published ones. They want to select an open model available on HuggingFace's model hub.

Which step is most appropriate for selecting an LLM?

- A. Pick any model in the Mistral family, as Mistral models are good with all types of use cases
- B. Select a model based on the highest number of downloads, as this indicates popularity, reliability, and general suitability
- C. Select a model that is most recently uploaded, as this indicates the model is the newest and highly likely to be the most performant
- D. Check for the model and training data description to identify if the model is trained on any medical data.

**Answer: D**

**Explanation:**

D. Check for the model and training data description to identify if the model is trained on any medical data.

Medical documents contain domain-specific language, terminology, and structure

Models trained or fine-tuned on medical or biomedical corpora:

Understand clinical terms

Interpret research papers more accurately

Reduce hallucinations in a high-risk domain

Hugging Face model cards explicitly describe:

Training data sources

Intended use cases

Limitations and risks

Selecting based on this information is the right-first-time approach.

### Question: 70

CertyIQ

Generative AI Engineer is building a RAG application that answers questions about technology-related news articles. The source documents may contain a significant amount of irrelevant content, such as advertisements, sports news, or entertainment news.

Which approach is NOT advisable for building a RAG application focused on answering technology-only questions?

- A. Include in the system prompt that the application is not supposed to answer any questions unrelated to technology.
- B. Filter out irrelevant news articles in the retrieval process.
- C. Keep all news articles because the RAG application needs to understand non-technological content to avoid answering questions about them.
- D. Filter out irrelevant news articles in the upstream document database.

**Answer: C**

**Explanation:**

The correct answer is C: Keep all news articles because the RAG application needs to understand non-technological content to avoid answering questions about them. Here's why:

RAG (Retrieval-Augmented Generation) applications are designed to provide answers based on relevant documents. Including irrelevant content defeats the purpose of efficient and accurate information retrieval. The presence of advertisements, sports news, or entertainment articles would dilute the relevant technology news, leading to potentially inaccurate or irrelevant responses.

Option A, including a constraint in the system prompt, is a good approach. It instructs the language model to avoid non-technology topics, acting as a safety net.

Option B, filtering irrelevant articles during retrieval, directly addresses the problem of content pollution. By ensuring only relevant articles are considered, the RAG model can produce better answers.

Option D, filtering irrelevant articles in the document database, is a proactive and preferred approach. It prevents irrelevant content from even being considered, improving overall efficiency. It's akin to data cleansing.

Keeping all articles (Option C) is counterproductive and resource-intensive. It would require the model to process and understand irrelevant content, increasing computational cost without providing any benefit. While the model could theoretically learn to ignore these sections, it's a much less efficient approach than pre-filtering. It adds noise to the data, making it harder for the model to find the relevant information.

RAG benefits from focused and clean data. The less irrelevant information, the better the RAG application can perform. Effective RAG implementation emphasizes relevant data ingestion, accurate information retrieval, and prompt engineering.

[LangChain RAG documentation](#)[Cohere RAG tutorial](#)

## Question: 71

CertyIQ

A Generative AI Engineer is building a RAG application that will rely on context retrieved from source documents that are currently in HTML format. They want to develop a solution using the least amount of lines of code.

Which Python package should be used to extract the text from the source documents?

- A. pytesseract
- B. numpy
- C. pypdf2

**Answer: D****Explanation:**

D is correct because BeautifulSoup is specifically designed for parsing HTML documents and extracting text with minimal lines of code.

BeautifulSoup is a highly efficient and versatile library tailored for HTML parsing. Its design allows developers to easily navigate the parse tree of HTML documents, making it straightforward to locate and extract textual content. In the context of a Retrieval-Augmented Generation (RAG) application, which relies heavily on context from source documents, the efficiency and simplicity provided by BeautifulSoup are vital. The library's implementation requires minimal boilerplate code, thus aligning perfectly with the requirement for a solution that utilizes the least amount of lines of code.

**Evaluation of Incorrect Options:**

**A. pytesseract:** While this library is useful for Optical Character Recognition (OCR) and extracting text from images, it is unsuitable for direct HTML extraction. HTML content is already textual, and using OCR would not only reduce efficiency but also add unnecessary complexity.

**B. numpy:** Numpy is a powerful library for numerical computations and array manipulations. It does not possess functionality for text extraction from HTML and thus is irrelevant to the task of context retrieval from source documents.

**C. pypdf2:** This library is designed for handling PDF files rather than HTML. As the problem specifically pertains to HTML documents, Pypdf2 would not be applicable for this task, leading to a misalignment with the requirements stated.

In conclusion, the optimal choice for extracting text from HTML documents, with a focus on efficiency and minimal code, is undoubtedly BeautifulSoup.

**References:**

<https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

<https://pypi.org/project/beautifulsoup4/>

<https://realpython.com/beautiful-soup-web-scraper-python/>

**Question: 72**

A Generative AI Engineer is building a RAG application for answering employee questions on company policies.

What are the steps needed to build this RAG application and deploy it?

A. Ingest documents from a source -> Index the documents and save to Vector Search -> User submits queries against an LLM -> LLM retrieves relevant documents -> Evaluate model -> LLM generates a response -> Deploy it using Model Serving

B. User submits queries against an LLM -> Ingest documents from a source -> Index the documents and save to Vector Search -> LLM retrieves relevant documents -> LLM generates a response -> Evaluate model -> Deploy it using Model Serving

C. Ingest documents from a source -> Index the documents and save to Vector Search -> Evaluate model -> Deploy it using Model Serving -> User submits queries against an LLM -> LLM retrieves relevant documents -> LLM generates a response

D. Ingest documents from a source -> Index the documents and save to Vector Search -> User submits queries against an LLM -> LLM retrieves relevant documents -> LLM generates a response -> Evaluate model -> Deploy it using Model Serving

**Answer: D**

**Explanation:**

Here's a detailed justification for why option D is the most logical sequence of steps for building and deploying a RAG application for employee policy questions, compared to the other options:

**Why Option D is Correct:**

Option D represents the most logical and standard workflow for building and deploying a Retrieval Augmented Generation (RAG) application. Let's break it down step-by-step:

1. **Ingest documents from a source:** This is the foundational step. You need to gather all the company policy documents (PDFs, Word documents, etc.) that contain the information employees will be querying. This data is the knowledge base for your RAG system.
2. **Index the documents and save to Vector Search:** Once you have the documents, you need to process them into a format suitable for retrieval. This typically involves:  
  
Chunking: Breaking the documents into smaller, more manageable pieces.  
Embedding: Using a pre-trained language model to convert each chunk into a vector representation (embedding). These embeddings capture the semantic meaning of the text.  
Indexing and Storage: Storing these embeddings in a vector database or vector search index. Vector databases (like ChromaDB, Pinecone, or Weaviate) are designed for efficient similarity search, allowing you to find the most relevant document chunks for a given query.
3. **User submits queries against an LLM:** This represents the interaction point. Employees ask questions about company policies. The query needs to be processed and potentially rephrased to better match the semantic understanding of the indexed documents.
4. **LLM retrieves relevant documents:** The user's query (or its rephrased version) is also converted into an embedding. A similarity search is performed within the vector database to find the document chunks that are most semantically similar to the query. This step retrieves the context needed to provide an accurate answer.
5. **LLM generates a response:** The retrieved document chunks (context) are combined with the original user query and fed into a Large Language Model (LLM) like GPT-3, GPT-4, or a similar open-source model. The LLM uses its general knowledge and the provided context to generate a coherent and informative answer to the employee's question.
6. **Evaluate model:** Crucial to assess the RAG pipeline's performance. Metrics can include accuracy, relevance, and coherence. Based on evaluation, adjust parameters (chunk size, embedding model, etc.) to improve performance.
7. **Deploy it using Model Serving:** Finally, the RAG application (including the LLM, the vector database, and the necessary code for processing queries and generating responses) needs to be deployed so that employees can access it. Databricks Model Serving is a suitable platform for this, allowing you to create an API endpoint for your RAG application.

**Why Other Options are Incorrect:**

**Option A:** Places evaluation before generation. Evaluation requires a response to assess. Also, querying occurs before indexing, which is illogical.

**Option B:** Starts with the user querying the LLM before the documents have even been ingested and indexed. This makes no sense because the LLM has no relevant context to draw upon.

**Option C:** Places evaluation and deployment before user interaction. This makes it impossible to evaluate the system's response to real-world queries and iterate on its design.

## Authoritative Links for Further Research:

**Databricks Documentation on Model Serving:** <https://www.databricks.com/product/machine-learning/model-serving>

**RAG Architecture:** <https://www.pinecone.io/learn/retrieval-augmented-generation/> (Pinecone is a popular vector database provider; their documentation explains RAG concepts well.)

**Vector Databases:** Research options like ChromaDB, Weaviate, and FAISS for storing and querying embeddings efficiently.

## Question: 73

CertyIQ

A Generative AI Engineer who was prototyping an LLM system accidentally ran thousands of inference queries against a Foundation Model endpoint over the weekend. They want to take action to prevent this from unintentionally happening again in the future.

What action should they take?

- A. Use prompt engineering to instruct the LLM endpoints to refuse too many subsequent queries.
- B. Require that all development code which interfaces with a Foundation Model endpoint must be reviewed by a Staff level engineer before execution.
- C. Build a pyfunc model which proxies to the Foundation Model endpoint and add throttling within the pyfunc model.
- D. Configure rate limiting on the Foundation Model endpoints.

**Answer: D**

### Explanation:

D is correct because configuring rate limiting on the Foundation Model endpoints directly controls the number of requests made, thereby preventing unintentional overload.

**Reasoning:** Rate limiting serves as a robust mechanism for managing API usage by restricting the number of requests that can be made within a specific timeframe. This proactive approach minimizes potential costs associated with excessive API calls, enhances overall system stability, and ensures fair access to resources. By implementing rate limiting, the Generative AI Engineer can safeguard endpoints against unanticipated spikes in demand, which might arise from faulty logic or unintended interactions during the prototyping phase.

### Evaluation of Other Options:

#### **A. Use prompt engineering to instruct the LLM endpoints to refuse too many subsequent queries.**

This approach lacks the granularity and control provided by rate limiting. Prompt engineering does not ensure that the system can manage external requests effectively and may still allow unintended queries through if the model is not properly trained to interpret all prompt variations.

#### **B. Require that all development code which interfaces with a Foundation Model endpoint must be reviewed by a Staff level engineer before execution.**

While code reviews are beneficial, they are reactive rather than preventive. This option may decrease the speed of development and does not directly mitigate the risk of excessive API calls occurring in the first place.

#### **C. Build a pyfunc model which proxies to the Foundation Model endpoint and add throttling within the pyfunc model.**

Although this solution introduces a form of throttling, it adds unnecessary complexity to the architecture. It requires additional maintenance and could introduce bottlenecks, whereas employing rate limiting on the



endpoint itself is a more straightforward and effective solution.

#### References:

<https://cloud.tencent.com/en/docs/guide/faq-what-is-rate-limiting>

<https://www.baeldung.com/spring-rate-limiting>

<https://www.nerdio.net/blog/rate-limiting-in-the-api-development-world/>

### Question: 74

CertyIQ

A Generative AI Engineer is setting up a Databricks Vector Search that will lookup news articles by topic within 10 days of the date specified. An example query might be “Tell me about monster truck news around January 5th 1992”. They want to do this with the least amount of effort.

How can they set up their Vector Search index to support this use case?

- A. Create separate indexes by topic and add a classifier model to appropriately pick the best index.
- B. Include metadata columns for article date and topic to support metadata filtering.
- C. Pass the query directly to the vector search index and return the best articles.
- D. Split articles by 10 day blocks and return the block closest to the query.

#### Answer: B

#### Explanation:

Option B is correct because including metadata columns for article date and topic facilitates precise filtering within the Vector Search, enhancing the retrieval of relevant news articles.

In generative AI applications, efficient retrieval hinges on the use of structured metadata, which allows for contextual filtering alongside vector similarity. By supporting both topic and temporal constraints, the proposed design enables the application to focus on articles that meet the specified criteria – within 10 days of the given date. This avoids unnecessary computational overhead and improves user experience by streamlining the retrieval process.

#### Evaluation of Other Options:

**Option A:** Creating separate indexes by topic introduces complexity in managing multiple indexes and may lead to performance inefficiencies. It complicates the query structure and does not facilitate timely retrieval across date ranges. The scalability of maintaining numerous topic-based indexes can hinder overall operational efficiency.

**Option C:** Passing the query directly to the vector search index neglects the need for contextual granularity that metadata filtering provides. Such an approach may retrieve irrelevant results, as it primarily relies on similarity rather than specific criteria such as article date or topic, leading to less accurate outcomes.

**Option D:** Splitting articles into 10-day blocks could potentially result in loss of relevant context, especially if articles span multiple blocks. This method could complicate retrieval, as the system would have to determine the optimal block rather than directly filtering relevant articles based on timestamp criteria.

In summary, leveraging metadata allows for efficient scaling and precise retrieval, ensuring that users receive the most relevant articles based on both topic and temporal relevance.

#### References:

[https://docs.databricks.com/\\_static/notebooks/azure-databricks-architecture.html](https://docs.databricks.com/_static/notebooks/azure-databricks-architecture.html)

<https://www.microsoft.com/en-us/research/publication/foundation-models-auto-regressive-language->

### Question: 75

CertyIQ

A Generative AI Engineer developed an LLM application using the pay-per-token Foundation Model API. Now that the application is ready to be deployed, they would like to ensure the model endpoint can serve high incoming volumes of requests in production.

What should the Generative AI Engineer consider?

- A. Switch to using External Models instead
- B. Throttle the incoming batch of requests manually to avoid rate limiting issues
- C. Change to a model with a fewer number of parameters in order to reduce hardware constraint issues
- D. Deploy the model using provisioned throughput as it comes with performance guarantees

**Answer: D**

#### Explanation:

D is correct because deploying the model using provisioned throughput ensures reliable handling of high request volumes with performance guarantees.

In production environments, especially for applications reliant on Foundation Model APIs, the ability to maintain availability under load is crucial. Provisioned throughput enables predictable performance by allocating a set amount of resources that can accommodate traffic spikes, thereby mitigating risks associated with unpredictable demand. This model reduces latency and improves user experience by allowing the system to handle numerous requests concurrently without performance degradation.

#### Evaluation of Other Options:

- A. Switching to an External Model might introduce dependency on external services, which can result in additional latency and potential unreliability, especially during peak usage times.
- B. Throttling incoming requests manually may prevent rate limiting issues; however, it does not scale well and can lead to poor user experience when requests are delayed or rejected.
- C. Changing to a model with a fewer number of parameters could simplify computations but may also undermine the model's effectiveness and performance, leading to lower overall quality and potentially compromising user satisfaction.

In conclusion, selecting option D ensures a strategic approach to scaling and managing requests efficiently, balancing performance and reliability.

References:

- <https://aws.amazon.com/architecture/what-is-provisioned-throughput/>
- <https://cloud.google.com/compute/docs/instances/provisioning>
- <https://www.microsoft.com/en-us/research/publication/performance-studies-for-api-services/>

### Question: 76

CertyIQ

A Generative AI Engineer at a home appliance company has been asked to design an LLM based application that accomplishes the following business objective: answer customer questions on home appliances using the associated instruction manuals.

Which set of high-level tasks should the Generative AI Engineer's system perform?

- A. Split instruction manuals into chunks and embed into a vector store. Use the question to retrieve best matched chunks of manual, and use the LLM to generate a response to the user based upon the manual retrieved.
- B. Create an interaction matrix of historical user questions and appliance instruction manuals. Use ALS to factorize the matrix and create embeddings. Calculate the embeddings of new queries and use them to find the best manual. Use an LLM to generate a response to the question based upon the manual retrieved.
- C. Calculate averaged embeddings for each instruction manual, compare embeddings to user query to find the best manual. Pass the best manual with user query into an LLM with a large context window to generate a response to the employee.
- D. Use an LLM to summarize all of the instruction manuals. Provide summaries of each manual and user query into an LLM with a large context window to generate a response to the user.

**Answer: A**

**Explanation:**

A is correct because it effectively utilizes vector embeddings to optimize the retrieval of relevant information from instruction manuals, which enhances the efficiency and accuracy of the response generation.

#### **Justification for Option A**

The approach outlined in option A is grounded in effective information retrieval techniques, utilizing Natural Language Processing (NLP) methods to split lengthy instruction manuals into manageable chunks. By embedding these chunks into a vector store, the system can efficiently retrieve the most relevant sections in response to customer inquiries. This targeted approach minimizes the cognitive load on the LLM, allowing it to generate precise responses based specifically on the content of the retrieved manual segments.

#### **Evaluation of Other Options**

**Option B:** While creating an interaction matrix might enable understanding of user behavior, the use of Alternating Least Squares (ALS) for factorization introduces unnecessary complexity and overhead. This method is more suited for collaborative filtering rather than direct document retrieval, reducing the effectiveness of the LLM to provide contextually accurate responses based on specific pieces of the manual.

**Option C:** Averaging embeddings might provide a generalized representation of manuals but lacks the granularity needed for effective retrieval. This could lead to responses that are too vague and not aligned with the user's specific question, ultimately compromising the quality of customer interactions.

**Option D:** Summarizing manuals may provide a broader overview, but it ignores the depth of information often required to accurately address specific customer queries. This method risks oversimplifying pertinent details that are essential for comprehensive understanding, leading to potentially incorrect or less informative responses.

#### **References**

<https://www.microsoft.com/en-us/research/publication/effective-information-retrieval-methods/>  
<https://towardsdatascience.com/a-beginners-guide-to-nlp-and-retrieval-based-chatbots-9f1fdc467e87>  
<https://www.aclweb.org/anthology/2020.lrec-1.109.pdf>

#### **Question: 77**

**CertyIQ**

A Generative AI Engineer is developing an LLM application to interact with users to provide personalized movie recommendations.

Given the potential for malicious user inputs, which technique would be most effective in safeguarding the application?

- A.Reduce the time that the users can interact with the LLM
- B.Increase the amount of compute that powers the LLM to process input faster
- C.Ask the LLM to remind the user that the input is malicious but continue the conversation with the user
- D.Implement a safety filter that detects any harmful inputs and ask the LLM to respond that it is unable to assist

**Answer: D**

**Explanation:**

Implementing a safety filter that detects any harmful inputs and directs the LLM to respond that it is unable to assist is the most effective safeguard against malicious user inputs.

A safety filter acts as a proactive defensive mechanism, systematically analyzing user inputs for harmful content before they can influence the LLM's responses. This method not only ensures that inappropriate content is mitigated, but it also protects the integrity and security of the application. By guiding the LLM to communicate its inability to assist under these circumstances, it reinforces user awareness and fosters a safer interaction environment.

**Evaluation of Other Options:**

**Option A: Reduce the time that users can interact with the LLM and increase compute resources.**

While reducing interaction time may limit exposure to harmful inputs, it is an inefficient approach that may frustrate users and does not address the root of the malicious input problem. Increased compute power does not correlate with mitigating harmful user input risks.

**Option B: Ask the LLM to remind the user that the input is malicious but continue the conversation.**

This option risks normalizing harmful content by allowing the conversation to proceed without firm boundaries. By acknowledging malicious inputs but still engaging in dialogue, it may inadvertently encourage further similar interactions rather than unequivocally rejecting them.

**Option C: Implement a safety filter that detects harmful inputs and state inability to assist.**

This option directly mitigates the risk of malicious content by preventing it from entering the response loop. It prioritizes safety over user engagement, reinforcing a responsible approach to interaction design.

Effective and adaptive safety measures are vital in generative AI applications, safeguarding against risk while ensuring user experience remains intact.

**Reference:**

<https://www.ibm.com/cloud/learn/generative-ai>

[target="\\_blank">https://www.ibm.com/cloud/learn/generative-ai](https://www.ibm.com/cloud/learn/generative-ai)

<https://www.microsoft.com/en-us/security/blog/2023/02/02/protecting-against-ai-generated-content-and-its-misuse/> target="\_blank"><https://www.microsoft.com/en-us/security/blog/2023/02/02/protecting-against-ai-generated-content-and-its-misuse/>

<https://www.nist.gov/publications/generative-audit-returns-recommendations-implementing-llm-technology-safely>

target="\_blank"><https://www.nist.gov/publications/generative-audit-returns-recommendations-implementing-llm-technology-safely>

## Question: 78

A Generative AI Engineer received the following business requirements for an internal chatbot. The internal chatbot needs to know what types of questions the user asks and route them to appropriate models to answer the questions. For example, the user might ask about historical failure rates of a specific electrical part. Another user might ask about how to troubleshoot a piece of electrical equipment.

Available data sources include a database of electrical equipment PDF manuals and also a table with information on when an electrical part experiences failure.

Which workflow supports such a chatbot?

- A. Parse the electrical equipment PDF manuals into a table of question and response pairs. That way, the same chatbot can query tables easily to answer questions about both historical failure rates and equipment troubleshooting.
- B. The chatbot should be implemented as a multi-step LLM workflow. First, identify the type of question asked, then route the question to the appropriate model. If it's a historical failure rate question, send the query to a text-to-SQL model. If it's a troubleshooting question, then send the query to another model that summarizes the equipment-specific document and generates the response.
- C. There should be two different chatbots handling different types of user queries.
- D. The table with electrical part failures should be converted into a text document first. That way, the same chatbot can use the same document retrieval process to generate answers regardless of question types.

### Answer: B

#### Explanation:

Option B is correct because it effectively utilizes a multi-step workflow to intelligently identify and route user inquiries to specialized models, enhancing response accuracy and relevance.

#### Justification for Option B:

1. **Granularity in Problem Solving:** By first identifying the type of question, the workflow aligns with natural language processing best practices, ensuring that complex queries are handled by the most appropriate model.
2. **Model Specialization:** The workflow optimizes performance by routing historical failure rate inquiries to a text-to-SQL model, which excels at database queries, while troubleshooting questions go to a model tailored for summarizing equipment manuals. This maximizes the effectiveness of each model based on the inquiry type.
3. **Scalability:** The multi-step approach allows for easy additions of new question types and corresponding models, facilitating rapid adaptation to changing information and requirements.

#### Evaluation of Other Options:

**Option A:** Parsing PDF manuals into a table of question and response pairs oversimplifies the complexities of user queries and limits the flexibility required for varied inquiries. It lacks the dynamic routing approach that enhances accuracy based on query type.

**Option C:** Having two separate chatbots introduces unnecessary complexity and redundancy in the chatbot ecosystem. This can lead to inconsistent user experiences and complicates maintenance and updates.

**Option D:** Converting the failure data into a text document does not leverage the structured data present, which diminishes the efficiency of querying for specific insights. It compromises the responsiveness and utility of the chatbot by forcing a uniform retrieval process across varied question types.

#### References:

<https://www.oreilly.com/library/view/deep-learning-for/9781492035624/>

<https://www.oreilly.com/library/view/programming-deep-learning/9781492076108/>

### Question: 79

CertyIQ

A Generative AI Engineer is building a system that will answer questions on currently unfolding news topics. As such, it pulls information from a variety of sources including articles and social media posts. They are concerned about toxic posts on social media causing toxic outputs from their system.

Which guardrail will limit toxic outputs?

- A.Reduce the amount of context items the system will include in consideration for its response.
- B.Use only approved social media and news accounts to prevent unexpected toxic data from getting to the LLM.
- C.Log all LLM system responses and perform a batch toxicity analysis monthly.
- D.Implement rate limiting.

**Answer: B**

**Explanation:**

B is correct because using only approved social media and news accounts minimizes the risk of integrating toxic data into the LLM's input.

**Reasoning:** Utilizing selected, vetted sources ensures that the information fed into the model adheres to specific quality standards, reducing the likelihood of encountering and thus reproducing toxic language or sentiments. This method fundamentally enhances the reliability and safety of the output generated by the LLM. The reinforcement of trusted, authoritative sources serves as a proactive approach against the infusion of toxic data, ensuring a more controlled and monitored input stream.

**Critical Evaluation of Other Options:**

**A. Reduce the amount of context items the system will include in consideration for its response.**

This approach assumes that quantity is the sole factor contributing to toxicity, which overlooks the significant impact of the content's quality. Reducing context does not inherently filter out toxicity and may lead to less informative responses.

**C. Log all LLM system responses and perform a batch toxicity analysis monthly.**

While logging responses may yield insights over time, it is a reactive measure rather than proactive. This could allow toxic content to influence user interactions and perceptions in the interim, before any corrective actions are implemented. Furthermore, the lag in analysis could leave significant vulnerabilities unaddressed.

**D. Implement rate limiting.**

Rate limiting primarily serves to manage system load and user capacity, but it does not influence the quality or toxicity of the content being produced. It neither prevents the underlying issues associated with bad data nor does it enhance the integrity of the content processed.

**References:**

[https://www.researchgate.net/publication/343204289\\_Tackling\\_toxicity\\_in\\_machine\\_learning\\_models](https://www.researchgate.net/publication/343204289_Tackling_toxicity_in_machine_learning_models)

<https://arxiv.org/abs/1908.08636>

<https://www.frontiersin.org/articles/10.3389/frai.2020.00008/full>

### Question: 80

CertyIQ

A Generative AI Engineer has created a RAG application which can help employees interpret HR documentation.

The prototype application is now working with some positive feedback from internal company testers. Now the Generative AI Engineer wants to formally evaluate the system's performance and understand where to focus their efforts to further improve the system

How should the Generative AI Engineer evaluate the system?

- A. Use ROUGE score to comprehensively evaluate the quality of the final generated answers.
- B. Use an LLM-as-a-judge to evaluate the quality of the final answers generated.
- C. Curate a dataset that can test the retrieval and generation components of the system separately. Use MLflow's built in evaluation metrics to perform the evaluation on the retrieval and generation components.
- D. Benchmark multiple LLMs with the same data and pick the best LLM for the job.

**Answer: C**

**Explanation:**

The most appropriate evaluation strategy for a RAG application's performance, especially when transitioning from prototype to formal evaluation, is to curate a dataset that separately tests retrieval and generation components, coupled with MLflow's evaluation metrics. This approach allows for granular insight into system weaknesses, enabling targeted improvements.

Option A, using ROUGE score alone, is insufficient. ROUGE focuses on text overlap between generated and reference texts. While it can highlight some issues, it doesn't directly measure retrieval accuracy or semantic correctness which are vital in a RAG system. Furthermore, relying solely on ROUGE overlooks the complexity of generative AI and the nuances of human language.

Option B, utilizing an LLM-as-a-judge, can be useful for qualitative assessment, but it introduces subjectivity and lacks the repeatability required for rigorous performance tracking. An LLM judge also does not evaluate the retrieval stage in isolation.

Option D, benchmarking multiple LLMs, is more focused on model selection than a comprehensive system evaluation. It's a step that might be done after understanding the specific issues within the RAG application architecture.

Option C is the most thorough. First, creating a dedicated dataset enables controlled testing. This dataset should include questions relevant to the HR documentation and expected answers. Second, the dataset allows separate evaluation of the retrieval component (does the system retrieve the relevant documents?) and the generation component (does the LLM generate a relevant answer from the retrieved context?). Specifically, relevant metrics for retrieval might include Hit Rate and Mean Reciprocal Rank (MRR). For generation, metrics like context relevance, faithfulness, answer relevance, and answer quality can be measured using LLM-based evaluation or traditional metrics. Finally, MLflow provides a structured framework for tracking these metrics over time, making it easy to monitor improvements and compare different versions of the RAG system.

By isolating the components and using appropriate metrics, the engineer can pinpoint whether the issues lie in retrieving the correct HR document or in the LLM's ability to synthesize information effectively. This targeted approach to testing, using a controlled dataset, yields actionable insights, unlike relying on blanket metrics like ROUGE or subjective evaluations from LLMs acting as judges.

For further research, explore:

**MLflow evaluation metrics:** <https://mlflow.org/docs/latest/llm-evaluate/index.html>

**Evaluating Retrieval-Augmented Generation:** <https://arxiv.org/abs/2305.14653> (This is just one example of many relevant papers; search for more on Arxiv)

**Metrics for Information Retrieval:** Search "information retrieval evaluation metrics" for classic IR metrics like MRR.



### Question: 81

CertyIQ

A Generative AI Engineer is using LangChain to assist a museum in classifying documents and using this code:

```
from langchain import PromptTemplate

template_text = """
You are an automated archeological assistant helping identify documents relevant to ancient Egypt. If the
document is relevant return "relevant", else return "irrelevant" with no additional text.
Document:" {document_input}"
Response:
"""

prompt_template = PromptTemplate(
 input_variables=["document_input"]
)

completed_prompt = prompt_template.format(
 document_input=document_input
)
```

Their code results in an error.

What do they need to change in order to fix this template?

- A. Provide an LLM argument to PromptTemplate()
- B. Provide template and LLM arguments to PromptTemplate()
- C. Omit PromptTemplate(), it is only used for multi-part templates
- D. Provide a template argument to PromptTemplate()

**Answer: D**

**Explanation:**

D. Provide a template argument to PromptTemplate()

The question refers to a LangChain or LLM application scenario where a PromptTemplate is being created incorrectly — likely missing the template argument, which is required.

A PromptTemplate in LangChain defines the structure of a prompt with placeholders that can be filled with variables at runtime.

### Question: 82

CertyIQ

A Generative AI Engineer has been reviewing issues with their company's LLM based question-answering assistant and has determined that a technique called prompt chaining could help alleviate some performance concerns. However, to suggest this to their team, they have to clearly explain how it works and how it can benefit their question-answering assistant.

Which explanation do they communicate to the team?

- A. It allows you to break down complex tasks into multiple independent subtasks. This enables the assistant to generate more comprehensive and accurate responses.
- B. It allows you to reduce the latency of your applications. By having multiple chains participating in the response as a chain, you increase the rate at which the response is generated.
- C. It allows you to decrease the effort involved in crafting a prompt. Chains make it possible to reuse prompt text across multiple different use cases.

D. It reduces the average cost of a typical request. Chains make more efficient use of the tokens produced to generate higher quality responses with fewer tokens.

**Answer: A**

**Explanation:**

Here's a detailed justification for why option A is the most accurate explanation of prompt chaining and its benefits for a question-answering assistant:

Prompt chaining, in the context of Large Language Models (LLMs) and Generative AI, is a technique where the output of one prompt is used as the input to another prompt, and this process can be repeated in a sequence. This enables the breakdown of a complex task into smaller, more manageable subtasks. Instead of asking an LLM to directly answer a multifaceted question, the problem is decomposed. For example, the first prompt might extract relevant information from a document, the second prompt could summarize that information, and the third prompt could use the summary to answer the user's question.

This modular approach of prompt chaining offers several advantages. First, it can improve the accuracy and coherence of the responses, as each subtask can be optimized independently. Second, it simplifies debugging and maintenance because issues can be traced to specific stages in the chain. Third, it can improve the overall quality of responses for question answering systems, which often require multiple steps of reasoning or information retrieval.

Option B is incorrect because while parallel processing may exist, prompt chaining's primary focus is not latency reduction through parallel chains. Option C is also incorrect; although prompt templates exist, the main advantage of prompt chaining is task decomposition, not prompt reuse. Option D is partially true as carefully crafted prompts can optimize token usage but the primary benefit isn't cost reduction, it's improved quality of results with complex tasks.

Therefore, option A is the correct explanation. It accurately describes prompt chaining as a means to break down complex tasks, which results in more comprehensive and accurate responses from the question-answering assistant.

**Supporting Resources:**

**LangChain Documentation:** [https://python.langchain.com/docs/get\\_started/introduction](https://python.langchain.com/docs/get_started/introduction) (LangChain is a popular framework for building LLM-powered applications, and its documentation extensively covers prompt chaining.)

**Towards Data Science Article on Prompt Engineering:** <https://towardsdatascience.com/prompt-engineering-guide-to-mastering-large-language-models-like-chatgpt-4e96305c4093> (While not solely about prompt chaining, this provides context on the importance of well-structured prompts for LLMs.)

**Question: 83**

**CertyIQ**

An AI developer team wants to fine tune an open-weight model to have exceptional performance on a code generation use case. They are trying to choose the best model to start with. They want to minimize model hosting costs, and are using Huggingface model cards and spaces to explore models.

Which TWO model attributes and metrics should the team focus on to make their selection? (Choose two.)

- A. Big Code Models Leaderboard
- B. Number of model parameters
- C. MTEB Leaderboard
- D. Chatbot Arena Leaderboard
- E. Number of model downloads last month

**Answer: AB**

**Explanation:**

The correct answer is A and B: **Big Code Models Leaderboard** and **Number of model parameters**. Here's a justification:

**Big Code Models Leaderboard (A):** This leaderboard on Hugging Face specifically benchmarks models on code generation tasks. It directly addresses the team's primary goal: exceptional performance in code generation. It provides a comparative view, allowing the team to identify models that have already demonstrated strong code generation capabilities. This saves time and resources by pre-selecting models known to perform well in the desired domain. <https://huggingface.co/spaces/bigcode/bigcode-models-leaderboard>

**Number of model parameters (B):** Model size, represented by the number of parameters, significantly impacts hosting costs. Larger models generally require more computational resources (memory, processing power) for serving inferences, leading to higher infrastructure costs on platforms like Databricks. Choosing a model with a parameter count that balances performance and size is crucial for minimizing expenses. Fine-tuning can further optimize the model's performance without necessarily increasing the number of parameters dramatically. Generally, a smaller model with fewer parameters is preferred when cost is a significant constraint.

**Why other options are less relevant:**

**MTEB Leaderboard (C):** MTEB focuses on general text embeddings and is not specific to code generation.

**Chatbot Arena Leaderboard (D):** This leaderboard focuses on general conversational abilities and is not specific to code generation.

**Number of model downloads last month (E):** While popularity can be an indicator, it doesn't guarantee performance in a specific domain like code generation or correlate directly with hosting costs. A widely downloaded model might be large and expensive to host or be used for unrelated tasks.

Therefore, prioritizing the Big Code Models Leaderboard to identify models proficient in code generation and considering the number of model parameters to control hosting costs is the most effective strategy for the AI developer team. Choosing a model with these parameters ensures focus on their project needs and cost reduction.

**Question: 84**

**CertyIQ**

A Generative AI Engineer at an automotive company would like to build a question-answering chatbot to help customers answer specific questions about their vehicles. They have:

- A catalog with hundreds of thousands of cars manufactured since the 1960s
- Historical searches, with user queries and successful matches
- Descriptions of their own cars in multiple languages

They have already selected an open source LLM and created a test set of user queries. They need to discard techniques that will not help them build the chatbot.

Which do they discard?

- A. Setting chunk size to match the model's context window to maximize coverage
- B. Implementing metadata filtering based on car models and years
- C. Fine-tuning an embedding model on automotive terminology
- D. Adding few-shot examples for response generation

**Answer: A**

**Explanation:**

A.Setting chunk size to match the model's context window to maximize coverage.

Chunk size should not be as large as the model's context window

When building a retrieval-augmented Q&A chatbot, chunks must be:

Semantically coherent

Small enough for accurate embeddings

Focused on one topic at a time

If chunks are too large:

Embeddings lose meaning because they represent mixed topics

Retrieval becomes noisy and less accurate

Irrelevant information is pulled into answers ("embedding dilution")

It becomes slower and more expensive to embed and search

So matching the chunk size to the full context window is not only unnecessary but harmful.

### Question: 85

**CertyIQ**

A Generative AI Engineer at a legal firm is designing a RAG system to analyze historical legal case precedents. The system needs to process millions of court opinions and legal documents, already organized by time and topic, to track how interpretations of specific laws have evolved over time. All of these documents are in plain-text. The engineer needs to choose a chunking method that would most effectively preserve continuity and the temporal nature of the cases.

Which method do they choose?

- A. Implement windowed summarization with overlapping chunks.
- B. Implement a hierarchical tree structure, like RAPTOR, to group similar legal concepts.
- C. Implement paragraph level embeddings with each chunk.
- D. Implement sentence level embeddings with each chunk tagged with the time to enable metadata filtering.

**Answer: A**

**Explanation:**

A. Implement windowed summarization with overlapping chunks.

The key requirements are:

Millions of documents

Strong temporal ordering (how interpretations evolve over time)

Preserve continuity of legal reasoning

Documents already organized by time and topic

Windowed summarization with overlap

Maintains narrative continuity across adjacent chunks

Overlap ensures that:

Legal arguments spanning multiple sections aren't broken

Subtle shifts in interpretation are preserved

Especially effective for:

Long court opinions

Multi-stage legal reasoning

Tracking doctrinal evolution over time

This aligns perfectly with legal analysis, where context before and after a ruling matters.

### Question: 86

CertyIQ

A Generative AI Engineer is developing an agent system using a popular agent-authoring library. The agent comprises multiple parallel and sequential chains. The engineer encounters challenges as the agent fails at one of the steps, making it difficult to debug the root cause. They need to find an appropriate approach to research this issue and discover the cause of failure.

Which approach do they choose?

- A. Enable MLflow tracing to gain visibility into each agent's behavior and execution step.
- B. Run MLflow.evaluate to determine root cause of failed step.
- C. Implement structured logging within the agent's code to capture detailed execution information.
- D. Deconstruct the agent into independent steps to simplify debugging.

**Answer: A**

**Explanation:**

A. Enable MLflow tracing to gain visibility into each agent's behavior and execution step.

The engineer is using an agent-authoring library.

The agent has multiple sequential and parallel chains (common in LangChain, LlamaIndex, etc.).

A step is failing, and it's hard to debug the root cause.

They need an appropriate approach to research the issue and see exactly where the failure happens.

### Question: 87

CertyIQ

A Generative AI Engineer is experimenting with using parameters to configure an agent in Mosaic Agent Framework. However, they are struggling to get the agent to respond with relevant information with this configuration:

```
config_dict = {
 "prompt_template": "You are a trivia bot. Generate a trivia question based on the user's input.",
 "prompt_template_input_vars": ["user_input"],
 "model_serving_endpoint": "databricks-dbrx-instruct",
 "llm_parameters": {"temperature": 0.01, "max_tokens": 500},
}
```

Which error is causing the problem?

- A. The prompt does not parse the user's input vars
- B. The prompt does not set the retriever schema
- C. The prompt does not list available agents for the LLM to call
- D. The prompt is not wrapped in ChatModel

**Answer: A**

**Explanation:**

A. The prompt does not parse the user's input vars

This question refers to a LangChain or LLM-based application scenario where a PromptTemplate or chain fails due to a mismatch or omission of input variables.

When using a PromptTemplate, you must define which variables it expects to receive and ensure they are correctly referenced in the template string.

## Question: 88

CertyIQ

A Generative AI Engineer is using LangGraph to define multiple tools in a single agentic application. They want to enable the main orchestrator LLM to decide on its own which tools are most appropriate to call for a given prompt. To do this, they must determine the general flow of the code.

Which sequence will do this?

- A. 1. Define or import the tools 2. Add tools and LLM to the agent 3. Create the ReAct agent
- B. 1. Define or import the tools 2. Define the agent 3. Initialize the agent with ReAct, the LLM, and the tools
- C. 1. Define the tools 2. Load each tool into a separate agent 3. Instruct the LLM to use ReAct to call the appropriate agent
- D. 1. Define the tools inside the agents 2. Load the agents into the LLM 3. Instruct the LLM to use CoT reasoning to determine the appropriate agent

**Answer: B**

**Explanation:**

**B. 1. Define or import the tools 2. Define the agent 3. Initialize the agent with ReAct, the LLM, and the tools.**

To build an agentic application where an LLM orchestrates multiple tools, you must follow a logical sequence that establishes the "brain" (LLM), the "hands" (tools), and the "reasoning loop" (ReAct).

Define or import the tools: Before an LLM can decide which tool to use, it must have access to the function definitions and their schemas (names, descriptions, and input parameters).

Define the agent: In LangGraph, this involves defining the graph structure — specifically the nodes (the LLM's thought process) and the edges (the conditional transitions that decide whether to call a tool or provide a final

answer).

Initialize the agent with ReAct, the LLM, and the tools: The "ReAct" (Reason + Act) pattern is the architectural backbone here. You bind the tools to the LLM so it is "tool-aware." When initialized, the LLM is instructed to output a specific format (usually structured JSON) when it determines a tool is needed, which the orchestrator then executes.

### Question: 89

CertyIQ

All of the following are python APIs used to query Databricks foundation models. When running in an interactive notebook, which of the following libraries does not automatically use the current session credentials?

- A. OpenAI client
- B. REST API via requests library
- C. MLflow Deployments SDK
- D. Databricks Python SDK

**Answer: B**

**Explanation:**

B. REST API via requests library.

Can use the Databricks workspace session token

When run inside a Databricks notebook, auth can be passed automatically via the session context

### Question: 90

CertyIQ

A Generative AI Engineer is deploying a customer-facing, fine-tuned LLM on their public website. Given the large investment the company put into fine tuning this model, and the proprietary nature of the tuning data, they are concerned about model inversion attacks.

Which of the following Databricks AI Security Framework (DASF) risk mitigation strategies are most relevant to this use case?

- A. Implement AI guardrails to allow users to configure and enforce compliance
- B. Leverage Databricks access control lists (ACLs) to configure permissions for accessing models
- C. Use secure model features with Databricks Feature Store
- D. Apply attribute-based access controls (ABAC) to limit unauthorized access

**Answer: A**

**Explanation:**

**A. Implement AI guardrails to allow users to configure and enforce compliance.**

Model inversion attacks occur when an adversary queries a model repeatedly to reconstruct the training or fine-tuning data, potentially exposing your proprietary intellectual property.

Prevention vs. Access: While access controls (Options B and D) are vital for managing who can invoke a model, they do not prevent a legitimate (or authenticated) user from crafting malicious prompts designed to extract training data. AI Guardrails act as a filter between the user and the model.



**Mitigating Data Exfiltration:** Guardrails can be configured to detect and block "extraction attempts" — such as prompts asking the model to repeat large portions of its training set or prompts that look like high-entropy queries typical of model inversion tactics.

**Compliance and Safety:** In the Databricks AI Security Framework (DASF), guardrails are the designated layer for enforcing safety policies that ensure the model behaves within expected boundaries, directly addressing the risks associated with public-facing, fine-tuned models.

### Question: 91

CertyIQ

A team uses Mosaic AI Vector Search to retrieve documents for their Retrieval-Augmented Generation (RAG) pipeline. The search query returns five relevant documents, and the first three are added to the prompt as context. Performance evaluation with Agent Evaluation shows that some lower-ranked retrieved documents have higher context relevancy scores than higher-ranked documents.

Which option should the team consider to optimize this workflow?

- A. Use a reranker to order the documents based on the relevance scores.
- B. Modify the prompt to instruct the LLM to order the documents based on the relevance scores.
- C. Use a different embedding model for computing document embeddings.
- D. Increase the number of documents added to the prompt to improve context relevance.

**Answer: A**

#### Explanation:

The problem highlights a disconnect between the initial ranking of documents retrieved by Mosaic AI Vector Search and their actual relevance to the query's context, as indicated by the Agent Evaluation context relevancy scores. The current approach relies on the initial vector search ranking, which might not perfectly align with the nuances of the user query. Simply adding more documents (Option D) could dilute the relevant context with less relevant information, potentially hindering the LLM's ability to generate accurate and coherent responses. Modifying the prompt to instruct the LLM to reorder the documents based on relevance scores (Option B) is unlikely to be effective, as LLMs are primarily designed for text generation and comprehension, not precise numerical sorting. While a different embedding model (Option C) might improve the initial retrieval, it doesn't address the core issue that some lower-ranked documents are already identified as more contextually relevant.

Option A, using a reranker, directly addresses the problem. A reranker is a model specifically designed to re-order a set of retrieved documents based on a more sophisticated understanding of relevance, often using techniques like cross-encoders or transformer-based models to compare the query and document content. This allows the team to leverage the initial vector search for its speed and broad recall, then refine the results with a reranker that focuses on maximizing contextual relevance. By re-ordering the retrieved documents based on a more accurate relevance assessment, the team can ensure that the most contextually relevant documents are included in the prompt, even if they were initially ranked lower by the vector search. This approach improves the quality of the context provided to the LLM, leading to better RAG performance. Reranking is a common and effective technique in RAG pipelines to enhance the relevance and quality of retrieved context.

Refer to these resources for more information:

**Reranking in RAG:** <https://www.pinecone.io/learn/rerankers/>

**Cohere Rerank Endpoint:** <https://docs.cohere.com/reference/rerank>

**Sentence Transformers Reranking:** <https://www.sbert.net/examples/applications/reranking/README.html>

## Question: 92

A generative AI engineer is deploying an AI agent authored with MLflow's ChatAgent interface for a retail company's customer support system on Databricks. The agent must handle thousands of inquiries daily, and the engineer needs to track its performance and quality in real-time to ensure it meets service-level agreements.

Which metrics are automatically captured by default and made available for monitoring when the agent is deployed using the Mosaic AI Agent Framework?

- A. Operational metrics like request volume, latency, and errors
- B. Quality metrics like correctness and guideline adherence
- C. Both operational and quality metrics
- D. No metrics are automatically captured

**Answer: A**

### Explanation:

The correct answer is A. Operational metrics like request volume, latency, and errors. Here's why:

When deploying a ChatAgent using the Mosaic AI Agent Framework on Databricks, certain operational metrics are automatically logged by default, providing valuable insights into the agent's performance. This is a crucial aspect of maintaining the health and reliability of a high-volume application like a customer support system.

Specifically, the framework provides monitoring capabilities by automatically capturing metrics such as the number of requests received, the time taken to process each request (latency), and any errors encountered during processing. This data helps engineers quickly identify and address any performance bottlenecks or issues arising in the system.

This automatic tracking is achieved using Databricks' robust monitoring and logging infrastructure, which is designed to seamlessly integrate with MLflow. It's also an important aspect of observability and can be enhanced with Databricks monitoring features.

While quality metrics like correctness and adherence to guidelines are critical for evaluating the effectiveness of a Generative AI agent, these are often more complex to capture automatically. These metrics usually require custom logging and evaluation pipelines or integrating other tools. The Mosaic AI Agent Framework provides the tools for custom logging but it doesn't implement them automatically. This means implementing custom logic to perform evaluations and log the respective metrics.

Therefore, without explicit configuration and custom logging to track quality metrics, they will not be available out-of-the-box. The system provides infrastructure that allows for custom logging but it is not enabled by default.

Therefore, while essential for evaluating agent performance, quality metrics like correctness and guideline adherence are not automatically captured by the Mosaic AI Agent Framework by default. Instead, the focus is on fundamental operational characteristics.

Here are some resources for further research:

**MLflow ChatAgent:** <https://mlflow.org/docs/latest/llms/llm-evaluate/index.html>

**Databricks Monitoring:** <https://docs.databricks.com/en/administration-guide/index.html#monitoring>

## Question: 93

A Generative AI Engineer is developing an internal Retrieval-Augmented Generation (RAG) chatbot for the support team to query product documentation. The engineer recognizes the need for continuous monitoring of response quality once the system is in production. This quality monitoring will involve analyzing response text, model inputs, and the parameters used to generate responses.

Which approach would most effectively enable the engineer to monitor the chatbot's quality?

- A. Enable inference tables and display key metrics from the logged request/response pairs in a dashboard.
- B. Implement logging of request/response pairs within the model and ingest the logs from stdout into a table that can be visualized in a dashboard.
- C. Develop a proxy service that logs requests to the model endpoint and displays this data in a dashboard.
- D. Regularly survey users about their experience with the chatbot to assess its quality.

**Answer: A**

**Explanation:**

**Correct Answer**

Option A: Enable inference tables and display key metrics from the logged request/response pairs in a dashboard.

Option A is the most effective approach because inference tables provide comprehensive automated logging of all request/response pairs. This includes built-in infrastructure for capturing model inputs, outputs, and parameters.

It enables real-time monitoring capabilities through dashboard integration and provides a structured data format suitable for quality analysis with minimal additional development overhead.

This approach enables continuous, automated monitoring of response quality while capturing all the technical details necessary for a RAG system, making it the optimal solution for production quality monitoring.

**Analysis of Each Option**

Option A: This option is correct because it provides comprehensive logging of inputs, outputs, and parameters. It enables automated monitoring through built-in inference logging capabilities and allows for real-time quality metrics and analysis. It captures all necessary data for RAG system monitoring (queries, retrieved documents, generated responses, model parameters) and integrates seamlessly with existing ML infrastructure.

Option B: This option is incorrect because it requires custom implementation and maintenance. stdout logging is less reliable and harder to structure, requiring a more complex data pipeline setup. It also has the potential to impact the model's performance.

Option C: This option is incorrect because it only captures requests, missing response quality data. It doesn't log internal RAG processes (document retrieval, generation parameters), providing an incomplete picture for quality monitoring and adding additional infrastructure complexity.

Option D: This option is incorrect because it provides valuable user feedback but lacks technical depth. It is not automated or continuous and doesn't capture technical metrics needed for system optimization. The data is subjective and potentially biased.

**Key Takeaways**

The key takeaway is that effective monitoring of a RAG chatbot requires comprehensive, automated logging of requests, responses, and internal parameters.

A common mistake is relying solely on user feedback without capturing technical metrics.

When approaching this type of problem, prioritize solutions that offer automated, continuous monitoring with

### Question: 94

A Generative AI Engineer is working on a RAG application for an architecture firm that has many similarly structured building plan documents. They find that architects are asking questions about a specific building, but the system is currently providing responses about the wrong buildings. They have a list of approaches that may resolve the issue but have limited time and must immediately discard any that will not be effective.

Which approach do they discard?

- A. Fine tune the response generating LLM on thousands of similar documents
- B. Increase number of retrieved documents and add a reranker
- C. Use hybrid search to include building metadata like site name
- D. Fine tune the embedding model on thousands of similar documents

**Answer: A**

**Explanation:**

Correct Answer

**Option A: Fine tune the response generating LLM on thousands of similar documents**

The problem is that the RAG application is retrieving information about the wrong buildings. Fine-tuning the response generating LLM focuses on improving the quality of the generated response, but it doesn't address the underlying issue of retrieving the correct documents in the first place. Even if the LLM is better at generating responses, it will still generate responses about the wrong building if the wrong documents are retrieved.

The core issue is retrieval accuracy, and fine-tuning the LLM doesn't directly impact the retrieval process.

**Analysis of Each Option**

Option A: This option is correct. Fine-tuning the response generating LLM does not address the retrieval problem and should be discarded as it won't resolve the issue of retrieving documents about wrong buildings.

Option B: This option is incorrect. Increasing the number of retrieved documents and adding a reranker can improve retrieval accuracy by casting a wider net and then selecting the most relevant documents.

Option C: This option is incorrect. Using hybrid search to include building metadata like site name can help disambiguate between buildings by incorporating explicit building identification signals.

Option D: This option is incorrect. Fine-tuning the embedding model can improve the semantic understanding during retrieval and help distinguish between different buildings in the embedding space.

**Key Takeaways**

The key takeaway is to focus on the root cause of the problem. In this case, the problem is retrieval accuracy, so the solution should address the retrieval process directly.

A common mistake is to focus on improving the generation phase when the problem lies in the retrieval phase.

When troubleshooting RAG applications, it's important to identify the specific stage where the problem occurs (retrieval or generation) and focus on solutions that address that stage.

**Correct Answer**

### Question: 95

A Generative AI Engineer has deployed a RAG application that processes technical documentation - including API specs, troubleshooting guides, and product manuals - for a customer support system. The application currently uses uniform chunks of 512 tokens with 50 -token overlap. Users report receiving fragmented responses that miss critical context. They need to choose an approach that can address this issue while maintaining query response time. Which approach do they choose?

- A. Implement semantic chunking with windowed summarization.
- B. Implement hierarchical chunking based on document headers.
- C. Increase the fixed chunk size to 1024 tokens.
- D. Add more overlap between existing fixed-size chunks.

**Answer: B**

**Explanation:**

**Correct Answer**

**Option B: Implement hierarchical chunking based on document headers**

**Explanation of the Correct Answer**

The problem is that the current fixed-size chunking method (512 tokens with 50-token overlap) leads to fragmented responses that miss critical context. The goal is to find a chunking approach that addresses this issue while maintaining query response time.

Option B, hierarchical chunking based on document headers, is the best approach because it leverages the natural structure of technical documentation (headers, sections, subsections) to preserve logical relationships and maintain context within semantic boundaries. This method doesn't add computational overhead during retrieval, making it suitable for structured technical documents while maintaining query response time.

**Analysis of Each Option**

Option A: Semantic chunking with windowed summarization may improve context but likely increases response time due to summarization processing, conflicting with the response time constraint. Therefore, it is incorrect.

Option B: Hierarchical chunking based on document headers leverages the natural document structure, preserves logical relationships within technical documentation, maintains context within semantic boundaries, and has no additional computational overhead during retrieval. Therefore, it is correct.

Option C: Increasing the fixed chunk size to 1024 tokens may capture more context per chunk but still suffers from arbitrary boundary cuts and doesn't address the fundamental issue of semantic fragmentation. Therefore, it is incorrect.

Option D: Adding more overlap between chunks increases redundancy and storage requirements, may help with boundary issues but doesn't solve semantic fragmentation, and could introduce noise in retrieval results. Therefore, it is incorrect.

**Key Takeaways**

Fixed-size chunking can lead to semantic fragmentation, especially with structured documents.

Hierarchical chunking is effective for preserving context in documents with a clear hierarchical structure.

Consider the trade-offs between context preservation and computational overhead when choosing a chunking strategy.

### Question: 96

CertyIQ

A Generative AI Engineer is part of a team developing an AI-powered web analyst advisor agent. This agent will provide personalized guidance and recommendations to web development teams on how to optimize their content for better engagement and monetization. It's crucial that system be flexible and adapt its approach based on various contexts and tasks, without relying on a static repository of prompts. To fulfill these requirements for dynamic adaptation and context-dependent prompting, they need to select the correct framework that can support this. Which framework do they select?

- A. DSPy
- B. LangChain
- C. LlamaIndex
- D. Haystack

**Answer: A**

**Explanation:**

Correct Answer

**Option A: DSPy**

**Explanation of the Correct Answer**

DSPy (Declarative Self-improving Python) is the correct answer because it is specifically designed for dynamic prompt optimization. It uses automatic prompt engineering and adapts prompts based on context and performance feedback, without relying on static prompt repositories. This aligns perfectly with the requirements of the AI-powered web analyst advisor agent, which needs to provide personalized guidance and recommendations based on various contexts and tasks.

The key concept here is dynamic prompt optimization, where the system learns and adapts its prompts based on the situation, rather than relying on pre-defined prompts.

**Analysis of Each Option**

Option A: DSPy is correct because it is designed for dynamic prompt optimization and context-dependent adaptation, automatically generating and adapting prompts based on context and task requirements.

Option B: LangChain is incorrect because it primarily focuses on chaining LLM operations and relies heavily on static prompt repositories, making it less suitable for dynamic adaptation.

Option C: LlamaIndex is incorrect because it specializes in data indexing and retrieval and uses relatively static prompting approaches, not designed for dynamic prompt optimization.

Option D: Haystack is incorrect because it is designed for search and question-answering pipelines and has limited dynamic prompting capabilities, focusing more on document processing.

**Key Takeaways**

The key takeaway is understanding the difference between static and dynamic prompting. Static prompting uses pre-defined prompts, while dynamic prompting involves generating and adapting prompts based on context and feedback.

A common mistake is to choose a framework based on its general popularity or versatility, rather than its specific capabilities for dynamic prompt optimization.

When approaching this type of problem, focus on identifying the core requirements (dynamic adaptation, context-dependent prompting) and then evaluating each framework based on its ability to meet those requirements.

Correct Answer

Option A: DSPy

DSPy is the correct framework because it is specifically designed for dynamic prompt optimization and context-dependent adaptation, making it ideal for an AI agent that needs to provide personalized, context-aware recommendations without being constrained by pre-written prompt repositories

### Question: 97

CertyIQ

A Generative AI Engineer is integrating Mosaic AI Vector Search into a Retrieval-Augmented Generation (RAG) system. The source data, comprising simple text entries, is stored in a Delta table. To simplify the workflow, the engineer plans to use an embedding model served via Mosaic AI Model Serving endpoint to automatically compute embeddings during data synchronization from the Delta table to the vector search index. Which method should the engineer use to achieve this integration?

- A. Delta sync index with managed embeddings
- B. Delta sync index with self-managed embeddings
- C. Direct vector access index
- D. Hybrid embedding computing

**Answer: A**

**Explanation:**

Correct Answer

**Option A: Delta sync index with managed embeddings**

**Explanation of the Correct Answer**

The correct option is Delta sync index with managed embeddings. This is because the engineer wants to automatically compute embeddings during data synchronization from the Delta table to the vector search index using an embedding model served via Mosaic AI Model Serving endpoint. Managed embeddings allow the vector search service to handle embedding computation automatically, which simplifies the workflow.

The key concept here is that "managed embeddings" in Mosaic AI Vector Search can automatically call external embedding models, such as those served via Model Serving endpoints, during data synchronization from a Delta table.

**Analysis of Each Option**

Option A: Delta sync index with managed embeddings - This is the correct option because it allows for automatic synchronization from Delta tables and automatic embedding computation using the Model Serving endpoint, simplifying the workflow.

Option B: Delta sync index with self-managed embeddings - This option is incorrect because it requires manual embedding computation before storing data in the Delta table, which does not achieve automatic computation during synchronization and results in a more complex workflow.



Option C: Direct vector access index - This option is incorrect because it requires pre-computed embeddings to be directly inserted and does not offer automatic synchronization from Delta tables.

Option D: Hybrid embedding computing - This option is incorrect because it is not a standard Mosaic AI Vector Search method and does not specifically address Delta table synchronization.

### Key Takeaways

The key takeaway is that "managed embeddings" in Mosaic AI Vector Search can automate the embedding computation process during data synchronization, especially when using models served via Model Serving endpoints.

A common mistake is to overlook the "automatic computation" requirement and choose an option that requires manual embedding management.

When faced with similar questions, focus on identifying the method that provides the most automation and simplification of the workflow, especially concerning embedding computation and data synchronization.

### Question: 98

CertyIQ

A Generative AI Engineer has been asked to create an AI system that can answer questions of a company's PDF knowledge base, but many of the PDFs have diagrams in them. The engineer is considering two approaches:

- Using an image captioning model to generate text descriptions of diagrams and embedding those alongside the text components of the PDFs
- Using a multimodal model like CLIP to embed the diagrams and text components of the PDFs to a shared vector space

The engineer needs to evaluate the tradeoffs of these two model options to choose their approach. Which statement correctly reflects the tradeoffs?

- A. Option 1 is simpler due to use of standard text pipelines, but is strictly reliant on the caption accuracy Option 2 is more flexible in query modality, but can be more expensive due to infrastructure requirements
- B. Option 1 is natively supported via Delta Sync API, but the accuracy will be harder to maintain due to multiple embeddings Option 2 is simpler to maintain due to unified embedding model, but isn't natively supported by Delta Sync API
- C. Option 1 is more accurate due to the strength of text embedding models, but the multi-step embedding process isn't natively supported by Delta Sync API Option 2 has a singular embedding model and is therefore natively supported by Delta Sync API, but multimodal embedding models are weaker for retrieval
- D. Option 1 is higher quality due to the relative performance of text-only embedding models, but is more expensive due to the multiple embedding models Option 2 is cheaper due to the single embedding model, but won't perform as well in multimodal settings because of the limited visual information Embedded

**Answer: A**

**Explanation:**

Correct Answer

**Option A is correct: Option 1 is simpler due to use of standard text pipelines, but is strictly reliant on the caption accuracy Option 2 is more flexible in query modality, but can be more expensive due to infrastructure requirements**

### Explanation of the Correct Answer

Option A is correct because it accurately describes the trade-offs between the two approaches. Option 1 is simpler because it uses standard text pipelines, but its accuracy depends on the quality of the image captions. Option 2 is more flexible because it can handle different query types, but it can be more expensive

due to infrastructure requirements.

The thought process correctly identifies that Option 1 relies on the accuracy of the image captioning model, while Option 2 requires more computational resources.

### Analysis of Each Option

Option A: This option is correct. It accurately describes the trade-offs between the two approaches.

Option B: This option is incorrect. It mentions Delta Sync API, which is not relevant to the core trade-offs.

Option C: This option is incorrect. It makes inaccurate claims about the relative performance of text and multimodal embedding models.

Option D: This option is incorrect. It makes inaccurate claims about the relative performance of text and multimodal embedding models and the cost of the two approaches.

### Key Takeaways

When evaluating different approaches for handling PDFs with diagrams, it is important to consider the trade-offs between simplicity, accuracy, flexibility, and cost.

Image captioning models can be useful for generating text descriptions of diagrams, but their accuracy is critical.

Multimodal models can be more flexible, but they may require more computational resources.

## Question: 99

CertyIQ

Databricks offers a number of built in AI judges that provide metrics and rationale for different types of quality issues a Gen AI application may have.

Which of the following pairs of judges both require a ground Truth label in the evaluation dataset field expected\_response to execute?

- A. correctness, groundedness
- B. guideline\_adherence, chunk\_relevance
- C. relevance\_to\_query, chunk\_relevance
- D. context\_sufficiency, correctness

**Answer: D**

**Explanation:**

Correct Answer

**Option D: context\_sufficiency, correctness**

### Explanation of the Correct Answer

The question asks which pair of AI judges in Databricks requires a ground truth label in the expected\_response field of the evaluation dataset.

The thought process identifies that the Correctness Judge requires ground truth because it compares the model's response to the expected answer.

The thought process also identifies that the Context Sufficiency Judge requires ground truth because it determines if the provided context contains enough information to generate the expected response.

Therefore, the correct answer is the pair containing Correctness and Context Sufficiency.

### Analysis of Each Option

Option A: correctness, groundedness - Correctness requires ground truth, but Groundedness does not. Therefore, this option is incorrect.

Option B: guideline\_adherence, chunk\_relevance - Neither Guideline Adherence nor Chunk Relevance requires ground truth. Therefore, this option is incorrect.

Option C: relevance\_to\_query, chunk\_relevance - Neither Relevance to Query nor Chunk Relevance requires ground truth. Therefore, this option is incorrect.

Option D: context\_sufficiency, correctness - Both Context Sufficiency and Correctness require ground truth. Therefore, this option is correct.

### Key Takeaways

Understanding the purpose of each AI judge is crucial for determining whether it requires a ground truth label.

Judges that evaluate the accuracy or completeness of a response typically require ground truth for comparison.

Judges that assess relevance or adherence to guidelines may not require ground truth.

When answering this type of question, carefully consider the specific requirements of each judge mentioned in the options

## Question: 100

CertyIQ

A company receives product reviews from their online store and from social media. The reviews come in many formats. The company needs to understand which product is being referenced in each review to improve their customer service. One of their data scientists designs an offline batch application using GenAI that will extract all product references from each review.

Which component is least useful for the application?

- A. Vector search
- B. Few-shot examples
- C. Return schema
- D. Prompt template

**Answer: A**

**Explanation:**

Correct Answer

**Option A: Vector search**

**Explanation of the Correct Answer**

Vector search is less critical in an offline batch application because the application can rely on direct text matching, predefined product catalogs, and rule-based extraction methods. The absence of real-time constraints allows for more deliberate processing, reducing the necessity for rapid similarity searches.

The core task of extracting product references from reviews in a batch processing context can be achieved without vector search, making it the least essential component.

## Analysis of Each Option

Option A: Vector search is less critical because the application processes reviews in batches without real-time constraints, allowing it to rely on direct text matching and predefined product catalogs. Therefore, this option is correct.

Option B: Few-shot examples are useful for teaching the GenAI model the expected format for product extraction, handling different review formats, and improving extraction accuracy. Therefore, this option is incorrect.

Option C: A return schema is essential for ensuring a consistent output format, structuring extracted product information, and enabling reliable downstream processing. Therefore, this option is incorrect.

Option D: A prompt template is crucial for providing consistent instructions to the GenAI model, defining the extraction task clearly, and ensuring reproducible results. Therefore, this option is incorrect.

## Key Takeaways

Vector search is most useful in real-time applications where speed and similarity matching are crucial.

In offline batch applications, simpler methods like direct text matching and rule-based extraction can often suffice.

Understanding the constraints and requirements of the application (real-time vs. batch) is essential for choosing the right components.

## Question: 101

CertyIQ

A company has about one million IT tickets in plain-text saved to a table in Unity Catalog. They want to both analyze the resolution and root cause analysis of the issue as well as extract various components from the text such as the ticket subject, time to resolution, and assigned engineer, using Llama 3.370 B or a similar sized model, and then they want to save those extracted fields to their own columns in the same SQL table for further downstream analytics.

Which of the following strategies will be the most cost and time effective assuming about 250 tokens per ticket?

- A. `ai_query()` calling a Provisioned Throughput endpoint
- B. `ai_query()` calling a Pay-per-Token endpoint
- C. `ai_query()` calling a custom GPU model serving endpoint
- D. Databricks notebook with the model loaded directly on a 8xA10 cluster

## Answer: B

### Explanation:

Correct Answer

### Option B: `ai_query()` calling a Pay-per-Token endpoint

### Explanation of the Correct Answer

The scenario involves processing a large volume (1 million) of IT tickets using a large language model (Llama 3.3 70B). The goal is to extract information and save it back to a SQL table in Unity Catalog in a cost and time-effective manner.

Option B, using `ai_query()` with a Pay-per-Token endpoint, is the most suitable because it charges only for the tokens processed. This avoids the costs associated with reserving capacity (Option A) or managing infrastructure (Options C and D).

The thought process correctly identifies that this is a batch processing job, not a real-time inference scenario, making pay-per-token the most economical choice.

The calculation of 250 million tokens (1 million tickets \* 250 tokens/ticket) highlights the scale of the task and reinforces the need for a cost-effective solution.

### Analysis of Each Option

Option A: Incorrect. Provisioned Throughput is more suitable for sustained, predictable workloads, not for a one-time or infrequent batch job. It would lead to unnecessary costs for unused capacity.

Option B: Correct. Pay-per-Token is ideal for batch processing because you only pay for what you use. It also integrates well with Unity Catalog via `ai_query()` and scales automatically.

Option C: Incorrect. Using a custom GPU model serving endpoint adds complexity and infrastructure management overhead, making it less cost and time-effective than pay-per-token.

Option D: Incorrect. Loading the model directly onto a Databricks cluster requires significant RAM and incurs costs for the entire cluster duration, including model loading time. An 8xA10 cluster is also expensive to run.

### Key Takeaways

When dealing with large-scale batch processing tasks involving LLMs, pay-per-token pricing models are often the most cost-effective.

Consider the overhead costs associated with infrastructure management and model deployment when choosing between different options.

`ai_query()` provides a seamless integration with Unity Catalog, simplifying the process of querying LLMs and saving the results back to SQL tables.

Avoid over-provisioning resources for infrequent tasks.

## Question: 102

CertyIQ

Generative AI guardrails can be implemented inside-the-model (during training or fine-tuning), or at inference time, either on input tokens or output tokens. Where does Mosaic AI Gateway implement guardrails and what is a benefit to this type of implementation?

- A. At inference time, Faster Iteration: implementing on input or output tokens requires no model retraining
- B. At inference time, Decreased Latency: by moving the guardrail logic outside of the model and into a separate guardrail call the over all latency is always reduced.
- C. Inside-the-model, Efficiency: inside-the-model guardrails reduces dependency on additional layers of processing during inference lowering latency
- D. Inside-the-model, Ease of deployment: inside-the-model guardrails operate independently across different applications without the need for external guardrails

**Answer: A**

**Explanation:**

**A. At inference time, Faster Iteration: implementing on input or output tokens requires no model retraining**  
**Correct.**

Mosaic AI Gateway applies **guardrails at inference time on input and output tokens**. This approach allows organizations to **modify or update guardrails without retraining the model**, enabling **faster iteration and easier updates**.

**B. At inference time, Decreased Latency    Incorrect.**

Moving guardrails outside the model does **not always reduce latency**. External checks can sometimes **add processing time**.

**C. Inside-the-model, Efficiency    Incorrect.**

Mosaic AI Gateway does **not implement guardrails inside the model**; it applies them during inference.

**D. Inside-the-model, Ease of deployment    Incorrect.**

This option is incorrect because Mosaic AI Gateway does **not use inside-the-model guardrails**.

**Question: 103**

**CertyIQ**

A Generative AI Engineer has written the following code snippet in the driver notebook using the Mosaic AI Agent Framework:  However, the resources are not being logged. Which aspect of their code is causing this?

- A. The code is missing the required resources parameter to specify dependencies like vector search index and serving endpoints
- B. There is no issue with their code; PyFunc automatically detects resources like LangChain
- C. Resources cannot be logged with PyFunc models
- D. The issue is in the artifact\_path parameter which should contain resource information

**Answer: A**

**Explanation:**

**A. The code is missing the required resources parameter to specify dependencies like vector search index and serving endpoints    Correct.**

When logging a model using **MLflow PyFunc** in the Mosaic AI Agent Framework, resources such as **vector search indexes or model serving endpoints** must be explicitly defined using the **resources parameter**. Since the code snippet does not include this parameter, the resources are not being logged.

**B. There is no issue with their code; PyFunc automatically detects resources like LangChain    Incorrect.**

PyFunc does **not automatically detect external resources**. These dependencies must be **explicitly declared**.

**C. Resources cannot be logged with PyFunc models    Incorrect.**

Resources **can be logged**, but they must be specified using the **resources parameter**.

**D. The issue is in the artifact\_path parameter which should contain resource information    Incorrect.**

artifact\_path only specifies **where the model artifacts are stored**, not the resources used by the model.

**Question: 104**

**CertyIQ**

A Generative AI Engineer processes PDF files by parsing the text, dividing it into chunks, and storing the computed embeddings in a Delta table designated as the source table. However, when attempting to create a Mosaic AI Vector Index based on this source table, the index creation fails. They need to consider the most likely cause for the failure. Which is the most likely cause?

- A. Change Data Feed (CDF) is not enabled for the source table.
- B. The sync mode is set to "Triggered."
- C. The embedding column to be synced must be a string instead of precomputed embeddings.
- D. The embedding model to be used is not defined.

**Answer: A**

**Explanation:**

**A. Change Data Feed (CDF) is not enabled for the source table      Correct.**

When creating a **Mosaic AI Vector Index** from a Delta table, **Change Data Feed (CDF)** must be enabled on the source table so the index can track and sync changes in the data. If CDF is not enabled, the vector index creation can fail.

**B. The sync mode is set to "Triggered."      Incorrect.**

Triggered sync mode is a **valid configuration** and does not cause index creation to fail.

**C. The embedding column to be synced must be a string instead of precomputed embeddings      Incorrect.**

Vector indexes are designed to store **precomputed embeddings**, not strings.

**D. The embedding model to be used is not defined      Incorrect.**

If embeddings are **already precomputed and stored in the table**, defining an embedding model is not required.

## Question: 105

CertyIQ

A Data Engineer is configuring a Model Serving Endpoint that the Data Science team will use to interact with an external, proprietary Large Language Model (LLM). The Data Engineer wants to implement a mechanism to control costs associated with using the external model. Which mechanism would most effectively control costs using Databricks-native features?

- A. Add a limit to the number of output tokens returned by the LLM
- B. Define and enforce an appropriate rate limit, measured by queries per minute, for the data science group's usage of the model serving endpoint
- C. Define a monthly budget for the endpoint and disable the endpoint when the budget is exceeded
- D. Turn off the model serving endpoint on nights and weekends when it's not in use

**Answer: B**

**Explanation:**

**A. Add a limit to the number of output tokens returned by the LLM      Incorrect.**

Limiting output tokens can reduce the size of responses, but it does **not effectively control overall usage frequency**, which is the main driver of cost.

**B. Define and enforce an appropriate rate limit, measured by queries per minute, for the data science group's usage of the model serving endpoint      Correct.**

Databricks **Model Serving Endpoints support rate limiting**, which allows administrators to **control how many requests can be sent per minute**. This is an effective native mechanism to **control costs when accessing external LLMs**.



**C. Define a monthly budget for the endpoint and disable the endpoint when the budget is exceeded**  
**Incorrect.**

Databricks does not provide a **native automatic budget-based shutdown feature** for model serving endpoints.

**D. Turn off the model serving endpoint on nights and weekends when it's not in use**      **Incorrect.**

Manually turning off endpoints may reduce usage but is **not an automated or practical cost control mechanism**.

### Question: 106

CertyIQ

Which of the following statements accurately identifies differences between the evaluation phase and the monitoring phase in the generative AI application lifecycle within Databricks?

- A. The evaluation phase uses all inference history to assess agent performance and readiness for production, while in the monitoring phase uses only new inference table records to monitor performance.
- B. The evaluation phase uses Mosaic AI Agent Evaluation and an evaluation dataset to assess an agent's performance metrics and traces, while the monitoring phase relies on inference tables as source data for creating a metrics profile.
- C. The evaluation phase ensures the agent's responses comply with business rules in production, whereas the monitoring phase is focused on SLA and performance metrics.
- D. The evaluation phase logs and traces live API calls in production, while the monitoring phase runs metrics on inference tables containing those traces.

**Answer: B**

**Explanation:**

**A. The evaluation phase uses all inference history to assess agent performance and readiness for production, while in the monitoring phase uses only new inference table records to monitor performance**  
**Incorrect.**

Evaluation does not rely on inference history; it typically uses **prepared evaluation datasets**, while monitoring analyzes **production inference data**.

**B. The evaluation phase uses Mosaic AI Agent Evaluation and an evaluation dataset to assess an agent's performance metrics and traces, while the monitoring phase relies on inference tables as source data for creating a metrics profile**      **Correct.**

In Databricks, the **evaluation phase** uses **Mosaic AI Agent Evaluation with evaluation datasets** to measure metrics and analyze traces before deployment. The **monitoring phase** analyzes **production inference tables** to track real-world performance and behavior.

**C. The evaluation phase ensures the agent's responses comply with business rules in production, whereas the monitoring phase is focused on SLA and performance metrics**      **Incorrect.**

Evaluation occurs **before production** and is not responsible for enforcing production rules.

**D. The evaluation phase logs and traces live API calls in production, while the monitoring phase runs metrics on inference tables containing those traces**      **Incorrect.**

Logging and tracing of **live production calls** belong to the **monitoring phase**, not the evaluation phase.

### Question: 107

CertyIQ

A Generative AI Engineer is developing a large-scale RAG application that requires processing millions of HTML documents from a company's website. The document contains product descriptions with nested sections, pricing tables, and user reviews. The system needs to:

- Support semantic search across product features
- Preserve price comparison tables
- Handle frequent document updates
- Process 100,000 documents per hour
- Maintain contextual relationships between product descriptions and reviews

They need to select a chunking strategy that would optimize both search relevance and processing speed. Which strategy do they select?

- A. Process the HTML using recursive tag-based chunking that splits on semantic elements, with separate embedding pipelines for tabular and textual content.
- B. Convert HTML to structured JSON objects, chunk based on logical product sections, and maintain cross-references between related chunks using metadata.
- C. Use a Recursive TextSplitter with max chunk size of 512 tokens.
- D. Use an LLM to analyze and summarize each HTML section before chunking, storing both the summary and original content.

**Answer: A**

**Explanation:**

**A. Process the HTML using recursive tag-based chunking that splits on semantic elements, with separate embedding pipelines for tabular and textual content**     **Correct.**

This approach **preserves the structure of HTML documents** by splitting based on semantic tags (such as sections, tables, and reviews). It allows **tables (pricing comparisons) to be processed differently from textual content**, improving search relevance. It also supports **high processing throughput** and maintains context relationships between product descriptions and reviews.

**B. Convert HTML to structured JSON objects, chunk based on logical product sections, and maintain cross-references between related chunks using metadata**     **Incorrect.**

Although this preserves structure, converting millions of documents into structured JSON and maintaining cross-references adds **significant processing overhead**, making it less suitable for **100,000 documents per hour**.

**C. Use a Recursive TextSplitter with max chunk size of 512 tokens**     **Incorrect.**

A generic token-based splitter ignores **HTML structure and tables**, which could break pricing tables and lose contextual relationships.

**D. Use an LLM to analyze and summarize each HTML section before chunking**     **Incorrect.**

Using an LLM for summarization before chunking would be **computationally expensive and slow**, making it impractical for **large-scale processing at 100,000 documents per hour**.

### Question: 108

CertyIQ

A Generative AI Engineer is working on a RAG application that is heavily based on data scraped from the web and they're encountering two main problems:

- the data is taking up much more space in the vector database than expected
- performance has been much worse than expected.

Which cause explains both issues?

- A. The scraped data is in HTML format and should be converted to plain text before chunking and embedding
- B. The chunk size and chunking overlap window are too small resulting in lost context between chunks
- C. The embedding model dimension is too large for the application and a smaller model would be more efficient
- D. The engineer is using a summarizer on too small of chunks, resulting in longer summaries than the source material

**Answer: C**

**Explanation:**

**A. The scraped data is in HTML format and should be converted to plain text before chunking and embedding    Incorrect.**

While HTML can add noise, it does not typically cause **significant vector database size increases** or major performance issues compared to other factors.

**B. The chunk size and chunking overlap window are too small resulting in lost context between chunks    Incorrect.**

Small chunks may reduce context quality, but they **do not directly explain excessive storage usage** in the vector database.

**C. The embedding model dimension is too large for the application and a smaller model would be more efficient    Correct.**

If the embedding model has **very high dimensional vectors**, each stored embedding requires **more storage space**, increasing vector database size. Higher-dimensional vectors also **slow similarity searches**, leading to reduced performance.

**D. The engineer is using a summarizer on too small of chunks, resulting in longer summaries than the source material    Incorrect.**

This would increase processing time but is **unlikely to be the primary cause of vector database storage size and search performance issues**.

### Question: 109

CertyIQ

A Generative AI Engineer is developing a real time customer support bot that can answer questions about new product features, troubleshooting issues, and inventory or future stocking questions. Which set of tools best allows the bot to address these requirements?

- A. A RAG agent for the product features and troubleshooting information, plus a Text-to-SQL agent for querying availability from the inventory and stocking database
- B. Few shot learning for the product features and troubleshooting information, plus a Feature Store for retrieving availability from the inventory and stocking database
- C. A retrieval agent for the product features, few shot learning for the troubleshooting information, plus Online Tables for retrieving availability from the inventory and stocking database
- D. A fine tuned LLM for the product features and troubleshooting information, plus a SQL agent for querying availability from the inventory and stocking database

**Answer: A**

**Explanation:**

**A. A RAG agent for the product features and troubleshooting information, plus a Text-to-SQL agent for querying availability from the inventory and stocking database    Correct.**

A **RAG (Retrieval-Augmented Generation) agent** is ideal for answering questions about **product features and troubleshooting**, since it retrieves up-to-date documentation or knowledge base content. A **Text-to-SQL agent** can dynamically query the **inventory and stocking database** to provide real-time availability information.

**B. Few shot learning for the product features and troubleshooting information, plus a Feature Store for retrieving availability from the inventory and stocking database**    **Incorrect.**

Few-shot learning is not reliable for **large knowledge bases**, and a **Feature Store** is designed for ML feature management, not operational inventory queries.

**C. A retrieval agent for the product features, few shot learning for the troubleshooting information, plus Online Tables for retrieving availability from the inventory and stocking database**    **Incorrect.**

Few-shot learning is not suitable for complex troubleshooting knowledge, and **Online Tables** are not a standard approach for querying operational inventory systems.

**D. A fine tuned LLM for the product features and troubleshooting information, plus a SQL agent for querying availability from the inventory and stocking database**    **Incorrect.**

Fine-tuning for constantly changing product documentation and troubleshooting data would require **frequent retraining**, making it less practical than RAG.

### Question: 110

CertyIQ

A company selling gourmet mushroom growing supplies has a script that runs once per day to scrape various social media platforms for posts that mention its name. The scraped text data is loaded into a Delta table each night for a downstream processing task that summarizes each post and its sentiment for internal use. Given the small size of the company, they only get a couple hundred posts per day. Which solution best optimizes for cost and ease of implementation?

- A. Schedule a nightly SQL query with `ai_query()` calling a Pay-Per-Token endpoint
- B. Schedule a nightly SQL query with `ai_query()` calling a Provisioned Throughput endpoint
- C. Schedule a nightly job that runs a notebook to download the model locally, process the records, and terminate the cluster
- D. Schedule a nightly job to call OpenAI's batch inference API, save the results, and terminate the cluster

**Answer: A**

**Explanation:**

**A. Schedule a nightly SQL query with `ai_query()` calling a Pay-Per-Token endpoint**    **Correct.**

Since the company processes **only a few hundred posts per day**, a **Pay-Per-Token endpoint** is the most cost-efficient option. Using `ai_query()` in a scheduled SQL query is also **simple to implement**, avoiding additional infrastructure or cluster management.

**B. Schedule a nightly SQL query with `ai_query()` calling a Provisioned Throughput endpoint**    **Incorrect.**

Provisioned throughput endpoints are designed for **high and consistent workloads**. For a few hundred posts per day, this would be **more expensive than necessary**.

**C. Schedule a nightly job that runs a notebook to download the model locally, process the records, and terminate the cluster**    **Incorrect.**

Downloading and running a model locally requires **cluster compute resources and more operational**

**complexity**, increasing cost and implementation effort.

**D. Schedule a nightly job to call OpenAI's batch inference API, save the results, and terminate the cluster**  
**Incorrect.**

This approach introduces **additional integration complexity** and cluster management, which is unnecessary for such a small workload.

# Thank you

Thank you for being so interested in the premium exam material.  
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.  
Your insights can help me improve our writing and better understand our readers.

## Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam  
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers



Future is Secured

100% Pass Guarantee



24/7 Customer Support

Mail us - [certyiqofficial@gmail.com](mailto:certyiqofficial@gmail.com)



Free Updates

Lifetime Free Updates!

Total: **110 Questions**

Link: <https://certyiq.com/papers/databricks/certified-generative-ai-engineer-associate>